Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version)

Ehud Shapiro

London School of Economics and Weizmann Institute of Science
e.shapiro2@lse.ac.uk

Abstract. Grassroots Logic Programs (GLP) is a concurrent logic programming language in which logic variables are partitioned into paired readers and writers. An assignment is produced at most once via a writer and consumed at most once via its paired reader, and may contain additional readers and/or writers. This enables the concise expression of rich multidirectional communication modalities.
The language was introduced together with concurrent (cGLP) and multiagent (maGLP) operational semantics. Here, we derive from these (*i*) dGLP, a deterministic counterpart of cGLP, and (*ii*) madGLP, a counterpart of maGLP in which deterministic agents communicate solely by asynchronous message passing, and prove them correct against their abstract counterparts. maGLP shared variable pairs spanning agents can be implemented by two local variable pairs joined by a *global link*, with correctness following from disjoint substitution commutativity (a consequence of GLP's single-occurrence invariant). We further prove that madGLP is grassroots. Both dGLP and madGLP serve as formal specifications for an AI-driven implementation discipline (math → informal spec → Dart) employed and described here: from dGLP, AI (Claude) developed a workstation-based GLP implementation in Dart, and from madGLP it is developing a smartphone-based multiagent one.

Keywords: concurrent logic programming · multiagent systems · transition systems · operational semantics · peer-to-peer · grassroots platforms

1 Introduction

Grassroots platforms. A digital platform is *grassroots* [51,54] if it can have multiple instances that can (*i*) operate independently of each other and of any global resource other than the network, and (*ii*) coalesce into ever larger instances, possibly resulting in a single global instance.
GLP. Grassroots Logic Programs (GLP) [53] is a concurrent logic programming language in which logic variables are partitioned into paired *readers* $X?$ and *writers* X, each occurring at most once (single-occurrence, SO), with a variable occurring in a clause iff its paired variable also does (single-reader single-writer, SRSW). GLP eschews unification in favour of simple term matching, is linear-logic-like [17], and is futures/promises-like [5,16]: each assignment $X := T$ is

produced at most once via the sole occurrence of a writer (promise) X, and consumed at most once via the sole occurrence of its paired reader (future) $X?$; the term T may contain further readers and/or writers, enabling the concise expression of rich multidirectional communication modalities.

Being relational and nondeterministic (and thus not functional), GLP—like its predecessor concurrent logic languages [48]—can express nondeterministic asynchronous concurrent processes such as the fair merging of streams. The SRSW restriction is what makes GLP suitable for grassroots multiagent distributed execution: each variable assignment has exactly one producer and one consumer, so an assignment can be communicated asynchronously from its writer to its reader without consensus. The intended deployment of GLP is grassroots systems consisting of people operating their smartphones.

From specifications to implementations. Prior work has defined concurrent and multiagent nondeterministic operational semantics for GLP, cGLP and maGLP [53], via transition systems and multiagent transition systems [49,29], and a type and module system for GLP [55]. Here, we derive from cGLP and maGLP deterministic counterparts dGLP and madGLP, and prove each of them correct with respect to its abstract counterpart, madGLP for *authorised runs* (Section 6.7).

Implementing a shared variable, by example. A maGLP shared pair couples a writer at one agent with its reader at another—say writer X at p, reader $X?$ at q, so values flow $p \rightarrow q$. madGLP realises this with two local pairs, $(X_p, X_p?)$ at p and $(X_q, X_q?)$ at q, joined by a *global link*: a spawned goal at p suspends on $X_p?$ and, when X_p is assigned, sends the value to q, which assigns X_q. The maGLP writer and reader are the outer ends (X_p, $X_q?$); the link bridges the inner ends ($X_p?$, X_q). Each shared variable is realised by two local pairs, the reader of one linked to the writer of the other.

AI development methodology. The AI-based development employs three layers of abstraction—math, an English+code informal specification, and Dart code. Authority flows math→spec→Dart but harmonised via back-and-forth in which running the implementation and deriving the specification surfaced defects, also at the mathematical level, prompting major redesigns of madGLP. madGLP and its single-agent specialisation dGLP serve as the formal specifications: from dGLP, AI (Claude) developed a workstation-based GLP implementation in Dart [18]; from madGLP, it is developing a smartphone-based multiagent one. See Section 7.

Related work. GLP belongs to the family of concurrent logic languages [48,47,60,10,39]; it can be understood as Flat Concurrent Prolog with SRSW added, simplifying read-only unification [27], with strong connections to mode and linearity analysis [62,61], linear logic and session types [17,9,66,23], and futures and promises [5,44]. Our correctness framework draws on I/O automata [37], refinement mappings [1], and atomic-transaction theory [32,20], using disjoint substitution commutativity (a consequence of SO) where linearizability uses linearization points. Grassroots [51,29] extends the CRDT [57] tradition with compositional Byzantine tolerance via the Blocklace [2]. We

prove that an agent-deterministic, peer-to-peer, message-passing implementation correctly implements maGLP's nondeterministic shared-variable semantics. Session types establish analogous implementation-correctness results for abstract models [59]; we are not aware of an actual concurrent programming language implementation [3,43,38,12,11,25] accompanied by such a proof. AI-driven derivation of implementations from formal specifications [15,42,7] is an emerging discipline; we contribute a three-layer (math→spec→Dart) variant at greater scale (Section 7). See Section 9 for fuller discussion.

Paper outline. Section 2 recalls transition systems and the notion of one transition system correctly implementing another. Section 3 presents GLP syntax and its concurrent operational semantics cGLP, together with the deterministic dGLP that correctly implements it. Section 4 recalls multiagent transition systems and atomic transactions, the setting in which agents share variable pairs across boundaries. Section 5 recalls the multiagent maGLP, defines its agent-deterministic, message-passing counterpart madGLP, and proves madGLP correctly implements maGLP for authorised runs. Section 6 provides the complete madGLP specification: full definitions, remarks, and correctness proofs. Section 7 describes the three-layer (math→spec→Dart) AI development methodology and the implementations derived through it, with concrete examples of revisions driven by implementation feedback. Section 8 proves that madGLP is grassroots. Section 9 discusses related work. Section 10 concludes. Appendix A provides detailed example traces, Appendix B the byte-level code format, Appendix C implementation notes on the runtime, and Appendix D the development history.

2 Transition Systems and Implementations

This section presents transition systems and implementations among them, providing the mathematical foundation for proving that one transition system correctly implements another.

2.1 Transition Systems

Definition 2.1 (Transition System, Computation, Run, Safe, Live, Correct [49,29]). *A **transition system** is a tuple $TS = (C, c_0, T, \sim)$ where C is an arbitrary set of **configurations**, $c_0 \in C$ a designated **initial configuration**, $T \subseteq C \times C$ a set of **transitions**, each a pair $c \to c'$ of non-identical configurations $c \neq c' \in C$, and $\sim$ a **partial equivalence relation** on T, symmetric and transitive; its domain $\{t \in T : t \sim t\}$ is partitioned into countably many **classes**, $[t]$ being the class of t, and a transition outside the domain belongs to no class.*

*A **computation** is a (nonempty, finite or infinite) sequence of configurations $c_1, c_2, \ldots$; it is a **run** if $c_1 = c_0$, and **safe** if $c_i \to c_{i+1} \in T$ for every two consecutive configurations. We write $c \xrightarrow{*} c'$ for the existence of a safe computation from c to c' (empty if $c = c'$). A class $[t] \in T/\sim$ is **enabled** in c if $c \to c' \in [t]$ for some c', and c is **terminal** if no class is enabled in it. A run is **live** if no*

class $[t]$ *is enabled in every configuration of some suffix in which no member of* $[t]$ *occurs, and* ***correct*** *if it is safe and live.*

Outcomes. A run is **complete** if it is infinite or ends in a terminal configuration. GLP transition systems additionally carry an **outcome**: a domain-specific function from complete runs to an outcome space—for GLP, the computed answer (Section 3). Outcomes serve only to state outcome-completeness of implementations (Definition 2.3).

Liveness. A transition in no class may occur in a correct run and is never required to [29]. The $\sim$ of cGLP (Section 3) and of dGLP are equivalences, every transition in a class; those of maGLP and madGLP (Section 5) leave the person's input outside their domain.

2.2 Implementations

Definition 2.2 (Implementation). *Given two transition systems* $TS = (C, c_0, T, \sim)$ *(the* ***specification****) and* $TS' = (C', c'_0, T', \sim')$*, an* ***implementation of*** TS ***by*** TS' *is a function* $\tau : C' \to C$ *where* $\tau(c'_0) = c_0$*, in which case the pair* (TS', τ) *is referred to as* ***an implementation of*** TS*.*

Given a computation $r' = c'_1 \to c'_2 \to \ldots$ *of* TS'*,* $\tau(r')$ *is the (possibly empty) computation* $\tau(c'_1) \to \tau(c'_2) \to \ldots$ *obtained from the sequence by removing consecutively repetitive elements so that* $\tau(r')$ *has no* ***stutter transitions*** *of the form* $c \to c$*.*

The implementation mapping τ (written σ in [49,29]; we keep σ for substitutions) need not preserve transitions: an implementation transition $c' \to d'$ may map to a stutter ($\tau(c') = \tau(d')$), or two consecutive specification transitions may correspond to a single implementation transition. Correctness of the implementation is captured by the conditions of Definition 2.3, which depend in turn on the outcome function defined above.

Definition 2.3 (Correct and Outcome-Complete Implementation). *The implementation* (TS', τ) *of* TS *is:*

- ***correct*** *if* τ *maps every correct run of* TS' *to a correct run of* TS
- ***outcome-complete*** *if for every complete* TS *run with outcome* O*, some complete* TS' *run has outcome* O

Correctness carries two obligations on the non-stutter image $\tau(r')$: that it is a run of TS—its consecutive distinct configurations are transitions of TS—and that it is live. We require outcome-completeness rather than run-completeness (every correct TS run being a τ-image), because the agent-deterministic implementations below realise every specification outcome without matching the specification's reduction order.

3 GLP and Its Implementation

3.1 GLP Syntax and Operational Semantics

This section presents Grassroots Logic Programs (GLP), a concurrent logic programming language in which logic variables are partitioned into paired *readers* and *writers*.

Definition 3.1 (GLP Variables). *Let $\mathcal{V}$ denote the set of* ***writers*** *(identifiers beginning with uppercase). Define $\mathcal{V}? = \{X? \mid X \in \mathcal{V}\}$, called* ***readers****. The set of all GLP variables is $\hat{\mathcal{V}} = \mathcal{V} \cup \mathcal{V}?$. A writer X and its reader $X?$ form a* ***variable pair****.*

Definition 3.2 (GLP Terms and Goals). *A* ***term*** *is a variable in $\hat{\mathcal{V}}$, a constant (numbers, strings, or the empty list* `[]`*), or a compound term $f(T_1, \ldots, T_n)$ with functor f and subterms T_i. Let $\mathcal{T}$ denote the set of all terms. We use standard list notation:* `[X|Xs]` *for a list cell,* `[X1,...,Xn]` *for finite lists. A term is* ***ground*** *if it contains no variables.*

A ***unit goal*** *is a compound term or a string. A* ***goal*** *is a multiset of unit goals; the empty goal is written* `true`*. A* ***clause*** *A `:-` B has head A (a unit goal) and body B (a goal); a* ***unit clause*** *has empty body.*

Definition 3.3 (Single-Occurrence (SO) Invariant). *A term, goal, or clause satisfies the* ***single-occurrence (SO) invariant*** *if every variable occurs in it at most once.*

Definition 3.4 (GLP Program). *A clause C satisfies the* ***single-reader/single-writer (SRSW) restriction*** *if it satisfies SO and a variable occurs in C iff its paired variable also occurs in C. A* ***GLP program*** *is a finite sequence of clauses satisfying SRSW; clauses for the same predicate form a* ***procedure****. The set of GLP goals $\hat{\mathcal{G}}(P)$ includes all goals over $\hat{\mathcal{V}}$ and the vocabulary of P that satisfy SO.*

The SO and SRSW restrictions ensure that GLP eschews unification in favour of simple term matching. A guard that implies its argument is ground relaxes the single-reader half of SRSW for that reader: a ground value has no writer left to pair with, so further occurrences of the reader in the clause consume nothing [53]. Program 5.2 relies on this.

Definition 3.5 (Substitutions and Assignments [53]). *A GLP* ***writer assignment*** *is a term of the form $X := T$, $X \in \mathcal{V}$, $T \notin \mathcal{V}$, satisfying SO. Similarly, a GLP* ***reader assignment*** *is a term of the form $X? := T$, $X? \in \mathcal{V}?$, $T \notin \mathcal{V}$, satisfying SO. A* ***writers (readers) substitution*** *σ is the substitution implied by a set of writer (reader) assignments that jointly satisfy SO. Given a writers assignment $X := T$, its* ***readers counterpart*** *is $X? := T$, and given a writers substitution σ, its* ***readers counterpart*** *$\sigma?$ is the readers substitution defined by $X?\sigma? = X\sigma$. Given a reader assignment $X? := T$, its* ***writers counterpart*** *is $X := T$, and given a readers substitution σ, its* ***writers counterpart*** *$\sigma!$ is the writers substitution defined by $X\sigma! = X?\sigma$. The* ***pair completion*** *of a readers substitution σ is $\sigma^\star = \sigma \cup \sigma!$, applied to a fixed point.*

A writers substitution applied to a term assigns only writers; readers are left unchanged.

Definition 3.6 (GLP Renaming, Renaming Apart). *A **GLP renaming** is an injective substitution $\rho : \hat{\mathcal{V}} \to \hat{\mathcal{V}}$ such that for each $X \in \mathcal{V}$: $X\rho \in \mathcal{V}$ and $X?\rho = (X\rho)?$. Two GLP terms **have a variable in common** if for some writer $X \in \mathcal{V}$, either X or $X?$ occurs in both. A GLP renaming σ **renames** T' **apart from** T if $T'\sigma$ and T have no variable in common.*

Definition 3.7 (Writer MGU [53]). *Given two GLP unit goals A and H, a **writer mgu** is a writers substitution σ such that $A\sigma = H\sigma$ and σ is most general among such substitutions.*

Here "most general" is under the standard subsumption ordering on substitutions: $\sigma \leq \tau$ iff there exists θ such that $\sigma\theta = \tau$. The operational algorithm for computing the writer mgu via term matching, including a table of all writer/reader/term combinations, is given below.

If two terms T_1 and T_2 that jointly satisfy SO are unifiable with an mgu σ, then σ maps any variable in T_1 to a subterm of T_2 and vice versa. Hence, the SO invariant of GLP allows eschewing unification in favour of *term matching* that performs joint term-tree traversal and collects variable assignments along the way.

Definition 3.8 (Term Matching). *Given two terms T_1 and T_2 that jointly satisfy SO, their **term matching** proceeds via the joint traversal of the term-trees of T_1 and T_2, consulting the following table at each pair of joint vertices, where X_1, X_2 denote writers, $X_1?, X_2?$ denote readers, and f/n denotes a non-variable term, a constant when $n = 0$ and a compound term when $n > 0$:*

$T_1 \backslash T_2$	*Writer* X_2	*Reader* $X_2?$	*Term* f_2/n_2
Writer X_1	*fail*	$X_1 := X_2?$	$X_1 := T_2$
Reader $X_1?$	$X_2 := X_1?$	*fail*	*suspend on* $X_1?$
Term f_1/n_1	$X_2 := T_1$	*fail*	*fail if* $f_1 \neq f_2$ *or* $n_1 \neq n_2$

The writer mgu is the union of all writer assignments if no fail *was encountered and the suspension set is empty.*

Example 3.9 (Fair Merge) *The concurrent logic program fairly merges two streams:*

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge(Xs, [], Xs?).
merge([], Ys, Ys?).
```

In the goal `merge([1,2,3|In1?],[a,b|In2?],Out)`*, the writer* `Out` *is the output stream* `merge` *produces, and the readers* `In1?`*,* `In2?` *are open input-stream tails that other processes extend by assigning their paired writers* `In1`*,* `In2`*. The first two clauses swap inputs in their recursive calls, ensuring fairness when both streams are available; when neither input has a list cell (both start with a reader), the goal suspends until one is assigned. Different scheduling of assignments to* `In1` *and* `In2` *may yield different output interleavings.*

Example 3.10 (Writer MGU and Reduction) *Consider reducing the goal of Example 3.9 with its first* `merge` *clause. The goal and clause have no variable in common, so no renaming apart is needed. Term matching proceeds position by position. In position 1,* `[1,2,3|In1?]` *matches* `[X|Xs]`*, yielding* `X := 1` *and* `Xs := [2,3|In1?]`*. In position 2,* `[a,b|In2?]` *matches* `Ys`*, yielding* `Ys := [a,b|In2?]`*. In position 3,* `Out` *matches* `[X?|Zs?]`*, yielding* `Out := [X?|Zs?]`*. The writer mgu is* $\hat{\sigma} = \{$`X:=1, Xs:=[2,3|In1?], Ys:=[a,b|In2?], Out:=[X?|Zs?]`$\}$*. Note that the readers* `In1?`*,* `In2?`*,* `X?`*,* `Zs?` *appear in assigned terms but are not themselves assigned—only writers are assigned by the writer mgu.*

Definition 3.11 (GLP Goal/Clause Reduction). *Given GLP unit goal A and clause C, with H* `:-` *B being the result of the GLP renaming of C apart from A, the* ***GLP reduction*** *of A with C* ***succeeds with result*** *(B, σ) if A and H have a writer mgu σ.*

GLP clauses may include *guards*—tests that determine clause applicability [53].

Definition 3.12 (Guarded Clause [53]). *A* ***guarded clause*** *has the form H* `:-` *G* `|` *B, where H is the head, G is a conjunction of guard predicates, and B is the body. Guard arguments are readers paired to head writers.*

Guards have three-valued semantics: each guard predicate defines its *success* condition; a guard *suspends* if it does not succeed but some instance of it under a readers substitution would succeed; it *fails* if no such instance exists. A conjunction succeeds if all members succeed, suspends if any member suspends and none fail, and fails if any member fails. The GLP reduction of Definition 3.11 is augmented to succeed if the guard also succeeds, and the suspension set of Definition 3.20 below likewise collects the readers blocking a suspended guard. The guard and system-predicate reference appears in [53].

Definition 3.13 (Goal Identity [53]). *Each unit goal in a cGLP computation carries a unique* ***identifier*** *assigned when it is spawned: the goals of the initial goal G_0 receive distinct identifiers, and each goal spawned by a reduction receives a fresh identifier. Instantiating a reader preserves the identifier of the goal in which it occurs.*

Preserving identity under reader instantiation lets equivalent Reduce transitions operate on differently instantiated forms of the same goal, on which the correctness results below rest.

The following definition provides a standard concurrent (interleaving-based) operational semantics for GLP, which we refer to as cGLP.

Definition 3.14 (cGLP Transition System [53]). *Given a GLP program P, an* ***asynchronous resolvent*** *over P is a pair (G, σ) where $G \in \hat{\mathcal{G}}(P)$ and σ is a readers substitution.*

A transition system $cGLP = (\mathcal{C}, c_0, \mathcal{T}, \sim)$ is a ***cGLP transition system*** *over P and initial goal G_0 satisfying SO if:*
1. $\mathcal{C}$ is the set of all asynchronous resolvents over P

2. $c_0 = (G_0, \emptyset)$
3. $\mathcal{T}$ *is the set of all transitions* $(G, \sigma) \to (G', \sigma')$ *satisfying either:*
   (a) ***Reduce:*** *there exists a unit goal* $A \in G$ *such that* $C \in P$ *is the first clause for which the GLP reduction of* A *with* C *succeeds with result* $(B, \hat{\sigma})$, $G' = (G \backslash \{A\}) \cup B(\hat{\sigma}?)^\star$, *and* $\sigma' = \sigma \circ \tau$, *where* τ *is the restriction of* $(\hat{\sigma}?)^\star$ *to the readers of the writers of* A.
   (b) ***Communicate:*** $\{X? := T\} \in \sigma$, $X? \in G$, $G' = G\{X? := T\}$, *and* $\sigma' = \sigma$.
4. $\sim$, *the* ***cGLP transition equivalence****, relates* $t_1 \sim t_2$ *iff either both are Reduce transitions reducing the same goal (by identity, Definition 3.13) with the same clause, or both are Communicate transitions applying the counterpart of the same writer assignment to the same goal (by identity)*

cGLP Reduce uses a writer mgu instead of standard unification, and chooses the first applicable clause instead of any clause. The first is the fundamental use of GLP readers for synchronisation. The second allows writing fair concurrent programs. The head writers' values are applied to the body, whose readers are theirs; the goal writers' values are communicated to their readers, which are elsewhere.

The cGLP Communicate rule realises the use of reader/writer pairs for asynchronous communication: it communicates an assignment from its writer to its paired reader.

cGLP has two forms of nondeterminism: the choice of $A \in G$ (*and-nondeterminism*), and the choice of which Communicate to apply.

Example 3.15 (Reduce and Communicate) *Continuing Example 3.10, the Reduce transition produces the new goal* $G' =$ {`merge([a,b|In2?], [2,3|In1?], Zs)`}*: the body of the clause with the head writers' values applied,* `Ys?` *taking* `[a,b|In2?]` *and* `Xs?` *taking* `[2,3|In1?]`*. The goal writer* `Out` *is assigned* `[X?|Zs?]`*, that is* `[1|Zs?]` *with* `X`*'s value applied, and its reader* `Out?` *is elsewhere, so* `Out?:`*[1/Zs?]*= *is added to* σ*, to be communicated. The new goal is ready for the next Reduce, which will match the second clause (swapping the inputs back).*

Definition 3.16 (Proper Run, Outcome [53]). *A cGLP run* $r : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *is* ***proper*** *if for any* $1 \leq i < n$, *a variable that occurs in* G_{i+1} *but not in* G_i *also does not occur in any* G_j, $j < i$. *If proper, the* ***outcome*** *of* r *is* $(G_0$ `:-` $G_n)\sigma_n^\star$.

Pair completion (Definition 3.5) is employed since σ_n accumulates only readers counterparts, leaving the writers of G_0 unbound.

Lemma 3.17 (Disjoint Substitution Commutativity) *Let* $\hat{\sigma}_1$ *and* $\hat{\sigma}_2$ *be writers substitutions produced by two cGLP Reduce transitions on distinct goals* A_1 *and* A_2 *in the same run. Then* $\hat{\sigma}_1$ *and* $\hat{\sigma}_2$ *assign disjoint sets of variables, and* $\hat{\sigma}_1 \circ \hat{\sigma}_2 = \hat{\sigma}_2 \circ \hat{\sigma}_1$.

Proof. By the SO invariant, each writer occurs at most once in the goal multiset G. Since $A_1 \neq A_2$ are distinct goals, they have disjoint writers. Each Reduce

assigns only writers from its reduced goal (by Definition 3.7). Therefore $\hat{\sigma}_1$ and $\hat{\sigma}_2$ have disjoint domains. Substitutions with disjoint domains commute.

Lemma 3.18 (Reduction Set Determines Outcome) *Let r_1 and r_2 be two proper cGLP runs from $(G_0, \emptyset)$ that perform pairwise equivalent sets of Reduce transitions—for each reduction of a goal (by identity) with a clause in r_1, there is an equivalent reduction in r_2, and vice versa. Then r_1 and r_2 have the same outcome.*

Proof. Let r_1 perform reductions in order $R_{\pi_1(1)}, \ldots, R_{\pi_1(k)}$ and r_2 in order $R_{\pi_2(1)}, \ldots, R_{\pi_2(k)}$ for permutations π_1, π_2, where R_i and the corresponding reduction in the other run are equivalent (same goal by identity, same clause).

Since each writer mgu assigns only writers (Definition 3.5), equivalent Reduce transitions produce the same writer mgu $\hat{\sigma}_i$ regardless of which reader substitutions have been applied. By Lemma 3.17, these substitutions have disjoint domains and commute. Therefore: $\hat{\sigma}_{\pi_1(1)} \circ \cdots \circ \hat{\sigma}_{\pi_1(k)} = \hat{\sigma}_{\pi_2(1)} \circ \cdots \circ \hat{\sigma}_{\pi_2(k)}$

The final readers substitution σ_n is the composition of the communicated parts of the readers counterparts, which likewise commute.

The final goal G_n consists of goals added by clause bodies (which depends only on which reductions occurred, not their order) with the accumulated substitution applied (which is the same by the above).

Communicate transitions apply reader assignments but do not affect the outcome: they do not change σ_n and merely propagate existing assignments within goals; a head writer's value is applied to the body at the Reduce, the same in both runs. Therefore both runs have the same σ_n, hence the same pair completion $\sigma_n^\star$ and the same outcome $(G_0 \texttt{ :- } G_n)\sigma_n^\star$.

Two equivalent Reduce transitions may operate on different instantiations of the same goal—with different reader substitutions applied—but produce the same writer mgu (since the writer mgu assigns only writers and is unaffected by reader substitutions).

Remark 3.19 (Properness) *The properness condition ensures fresh variables introduced by clause renaming are indeed fresh with respect to the entire computation history. This is guaranteed by the GLP renaming-apart requirement in Definition 3.6.*

A *cGLP run* is a run of the cGLP transition system; it starts at c_0 and follows only cGLP transitions, hence is safe, collapsing Definition 2.1's run/safe-run distinction. Correctness of a cGLP run thus reduces to liveness.

Deterministic implementations of GLP refine reduction with explicit suspension and failure, and reactivate a suspended goal when one of its blocking readers is assigned.

Definition 3.20 (Reduction Suspension and Failure). *Given unit goal A and clause C, with H :- B being the result of the GLP renaming of C apart from A:*

– *The reduction **succeeds** if A and H have a writer mgu σ (Definition 3.11)*

- *The reduction* ***suspends with suspension set*** W *if no writer mgu exists, but there exists a set* $W \subseteq \mathcal{V}?$ *of readers occurring in* A *such that for some ground substitution* θ *on* W*, the terms* $A\theta$ *and* H *have a writer mgu.* W *is the set of readers of* A *matched against a non-variable subterm of* H*.*
- *The reduction* ***fails*** *if it neither succeeds nor suspends*

Definition 3.21 (Reactivation Set). *Given a set* S *of pairs* (G, W)*, each a suspended goal* G *with its suspension set* $W \subseteq \mathcal{V}?$ *(Definition 3.20), and a readers substitution* $\sigma?$*, the* ***reactivation set*** *is:*

$$reactivate(S, \sigma?) = \{G : (G, W) \in S \wedge \exists X? \in W.\ X?\sigma? \neq X?\}$$

These are goals suspended on readers that have been instantiated by $\sigma?$*.*

3.2 Deterministic Implementation of cGLP

Motivation. dGLP serves as the formal specification for AI to implement GLP on workstations. The discipline is: from the mathematical definition (this section), AI derives an English+code specification, and from this specification derives Dart code.

While cGLP (Definition 3.14) is nondeterministic in goal selection and abstracts away suspension and failure, dGLP provides:

1. Deterministic FIFO scheduling of active goals
2. Explicit tracking of suspended goals with their suspension sets
3. Explicit tracking of failed goals
4. Automatic reactivation of suspended goals when blocking readers are instantiated

The key simplification compared to maGLP is that all variables are local, so reader assignments can be applied immediately rather than through asynchronous communication.

Example 3.22 (Suspension and Failure) *Consider the goal* `merge(Xs?, Ys?, Zs)` *where both input streams start with readers. Matching against the first clause head* `merge([X1|Xs1], Ys1, [X1?|Zs1?])` *fails at position 1:* `Xs?` *is a reader and* `[X1|Xs1]` *is a compound term, so no writer mgu exists. However, if* `Xs?` *were instantiated to a list cell (e.g.,* `[1|Xs1?]`*), a writer mgu would exist. Hence the reduction suspends with suspension set* $W = \{$`Xs?`$\}$*. The goal is suspended until* `Xs?` *is assigned.*

Conversely, `merge(hello, Ys?, Zs)` *fails outright against all clauses: the constant* `hello` *cannot match any clause head's first argument, regardless of what readers are instantiated to.*

Definition 3.23 (dGLP Configuration). *A* ***dGLP configuration*** *over program* P *is a triple* (Q, S, F) *where:*

- Q *is a sequence (queue) of* ***active*** *unit goals*
- S *is a set of* ***suspended*** *unit goals, each paired with its suspension set of blocking readers*
- F *is a set of* ***failed*** *unit goals*

Definition 3.24 (dGLP Transition System). *The transition system $dGLP(P) = (\mathcal{C}, c_0, \mathcal{T}, \mathrm{id})$ over GLP program P and initial goal G_0 satisfying SO is defined by:*

1. *$\mathcal{C}$ is the set of all dGLP configurations over P*
2. *$c_0 = (G_0, \emptyset, \emptyset)$*
3. *$\mathcal{T}$ is the set of all transitions $(Q, S, F) \to (Q', S', F')$ where $Q = A \cdot Q_r$ (with $\cdot$ denoting the queue with head element A and remainder Q_r) and exactly one of the following holds:*
   (a) ***Reduce:*** *There exists a first clause $C \in P$ for which the GLP reduction of A with C (Definition 3.11) succeeds with $(B, \hat{\sigma})$. Let $R = reactivate(S, \hat{\sigma}?)$. Then:*
   $$Q' = (Q_r \cdot B \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S' = S \setminus \{(G, W) : G \in R\}, \quad F' = F$$
   (b) ***Suspend:*** *No clause succeeds, but the GLP reduction of A with at least one clause $C \in P$ suspends. Let $W = \bigcup_{C \in P} W_C$ where W_C is the suspension set for clause C. Then:*
   $$Q' = Q_r, \quad S' = S \cup \{(A, W)\}, \quad F' = F$$
   (c) ***Fail:*** *No clause succeeds and no clause suspends (all clauses fail outright). Then:*
   $$Q' = Q_r, \quad S' = S, \quad F' = F \cup \{A\}$$
4. *$\sim$ is the identity relation: dGLP is deterministic, so each transition forms its own class*

Remark 3.25 (Deterministic Scheduling) *The dGLP transition system is deterministic: given any non-terminal configuration, exactly one transition is enabled. Goal selection follows FIFO order from the active queue Q, and clause selection uses the first applicable clause as in cGLP.*

Remark 3.26 (Immediate Reader Application) *In dGLP, the goal writers' values, which cGLP communicates, are applied to the goals holding their readers immediately during reduction rather than through separate Communicate transitions; the head writers' values the body takes, as in cGLP. This is sound for single-agent execution where all variables are local.*

dGLP Correctly implements cGLP.

Definition 3.27 (Implementation Mapping τ). *Given a dGLP computation $r' = c'_0 \to c'_1 \to \ldots \to c'_n$ where $c'_n = (Q, S, F)$, the implementation mapping τ is defined by:*

$$\tau(c'_n) = (G, \sigma_r)$$

where $G = Q \cup \{A : (A, W) \in S\} \cup F$ is the multiset union of active, suspended, and failed goals, and $\sigma_r = \tau_1 \circ \tau_2 \circ \ldots \circ \tau_k$ is the composition of the communicated parts of the readers substitutions produced by Reduce transitions along r', τ_i the restriction of $(\hat{\sigma}_i?)^\star$ to the readers of the writers of the reduced goal (Definition 3.14).

Lemma 3.28 (τ is Correct) *The implementation mapping τ is correct.*

Proof. Since dGLP is deterministic (Remark 3.25), every complete dGLP run is trivially live—there is at most one enabled transition at any configuration, and it is taken.

Reduce: Since dGLP's FIFO scheduling eventually processes every goal that enters the queue, for every cGLP Reduce equivalence class $[t]$ that becomes enabled, eventually some equivalent reduction is taken in $\tau(r')$.

Communicate: In dGLP, the communicated part of $\hat{\sigma}?$ is applied immediately during Reduce (Remark 3.26). Each such application corresponds to the cGLP Communicate equivalence classes that would apply those reader assignments. Hence every enabled cGLP Communicate equivalence class is taken when the corresponding dGLP Reduce fires.

Therefore τ maps correct dGLP runs to correct cGLP runs.

While dGLP cannot match cGLP's arbitrary reduction order step-by-step (cGLP is nondeterministic in goal selection; dGLP uses FIFO), it achieves the same *outcomes*. This follows from the commutativity of disjoint substitution compositions.

Lemma 3.29 (dGLP Performs Equivalent Reductions) *For every proper cGLP run r that performs Reduce transitions $\{R_1, \ldots, R_k\}$, there exists a dGLP run r' that performs a pairwise equivalent set of Reduce transitions.*

Proof. By induction on spawn order. Goals in the initial goal G_0 are in the initial dGLP queue with the same identifiers. For any R_i reducing goal A_i (by identity) with clause C_i: if A_i was spawned by an earlier reduction R_j, by induction dGLP performs an equivalent reduction, so A_i enters the queue with the same identity. Since R_i succeeds in r, A_i and the head of C_i have a writer mgu. Since the writer mgu assigns only writers (Definition 3.5), this writer mgu is invariant under any reader substitutions applied to A_i, so an equivalent Reduce—reducing A_i (by identity) with C_i—is enabled in dGLP. When A_i reaches the queue head, the equivalent reduction succeeds with the same writer mgu.

Lemma 3.30 (dGLP Outcome-Completeness) *For every proper cGLP run r with outcome O, there exists a dGLP run r' with outcome O.*

Proof. Let r be a proper cGLP run with outcome $O = (G_0 \text{ :- } G_n)\sigma_n^\star$.

Let r perform Reduce transitions $\{R_1, \ldots, R_k\}$.

By Lemma 3.29, there exists a dGLP run r' that performs a pairwise equivalent set of reductions, possibly in a different order and with interleaved Suspend/reactivation steps.

By Lemma 3.18, since r and r' perform pairwise equivalent reductions, they have the same outcome O.

Theorem 3.31 (dGLP Correctly Implements cGLP) *The implementation $(dGLP, \tau)$ of cGLP is correct and outcome-complete.*

Proof. Correct: By Lemma 3.28. *Outcome-complete:* By Lemma 3.30.

Remark 3.32 (Step-by-Step vs. Outcome Equivalence) *Step-by-step completeness would require dGLP to match cGLP's transitions in order, which is impossible due to cGLP's nondeterministic goal selection vs. dGLP's FIFO scheduling. Outcome-completeness is the appropriate notion: what matters for program correctness is the final result (the clause derived and final substitution), not the intermediate reduction order.*

Bounded Tail Reduction. Strict FIFO appends every body goal to the back of Q (Definition 3.24), so a tail-recursive goal—one whose body is a single continuation—is enqueued and dequeued at every reduction. A workstation implementation avoids this churn by reducing the continuation in place, ahead of the queue; left unbounded, this lets one goal monopolise the engine and starve the goals already queued behind it, breaking liveness. dGLP therefore admits a *bounded* tail optimisation, realised in the bytecode by the `requeue` instruction (Section B.3).

Definition 3.33 (b-Bounded Tail Schedule). *Fix $b \geq 1$. In a Reduce of the queue head A, at most one goal of the body B is designated the **tail goal** A'; the remaining goals of B and the reactivated goals R are appended to the back of Q as in Definition 3.24. A **b-bounded tail schedule** relaxes the FIFO order of Remark 3.25 only as follows: the tail goal A' may be reduced ahead of the goals standing before it in Q_τ, for at most b such tail reductions in succession; after b consecutive tail reductions the next tail goal is appended to the back of Q, restoring FIFO.*

The bound is finite but otherwise free: its value trades scheduling overhead against the latency the queued goals see, and carries no semantic weight; the implementation fixes it (currently $b = 26$). At $b = 1$ the schedule is strict FIFO—each tail goal is reduced once and then yields—correct, but forgoing the optimisation.

Corollary 3.34 (Bounded Tail Reduction Preserves Correctness) *For every $b \geq 1$, $(dGLP, \tau)$ confined to b-bounded tail schedules is a correct and outcome-complete implementation of cGLP.*

Proof. A b-bounded tail schedule performs exactly the Reduce transitions of the corresponding strict-FIFO run, only in a different order: each tail goal is reduced earlier than its FIFO position, and every goal standing behind it is delayed by at most b further reductions. Outcome-completeness is therefore immediate from Lemma 3.18, which depends on the set of reductions performed and not on their order. For correctness (liveness), b is finite, so the in-place continuation always yields within b steps; hence every goal that enters Q reaches the head in finitely many transitions and every enabled Reduce class is eventually taken, as in Lemma 3.28.

4 Multiagent Transition Systems

This section recalls the multiagent transition systems and atomic transactions needed to specify maGLP, drawing on the framework of volitional multiagent

atomic transactions [29] and the multiagent atomic transactions framework on which it builds [54].

We assume a potentially infinite set of *agents* Π, but consider only finite subsets of it, so when we refer to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible.

Remark 4.1 (Agent Names) *An agent $p \in \Pi$ is its public key, which the runtime makes and supplies to the networking layer (Appendix C.2), and has no other name. A readable name such as* `alice` *or* `bob`*, in a program, in a simulation or in this paper, is a variable standing for a public key, as* `Alice` *and* `Bob` *are in a contract.*

We use S^P to denote the set S indexed by the set P, and if $c \in S^P$ we use c_p to denote the member of c indexed by $p \in P$.

Definition 4.2 (Local States, Configuration, Transaction, Participants). *Given agents $Q \subset \Pi$ and an arbitrary set S of* ***local states****, a* ***configuration*** *over Q and S is a member of $C := S^Q$. An* ***atomic transaction****, or just* ***transaction****, over Q and S is any pair of configurations $t = c \to c' \in C^2$ such that $c \neq c'$, with $t_p := c_p \to c'_p$ for any $p \in Q$, and with p being an* ***active participant*** *in t if $c_p \neq c'_p$,* ***stationary participant*** *otherwise.*

4.1 Multiagent Transition Systems

Definition 4.3 (Multiagent Transition System). *Given agents $P \subset \Pi$ and an arbitrary set S of* ***local states*** *with a designated* ***initial local state*** *$s_0 \in S$, a* ***multiagent transition system*** *over P and S is a transition system $TS = (C, c_0, T, \sim)$ with* ***configurations*** *$C := S^P$,* ***initial configuration*** *$c_0 := \{s_0\}^P$,* ***transitions*** *$T \subseteq C^2$ being a set of transactions over P and S, and $\sim$ a partial equivalence on T.*

Definition 4.4 (Transaction Closure). *Let $P \subset \Pi$, S a set of local states, and $C := S^P$. For a transaction $t = (c \to c')$ over local states S with participants $Q \subseteq P$, the* ***P-closure of*** *t, $t{\uparrow}P$, is the set of transitions over P and S defined by:*

$$t{\uparrow}P := \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\}$$

If R is a set of transactions, each $t \in R$ over some $Q \subseteq P$ and S, then the ***P-closure of*** *R, $R{\uparrow}P$, is the set of transitions over P and S defined by:*

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

Given a relation $\sim$ on R, its ***P-closure*** *$\sim{\uparrow}P$ is the relation on $R{\uparrow}P$ with $\hat{t}$ $(\sim{\uparrow}P)$ $\hat{t}'$ iff $\hat{t} \in t{\uparrow}P$ and $\hat{t}' \in t'{\uparrow}P$ for some $t \sim t'$.*

The closure carries a partial equivalence on transactions to one on the induced transitions: where distinct transactions in R have disjoint P-closures—as

when every participant of every transaction changes its state—$\sim\!\uparrow\! P$ is a partial equivalence whenever $\sim$ is [29]. A transaction equivalence relates only transactions with the same participants [29]; the maGLP and madGLP equivalences below satisfy this.

Example 4.5 (Configurations, Transactions, Closure) *Let $P = \{p, q, o\}$ and $S = \{s_0, s_1\}$, writing a configuration as (c_p, c_q, c_o). Let $t = c \to c'$ be the transaction over $Q = \{p, q\}$ with $c_p = c_q = s_0$ and $c'_p = c'_q = s_1$; both participants are active. Its P-closure $t\!\uparrow\! P$ has two transitions, one per local state of the stationary o: $(s_0, s_0, s_0) \to (s_1, s_1, s_0)$ and $(s_0, s_0, s_1) \to (s_1, s_1, s_1)$.*

Definition 4.6 (Transactions-Based Multiagent Transition System). *Given agents $P \subset \Pi$, local states S with initial local state $s_0 \in S$, a set of transactions R each $t \in R$ over some $Q \subseteq P$ and S, and a partial equivalence $\sim$ on R, the **transactions-based multiagent transition system** over P, S, R, and $\sim$ is the multiagent transition system $TS = (S^P, \{s_0\}^P, R\!\uparrow\! P, \sim\!\uparrow\! P)$.*

5 Multiagent GLP and Its Implementation

5.1 Multiagent GLP

This section recalls maGLP—the multiagent operational semantics of GLP [53]—a transactions-based multiagent transition system (Section 4).

From GLP to Multiagent GLP. In extending GLP to multiple agents, each agent maintains its own asynchronous resolvent as its local state. GLP's variable pairs provide natural asynchronous channels for inter-agent communication: when agent p assigns a writer X for which the paired reader $X?$ is held by agent q, the assignment $X := T$ must be communicated to q.

A key difference between cGLP and maGLP is in the initial state. In a multiagent transition system all agents must have the same initial local state s_0 (Definition 4.3). This precludes setting up an initial configuration in which agents share logic variables, as this would imply different initial states for different agents.

We resolve this in two steps. First, we employ only anonymous logic variables "`_`" in the initial local states of agents: Anonymous variables are, on the one hand, syntactically identical, hence allow all initial states to be syntactically identical, and on the other hand represent unique variables, hence semantically all initial goals have unique, local, non-shared variables. The initial state of all agents is the atomic goal `agent(ch(_?,_),ch(_?,_))`, providing two bidirectional channels: the first to the person operating the machine, and the second to the network. The runtime writes its two anonymous readers: the person's responses to the first (the Input transaction below), and other agents' messages to the second (the Cold-call transaction below); a clause has no anonymous reader [53]. The first channel is a bidirectional stream of GLP messages between the machine and the person, mediated by runtime UI support below the GLP layer and by physical UI hardware; messages flowing toward the person are rendered as UI

elements, and the person's responses flow back as GLP messages. With appropriate UI runtime support, a GLP message to the person may contain a writer, in which case the runtime presents it as a question and an eventual response from the person becomes the corresponding assignment. The second channel carries communication with other agents.

Second, the Cold-call transaction enables agents to bootstrap communication by establishing shared variables through the network infrastructure, realising the cold-call protocol for connecting previously-disconnected agents.

Definition 5.1 (Variables of an Agent [53]). *Let $\mathcal{V}_p \subset \mathcal{V}$, for $p \in \Pi$, be pairwise disjoint infinite sets of writers. The* ***variables of agent*** *p are $\hat{\mathcal{V}}_p = \mathcal{V}_p \cup \{X? \mid X \in \mathcal{V}_p\}$.*

Definition 5.2 (Multiagent GLP [53]). *Given agents $P \subset \Pi$ and GLP program M, the* ***maGLP transition system*** *over P and M is the transactions-based multiagent transition system (Definition 4.6) over P, local states being asynchronous resolvents (G_p, σ_p) over M, initial local state $s_0 = (\{\texttt{agent(ch(_?,_),ch(_?,_))}\}, \emptyset)$, whose anonymous variables at agent p are variables of $\hat{\mathcal{V}}_p$, the following transactions $c \to c'$, and the equivalence $\sim$ given below:*

1. ***Reduce*** *p: A unary transaction with participant p where $c_p \to c'_p$ is a cGLP Reduce transition (Definition 3.14) whose GLP renaming maps the clause into $\hat{\mathcal{V}}_p$.*
2. ***Communicate*** *p* ***to*** *q: A transaction with participants $p, q \in P$ where $\{X? := T\} \in \sigma_p$, $X?$ occurs in G_q, and $c'_p = (G_p, \sigma_p \setminus \{X? := T\})$ and $c'_q = (G_q\{X? := T\}, \sigma_q)$ if $p \neq q$, and $c'_p = (G_p\{X? := T\}, \sigma_p \setminus \{X? := T\})$ if $p = q$.*
3. ***Cold-call*** *p* ***to*** *q: A binary transaction with participants $p \neq q \in P$ where the network output stream in c_p has a new message $\texttt{msg}(q, X)$, c'_p is the result of advancing the network output stream in c_p, and c'_q is the result of adding $\texttt{msg}(q, X)$ to the network input stream in c_q.*
4. ***Input*** *p: A unary transaction with participant p where $c'_p = (G_p, \sigma_p \cup \{X? := T\})$, $X?$ occurs in G_p, X is the person's — the writer of the tail of the person's input stream, or a writer in a message on the person's output stream in c_p — and T is a term whose variables are fresh variables of $\hat{\mathcal{V}}_p$.*
5. *$\sim$, the* ***maGLP transaction equivalence*** *on the transactions above, relates $t_1 \sim t_2$ iff both are Reduce transactions at the same agent p reducing the same goal (by identity, Definition 3.13) with the same clause, both are Communicate transactions from the same p to the same q applying the counterpart of the same writer assignment to the same goal (by identity), or both are Cold-call transactions from the same p to the same q delivering the same message; an Input transaction is related to none.*

In every transaction the non-participating agents are stationary (Definition 4.4), and every participant changes its state, so $\sim\!\uparrow P$ is a partial equivalence (Section 4); each agent has its own variables, so groups run apart share none. Input is the person's act at their own agent, writing what the program handed

them; it belongs to no class, since the person is under no obligation to act, as a volition change is in [29], and what the person writes may name any agent.

Example 5.3 (Cold-call) *With G_p writing* `msg`$(q, \texttt{intro}(X?))$ *on p's network output stream, Cold-call p to q advances that stream at p and appends* `msg`$(q, \texttt{intro}(X?))$ *to q's network input stream; the pair $(X, X?)$ is now shared, X at p and $X?$ at q, and subsequent Communicate transactions use it.*

Note that Communicate may be unary or binary, depending on whether $p = q$. Communicate transfers assignments from writers to readers within an agent or between agents sharing a paired reader and writer. Cold-call transfers a term with its variables to q through the network streams established in each agent's initial configuration, enabling the creation of paired variables among previously-disconnected agents.

The worked examples below start from application-level goals such as `client1(Xs)@p` and `monitor(Xs?)@q`, reachable from the universal initial state `agent(ch(_?,_),ch(_?,_))` by a boot phase.

Example 5.4 (A maGLP Round-Trip) *Suppose agents p and q already share a variable pair $(X, X?)$, with X held by p and $X?$ held by q, and that q's resolvent contains a goal that consumes $X?$ (and so suspends until $X?$ is assigned). A maGLP run proceeds: (i) Reduce p assigns $X := T$ in σ_p; (ii) Communicate p to q delivers the assignment to q and substitutes T for $X?$ in G_q; (iii) Reduce at q now proceeds. Detailed traces, including the friend-mediated introduction of two agents that eschews Cold-call, appear in Appendix A.*

Remark 5.5 (Cold-call as the Exception) *Cold-call is the exceptional means by which agents establish paired variables: once two agents share a pair, they can communicate via Communicate indefinitely; moreover, an agent with two friends (with which it shares channels) may introduce them to each other through its existing channels, eschewing Cold-call.*

Remark 5.6 (Transaction Degrees) *Reduce and Input are unary; Communicate is binary, unary when $p = q$; Cold-call is binary.*

A *maGLP run* is a run of the maGLP transition system, hence safe, collapsing Definition 2.1's run/safe-run distinction. Correctness of a maGLP run thus reduces to liveness.

Definition 5.7 (maGLP Proper Run, Outcome [53]). *An maGLP run is* ***proper*** *if the run at each agent is proper (Definition 3.16). The* ***outcome*** *of a proper maGLP run is the tuple, indexed by P, of the outcomes of the per-agent runs (Definition 3.16).*

Proposition 5.8 (maGLP SO Preservation [53]) *If the initial goals of all agents satisfy SO, then every goal in every agent's resolvent throughout a proper run satisfies SO.*

5.2 Implementing Multiagent GLP with Deterministic Agents

Motivation. madGLP serves as the formal specification for AI to implement maGLP on smartphones communicating peer-to-peer, without reliance on central servers. The discipline is: from the mathematical definition, AI derives an English+code specification, and from this specification derives Dart code.

This section specifies madGLP, an agent-deterministic transition system that implements maGLP by replacing each cross-agent shared variable pair with two local pairs—the reader of one linked to the writer of the other by a *global link*, realised through a global writers table and message passing, with forwarding handled by spawned GLP goals. A value crosses a link once, to the reader's current holder, and the holder's acknowledgement releases the link.

Realising the UI Channel. The maGLP initial state provides two bidirectional channels (Section 5.1): one to the person operating the machine, one to the network. madGLP implements the second via the global-link machinery described below. The first is realised by the Dart runtime below the madGLP layer, which bridges GLP messages and the physical UI hardware: outgoing GLP messages on this channel are rendered as UI elements, and the person's responses, captured by event handlers, flow back as GLP messages. When a message to the person contains a writer, the runtime presents it as a prompt and treats the person's response as the corresponding assignment, performed on the agent's local state: the Input transaction (Definition 6.21), which in madGLP as in maGLP is the person's and belongs to no class.

Implementing Shared Variables via Local Pairs. A maGLP shared variable pair $(X, X?)$ with writer X at agent p and reader $X?$ at agent q can be implemented by two local pairs, the reader of p's pair linked to the writer of q's pair by a *global link*:

- At agent p: a local pair $(X_p, X_p?)$ where both variables remain in p's resolvent
- At agent q: a local pair $(X_q, X_q?)$ where both variables remain in q's resolvent
- A global link connecting the reader $X_p?$ to the writer X_q, realised as a `global_send` goal at p that monitors $X_p?$ and an entry in q's global writers table that maps the global name to X_q

When X_p is assigned a term T at agent p, the value becomes available via $X_p?$. A spawned `global_send` goal detects this and sends the assignment message to q, where T is *localized* (Definition 5.10). Upon receipt, q looks up the target writer in its global writers table, assigns $X_q := T_q^{\downarrow}$, and removes the entry. This makes the value available through $X_q?$ in q's resolvent. Here $T^{\uparrow}$ and $T_q^{\downarrow}$ denote the *globalized* and *localized* forms of a term—local variables replaced by global names, and conversely. Beside its global writers table, each agent p keeps a set U_p of *imported-writer records*: a record (Y, o, k, s) marks the local writer Y as the sending end of the link anchored by $_w(o, k)$, with s the agent the name came from.

Definition 5.9 (Globalize). *Given agent p, remote agent q, and term T, the* ***globalization by*** *p, written $T_p^{\uparrow}$, may update the global writers table W_p' and the imported-writer records U_p', and spawn or remove goals in p's resolvent, as follows. For each variable Y occurring in T:*

1. *If Y is a writer: allocate the next index i, create entry (Y, q) at index i in W'_p, and replace Y in $T_p^{\uparrow}$ with $_w(p, i)$.*
2. *If $Y?$ is a reader: allocate the next index i, replace $Y?$ in $T_p^{\uparrow}$ with $_r(p, i)$, and spawn goal* `global_send`$(Y?, _r(p, i), q)$ *into p's resolvent.*
3. *If $Y?$ is the reader of an imported pair—its paired writer carries a global writers table entry (Y, o, k) with anchor o—then instead replace $Y?$ with the original name $_r(o, k)$ and remove the entry: the name is* ***forwarded****, and no goal is spawned.*
4. *If Y is the writer of an imported pair—U_p has the record (Y, o, k, s) with anchor o—then instead replace Y in $T_p^{\uparrow}$ with the original name $_w(o, k)$, remove the record and the* `global_send` *goal watching $Y?$, and create no entry: the name is* ***forwarded****, and p leaves the link. Single occurrence puts this case out of reach once the value has been sent: an assigned writer has no further occurrence to export.*

Globalize and Localize follow a uniform principle: a global writers table entry is created at the agent that will receive an assignment on the link, and a `global_send` goal is spawned at the agent that will send an assignment. Every global link has exactly one entry on one end and exactly one `global_send` goal on the other. An imported end is marked—a reader by its entry, a writer by its record—so re-exporting it forwards the original name instead of anchoring a new link at the exporter.

Definition 5.10 (Localize). *Given agent q, remote agent p, and globalized term $T_p^{\uparrow}$, the* ***localization by*** *q, written $T_q^{\downarrow}$, may update the global writers table W'_q and the imported-writer records U'_q, and spawn goals into q's resolvent, as follows. For each global name in $T_p^{\uparrow}$:*

1. *If $_w(p, i)$: create fresh local pair $(Y_q, Y_q?)$, replace $_w(p, i)$ with Y_q (the writer) in $T_q^{\downarrow}$, add the record (Y_q, p, i, s) to U'_q with s the sending agent, and spawn goal* `global_send`$(Y_q?, _w(p, i), p)$ *into q's resolvent. If the sending agent is not p, the name was forwarded, and the value that goal produces is held (Section 6.7).*
2. *If $_r(p, i)$: create fresh local pair $(Z_q, Z_q?)$, allocate the next index k in W'_q, add entry (Z_q, p, i), and replace $_r(p, i)$ with $Z_q?$ (the reader) in $T_q^{\downarrow}$. If the sending agent is not p, the name was forwarded, and q adds the request* `req`$(_r(p, i)), p)$ *to its outgoing messages—the request is how the anchor p learns the reader's current holder. The request is held (Section 6.7).*

This design keeps all variables in the resolvent, enabling correct behaviour even when both ends of a variable pair are exported to the same or different agents. A re-exported reader or writer travels under its original name, so a link keeps one anchor however far its end moves.

Definition 5.11 (Global Link). *A* ***global link*** *between agents p and q implements a maGLP shared variable pair $(X, X?)$ whose writer X is held by p and reader $X?$ by q; it consists of:*

– *At agent p: a local variable pair $(X_p, X_p?)$ where both occur in p's resolvent, and a spawned goal* `global_send`$(X_p?, _r(p, i), q)$

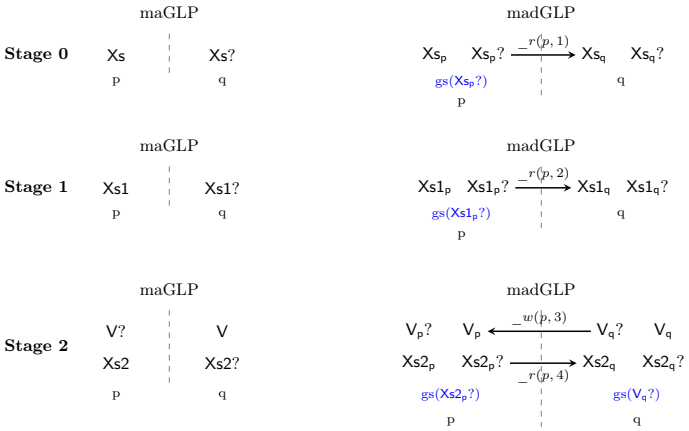


Fig. 1. maGLP vs madGLP variable linking through three stages. Left: maGLP shared variable pairs. Right: madGLP local pairs with global links (arrows from reader to writer, labelled with global names). Dashed lines separate agents p and q. "gs" denotes `global_send` goals watching readers. In Stage 2, the V link arrow points from q to p: when q assigns $\mathsf{V_q}$, the goal gs($\mathsf{V_q}$?) sends the value back to p.

– *At agent q: a local variable pair $(X_q, X_q?)$ where both occur in q's resolvent, and an entry (X_q, p, i) at index i in q's global writers table*

The maGLP writer X corresponds to X_p and the reader $X?$ to $X_q?$; the inner ends $X_p?$ and X_q have no maGLP counterpart. The mirror case, with writer held by q and reader by p, has a local pair $(Y_p, Y_p?)$ and entry (Y_p, q) at p, and a local pair $(Y_q, Y_q?)$, the record (Y_q, p, i, p) in U_q, and spawned goal `global_send`$(Y_q?, _w(p, i), p)$ at q.

A delivered value addressed by a reader name is retained by its sender as *pending* until acknowledged: the receiver, upon assigning the entry's writer, sends an acknowledgement, which releases the pending value and closes the link, and records the acknowledging holder as the holder of every link whose reader name the value carried; a request for a pending value redirects it to the requester. Writer-name and serializer values are not acknowledged—their entries are fixed. Traffic matching no entry or pending value is stale (indices are never reused) and is dropped, unless it reaches an end its receiver lost in a crash, which the receiver answers (Section 6.5).

Held Links. A link end whose other end is at an agent this runtime did not exchange the name with is *held*: nothing is sent on it, no assignment arriving on it is applied, and the runtime reports the link on the agent's network input stream. Four cases arise. At the holder of a forwarded reader, the request it creates is held, and at the holder of a forwarded writer, the value its `global_send` goal produces is held. At the anchor, an arriving request is held, and so is an assignment arriving from an agent the anchor did not export the name to. The system predicate `authorise_link/2` answers: authorise, and the runtime releases what it held; refuse, and it drops the link. How the holder's agent comes to know the anchor is decided above madGLP. A forwarded link is held at both ends, so two people

are asked: the holder before anything is sent on the link, and the anchor when the forwarded traffic reaches it. Section 6.7 gives the definitions.

Global links parallel the encodings of synchronous into asynchronous communication in the π-calculus [41,22,8]; Remark 5.12 records the design alternative and the cost.

Remark 5.12 (Global-Link Design and Cost) *The alternative we developed first and eliminated is variable migration—moving one end of a shared pair to the other end's agent—whose defects are recorded in Appendix D. A global link costs one* `global_send` *goal at one end and one global writers table entry at the other; the goal and entry are released when the value is acknowledged. Message traffic per reader-name assignment is the value and its acknowledgement, plus one request after a re-export; a writer-name assignment is one message, however far the writer has been forwarded.*

madGLP Local States and Transitions. Each madGLP agent executes its local resolvent with FIFO scheduling, tracking three kinds of goals: *active* (ready to reduce), *suspended* (waiting on readers), and *failed*. A reduction succeeds, suspends on a set W of readers, or fails (Definition 3.20); when a reader in W is later instantiated, the suspended goal is reactivated (Definition 3.21).

madGLP defines three transactions. *Reduce*, unary, performs a local reduction step at one agent. *Send p to q*, binary, takes a message from p's outgoing messages and applies it at q—a *value* is assigned to the local writer recorded for it in q's global writers table, reactivating any goals suspended on the corresponding reader, and is acknowledged; a *request* directs a pending value to its sender; an *acknowledgement* releases the pending value. *Input*, unary, writes the person's response at their agent, and belongs to no class. Cold-calls—messages between agents with no prior shared variables—are realised uniformly through a reserved index-0 serializer that lets multiple senders append to an agent's network input stream, so that a cold call is a Send like any other. Message delivery is the liveness of Send: a correct run takes every Send class that stays enabled. Two madGLP transitions are $\sim$-equivalent when they perform the same Reduce (same agent, goal by identity, clause) or the same Send (same sender, receiver and message). The formal Reduce, Send and Input transactions and this equivalence are defined in Definition 6.19.

Example: Client-Monitor Communication. We illustrate madGLP's variable linking with a client-monitor scenario.

Program 1: Concurrent Monitor

```
monitor(Reqs) :- monitor(Reqs?,0).
monitor([add|Reqs],Sum) :-
    Sum1 := Sum? + 1, monitor(Reqs?,Sum1?).
monitor([subtract|Reqs],Sum) :-
    Sum1 := Sum? - 1, monitor(Reqs?,Sum1?).
monitor([value(Sum?)|Reqs],Sum) :-
    integer(Sum?) |  monitor(Reqs?,Sum?).
monitor([],_).
```

The notation `integer(Sum?) | Body` is a *guard*: the body is executed only when the guard succeeds, i.e., when `Sum?` is instantiated to an integer. The notation `goal@p` designates that `goal` runs at agent p.

Consider the initial goal `client1(Xs)@p, monitor(Xs?)@q`, where `client1` stands for an arbitrary client process, external to the program, emitting `add`, `subtract`, and `value(_)` messages on the stream `Xs`. The initial goal establishes a shared pair with writer `Xs` at p and reader `Xs?` at q. The correspondence between maGLP shared variables and madGLP local pairs connected by global links proceeds through three stages (Figure 1):

- **Stage 0:** At boot time, the coordinator globalizes the initial goal, creating local pairs at each agent connected by global links. Agent p gets `client1`($\mathsf{Xs_p}$) and spawns `global_send`($\mathsf{Xs_p}?, _r(p,1), q$); agent q gets `monitor`($\mathsf{Xs_q}?$) and creates entry ($\mathsf{Xs_q}, p, 1$).
- **Stage 1:** After p assigns $\mathsf{Xs_p}$:= [`add`$|\mathsf{Xs1_p}?$], the `global_send` goal at p fires, sending the value to q. A new link is created for `Xs1` with its own `global_send` goal at p and entry at q.
- **Stage 2:** After p assigns $\mathsf{Xs1_p}$:= [`value`($\mathsf{V_p}$)$|\mathsf{Xs2_p}?$]. Note that p sends writer `V`, so p creates entry ($\mathsf{V_p}, q$) and q spawns `global_send`($\mathsf{V_q}?, _w(p,k), p$). When monitor assigns $\mathsf{V_q}$, the value flows $q \rightarrow p$.

Each delivered reader-name value is acknowledged, closing its link. Had q passed `Xs?` on to a third agent before a value arrived, the new holder's request would have redirected the pending value to it, and the stream would continue to the new holder directly.

6 madGLP Specification

This section contains the complete madGLP specification: full definitions, remarks, and correctness proofs. It serves as the formal specification from which AI derives the English+code spec and Dart implementation.

6.1 Global Variable Names

An agent counts its epochs: it starts in epoch 1, and a restart after a crash opens the next epoch (§6.9).

Definition 6.1 (Epoch). *The **epoch** of agent p is an integer $e_p \geq 1$ of its local state, 1 at the first start of p.*

Definition 6.2 (Global Variable Name). *A **global variable name** is a term of the form $_w(p,e,i)$ or $_r(p,e,i)$, where $p \in \Pi$ is an agent identifier, e is the epoch of p at the allocation and $i \in \mathbb{N}$ is an index allocated by p during globalization. The name $_w(p,e,i)$ denotes a writer globalized at p, and $_r(p,e,i)$ denotes a reader globalized at p.*

Where the epoch is not at issue we write $_w(p,i)$ for $_w(p,e_p,i)$ and $_r(p,i)$ for $_r(p,e_p,i)$, as the rest of this paper does.

A global name has two presentations. In a message between agents it stands in the place of a variable and is encoded as one (§B.2); it never stands for a variable in a resolvent. A ground structure $_w(P, E, I)$ or $_r(P, E, I)$ that a program puts in a message is a term, encoded as a structure and arriving as one (§B.2); only a variable is encoded as a global name. To a program it is a ground term—the 3-ary structures $_r(P, E, I)$ and $_w(P, E, I)$, the functor giving the polarity, the first argument the anchor, the second its epoch and the third the index—which is the form carried by `global_send` goals, by `send_to_net` (Remark 6.12), and by the reports and answers of §6.7. Global names identify the *anchor* of a global link and enable message routing; an unbound imported reader, and an imported writer, are forwarded under their original name (Definition 5.9), so a link's anchor is fixed however far its end travels. Over a real network, the agent name p in $_w(p, i)$ and $_r(p, i)$ is the public key of p (Remark 4.1); the simulation realisation uses symbolic agent names.

6.2 Global Writers Table

The global writers table tracks local writers that await incoming assignments from remote agents. Each entry maps a global name to the local writer that will be assigned when a message arrives.

Definition 6.3 (Global Writers Table Entry). *A **global writers table entry** at agent p is either:*

- (X, q) *for entries created by Globalize: $X \in \mathcal{V}$ is a local writer that will be assigned when a value arrives on the entry's name, and $q \in \Pi$ is the agent the name was exported to. A value from any other agent reaches the entry only after a forwarding, and is held (§6.7)*
- (X, q, i) *for entries created by Localize: $X \in \mathcal{V}$ is a local writer, $q \in \Pi$ is the remote agent, and $i \in \mathbb{N}$ is the index in q's global name (needed to match incoming messages)*

Definition 6.4 (Global Writers Table). *The **global writers table** W_p of agent p is an array of global writers table entries. For entries created by Globalize at index i, the index i is the index in the global name $_w(p, i)$. For entries created by Localize, the entry stores the remote index explicitly.*

Definition 6.5 (Imported-Writer Records). *The **imported-writer records** U_p of agent p is a set of quadruples (Y, o, k, s), where $Y \in \mathcal{V}$ is a local writer created by Localize from the global name $_w(o, k)$, $o \in \Pi$ is the link's anchor, $k \in \mathbb{N}$ its index at o, and $s \in \Pi$ the agent the name came from. The record marks Y as the sending end of that link. It is removed when the value is sent, and when Y is re-exported and the name forwarded (Definition 5.9).*

Remark 6.6 (What the Global Writers Table Stores) *The table contains only writers that await incoming assignments. When agent p globalizes a writer Y, it creates an entry for Y because p will receive the assignment: the agent the name was exported to holds the writer and sends the value back, or, after a forwarding, whichever agent then holds it. When agent q localizes*

a reader global name $_r(p,i)$*, it creates an entry for its local writer because* q *will receive the assignment. No entries are created for outgoing links—those are handled by* `global_send` *goals; an outgoing link whose writer was imported also carries a record (Definition 6.5).*

6.3 The global_send Predicate

Outgoing communication is handled by spawned goals rather than by table entries. This uniform approach correctly handles all cases, including forwarding when both ends of a variable pair are exported. The system predicate `global_send/3` takes a reader T whose value will be sent once known, a global variable name G (either $_w(p,i)$ or $_r(p,i)$) identifying the link, and a destination agent Q.

Definition 6.7 (global_send Predicate). *The system predicate* `global_send/3` *is defined as:*

```
global_send(T, G, Q) :- known(T?) | '_send'(T?, G?, Q?).
```

The guard `known(T?)` succeeds when T is assigned a non-variable term; the builtin `'_send'(T?, G?, Q?)` globalizes T and adds message $(G := T^{\uparrow}, Q)$ to the agent's outgoing message set. A sent reader-name value remains with its sender, *pending*, until acknowledged (§6.5), so a later request can redirect it to the reader's current holder. The full guard and body-kernel tables, including the row for `'_send'`, are catalogued in the companion GLP paper [53].

Remark 6.8 (Forwarding via global_send) *When an agent exports both ends of a variable pair (e.g., sending* $[\overline{X}, X?]$ *to another agent, or* X *to one agent and* $X?$ *to another), the Globalize operation spawns a* `global_send` *goal for the exported reader. If a value arrives on one global link and is assigned to a local writer* X*, then* $X?$ *becomes known, triggering any* `global_send` *goal watching* $X?$*. This automatically forwards the value without requiring special forwarding logic in the Send transaction.*

6.4 Networking Seam Predicates

Six system predicates give GLP code the networking layer's and the platform's services. Their effects lie outside the madGLP transition system, as those of `sign` and `self_module` do: the transactions of §6.8 do not mention them, and π does not see them.

Definition 6.9 (Seam Predicates). *The six seam predicates are defined as:*

```
peer_address(P, A?) :- ground(P?) | '_peer_address'(P?, A).
peer_app(P, A?)     :- ground(P?) | '_peer_app'(P?, A).
self_app(R?, I?)    :- '_self_app'(R, I).
punch_udp(A)       :- ground(A?) | '_punch_udp'(A?).
declare(place(P, R, E?)) :-
    ground(P?), ground(R?) | '_declare'(place(P?, R?, E)).
```

```
declare(nearby(U, S?)) :-
    ground(U?) | '_declare'(nearby(U?, S)).
declare(reachable(S?)) :- '_declare'(reachable(S)).
declare(public_address(S?)) :- '_declare'(public_address(S)).
declare(network(S?)) :- '_declare'(network(S)).
declare(D)          :- ground(D?) | '_declare'(D?).
remove(R)           :- ground(R?) | '_remove'(R?).
```

with `App ::= app(String, String, Integer) ; unattested ; none, Declaration ::= peer(Key) ; address(Key, String) ; rendezvous(Key, String) ; trust(Underlay, TrustLevel) ; place(Constant, Number, Stream(PlaceEvent)?) ; nearby(Underlay, Stream(Presence)?) ; reachable(Stream(Reach)?), Removal ::= peer(Key) ; rendezvous(Key) ; place(Constant) ; nearby(Underlay) ; reachable, Presence ::= seen(Key) ; gone(Key)` *and* `Reach ::= connected(Key) ; lost(Key)` *[53].*

`peer_address` assigns A `address(`S`)`, S the address at which the layer observes peer P, or `none` where it observes no address for P; `peer_app` assigns A `app(`D`, `N`, `V`)`, the developer D, the name N and the version number V of the app attested in the layer's session with peer P, the session reported last where P is connected over several underlays, `unattested` where that session is unattested, or `none` where the layer has no session with P; `self_app` assigns R `app(`D`, `N`, `V`)` for the app whose program the agent runs, which is the app the layer attests to the agent's peers, and I the same for the app installed on the device, the two differing after an update the runtime refused; `punch_udp` opens a path to address A and returns nothing. `declare(peer(`P`))` gives the layer the public key P of a peer the program knows, `remove(peer(`P`))` takes P, and any address recorded for it, off the layer's known peers, and `declare(address(`P`, `A`))` gives the address A at which peer P is reached: who learns an agent's address is the social graph's decision, and the layer holds no social graph. `declare(rendezvous(`P`, `A`))` gives the layer a rendezvous server, of key P at address A, or replaces its address, and `remove(rendezvous(`P`))` takes it off and ends the layer's session with it. `declare(trust(`U`, `L`))` sets the cold-call trust level of the proximity underlay U (`pan` or `lan`) to L (`open` or `closed`). `declare(place(`P`, `R`, `E`))` declares a place P — the region of radius R about the declaring agent's location at the time of the call — and assigns E a stream of that agent's own `entered` and `exited` events for P, carrying also `unobservable` where the layer stops reporting crossings and `observable` where it resumes, between the two seeing no crossing and delivering none late; `remove(place(`P`))` ends the declaration of P, and does nothing where P is not declared. `declare(nearby(`U`, `S`))` assigns S a stream of the agents nearby over the proximity underlay U, beginning with `seen` for every agent nearby when it is declared: `seen(`K`)` when the layer discovers on U an agent of key K that it recognises or, under open trust, contacts, and `gone(`K`)` when that agent can no longer be reached over U; `remove(nearby(`U`))` ends the declaration, and a further declaration on the same underlay supersedes the earlier one; each closes the stream, as for a place. `declare(reachable(`S`))` assigns S a stream of the agents the layer connects

to over the WAN: `connected(`K`)` when the layer has a session with the agent of key K over the WAN, beginning with every agent it has one with when it is declared, and `lost(`K`)` when that session ends; `remove(reachable)` ends the declaration, and a further declaration supersedes the earlier one; each closes the stream, as for a place.

A place event and a presence or a reach on a nearby or reachable stream are bridged below the madGLP layer, as the person's input on the UI channel is (§5.2): the layer reports the event, and the runtime assigns the tail writer of the named place's stream, reactivating whatever suspended on its reader. The stream is fed serializer-fashion, like the network input stream (Definition 6.10): the runtime retains the new tail rather than closing the stream, so one declaration yields one stream however many entries and exits follow. The stream is closed, its tail assigned `[]`, in exactly two cases: `remove(place(`P`))`, and a further declaration of the same place, which supersedes the earlier one. An event for a place removed or superseded is dropped. A declaration the layer refuses is not one of the two: E receives `unobservable`, the declaration stands, and `observable` follows if the layer later begins reporting. A refusal is the layer watching nothing, which is what `unobservable` already says; closing the stream instead would make a refusal permanent, which no refusal is. A refusal answers one declaration, so it is dropped where that declaration has been removed or superseded, as an event for such a place is. Neither an event nor the assignment it causes is a transaction, and neither has a maGLP counterpart; the correctness results concern the transition system, which external input does not enter.

The layer functions behind these predicates are the networking layer's, specified with the seam contract (§C.4).

6.5 Requests and Acknowledgements

Four message kinds cross agent boundaries: a *value* $G := T^{\uparrow}$, a *request* `req`(G) and an *acknowledgement* `ack`(G), these two carrying reader names only, and `gone`(G), which answers traffic for an end its receiver lost (below).

- A request is sent by an agent that localizes a forwarded reader name (Definition 5.10): the anchor did not itself export the name to that agent, and the request is how the anchor learns the reader's current holder.
- On receiving `req`$(_r(p,i))$ from requester s, the anchor p records s among the link's holders; if the value is pending, p addresses it to s as well. Delivery is unordered, so when a reader is forwarded twice the two holders' requests reach p in either order and p cannot tell the current holder from the past one: it sends the value to each, a past holder discards it (below), and the current holder acknowledges. A request repeating a (link, requester) pair already recorded — the link held, authorised or closed — is dropped.
- On receiving a value $_r(p,i) := T^{\uparrow}$, the holder assigns the entry's writer, removes the entry, and acknowledges to the sender. Writer-name values are not acknowledged—a writer-name value always finds its fixed anchor entry—and neither are serializer values, the index-0 entry being permanent. A forwarded

writer needs no request either: $_w(o,k)$ names its destination, so the value reaches the anchor's entry from whichever agent holds the name.
- On receiving $\texttt{ack}(_r(p,i))$ from s, the anchor records s as the holder of every link whose own reader name occurs in the acknowledged value—s localized them—and re-addresses their pending values to s; it then removes the acknowledged value: the link is closed.
- A value, request, or acknowledgement matching no entry and no pending value is stale, and is dropped, where it names a reader the agent has applied, a link the agent has closed, or an epoch of the agent before its current one: a value reaching a holder that forwarded the reader on, a request or acknowledgement for a closed link, traffic for an end the agent retired. Otherwise it reaches an end the agent lost (§6.9), and the agent answers $\texttt{gone}(G)$ to the sender, which retires its own end of the link.

Single occurrence gives a reader one holder at any time, so a link's holders form a chain and exactly one holder acknowledges its value.

6.6 Index-0 Serializer for Cold-Calls

Cold-calls—messages between agents with no prior shared variables—are implemented using a reserved global name at index 0. Each agent p has a well-known global reader $_r(p,0)$ that serves as a *serializer* for incoming cold-call messages.

Definition 6.10 (Index-0 Serializer). *Index 0 in each agent's global writers table is reserved for the* ***network input serializer****:*
- *At boot time, agent p creates a permanent entry (N_p, D_p) at index 0, where N_p is the local writer for p's network input stream and D_p, empty at boot, is the set of $(sender, epoch, serial)$ triples of the cold calls appended so far: a cold call carries a serial its sender allocates in its epoch, and one delivered twice is appended once*
- *The global name $_r(p,0,0)$, written $_r(p,0)$, is the serializer reader, of epoch 0 and of every epoch of p; any agent can send to $_w(p,0)$, and the entry survives every restart of p*
- *Index allocation for regular global links starts at 1*

Remark 6.11 (Cold-Call via Serializer) *To send a cold-call message T to agent q, agent p spawns the goal $\texttt{global_send}(T, _w(q,0), q)$. When T becomes known, the $\texttt{'_send'}$ builtin detects that index is 0 and uses serializer semantics:*
1. *The message sent is $_w(q,0) := [T^{\uparrow} \mid _w(q,0)]$, wrapping T in a list cell with the serializer writer as tail, under a serial i that p allocates from its global-name counter in its epoch e (§C.4), so that (p,e,i) is unique for the whole run as every global name is*
2. *When q receives this message from p and $(p,e,i) \notin D_q$, it assigns $N_q := [T_q^{\downarrow} \mid N'_q]$ where N'_q is a fresh local writer; when $(p,e,i) \in D_q$ the message is a repeat and is dropped*
3. *The entry at index 0 is updated to $(N'_q, D_q \cup \{(p,e,i)\})$, not removed*

This allows multiple agents to send cold-calls to q concurrently; messages are serialized onto q's network input stream in an arbitrary order determined by message arrival, and a message the network delivers twice is appended once.

Remark 6.12 (Network Output Processing) *Each agent spawns a* `send_to_net` *goal to process its network output stream:*

```
send_to_net([msg(Q, T) | In]) :- ground(Q?) |
    '_send'(msg(Q?, T?), '_w'(Q?, 0, 0), Q?), send_to_net(In?).
send_to_net([]).
```

When the agent writes `msg(q, T)` *to its network output stream, the* `send_to_net` *goal observes this and calls* `'_send'`$(\texttt{msg}(q,T), _w(q,0,0), q)$*, sending the whole message to q's index-0 serializer under a fresh serial; the groundness-implying guard* `ground(Q?)` *licenses the multiple occurrences of* `Q` *[53]. The serializer thus appends* `msg`(q,T)*, localized, to q's network input stream, matching the maGLP Cold-call (Definition 5.2).*

Remark 6.13 (Serializer vs Regular Global Links) *For index $i > 0$, the* `'_send'` *builtin sends $G := T^{\uparrow}$ directly and the receiver removes the entry after assignment and acknowledges. For index 0 (serializer), the builtin wraps the term in a list cell $[T^{\uparrow} \mid G]$ reusing the serializer writer, under a fresh serial, and the entry is updated rather than removed. This distinction is the only special case for cold-calls; all other machinery (globalization, localization, message routing) is shared with regular global links.*

Remark 6.14 (Potential Generalisation) *The many-to-one serializer mechanism at index 0 implements an efficient multiway merge for network input. This mechanism could be generalised as a language extension allowing programs to create additional serializers, but we leave such extensions to future work.*

6.7 Held Links

An agent *exchanges* a global name with the agent it sends that name to, or receives it from. A link end at agent p is *held* when the agent at its other end is not the one p exchanged the name with. While a link is held, p delivers nothing on it and applies no assignment delivered on it. A held outgoing message remains in M_p, marked *held*, and a held incoming assignment is retained keyed by its global name. A request from a requester already recorded for the link, and an assignment for a link already holding one, are dropped, so each party is reported once; a request from a further requester — the reader forwarded again — is held and reported once the program has answered the report before it, so that `authorise_link` answers one report at a time, the anchor's program asked once for each party that comes to hold the end; a request whose link closes before its turn is dropped unreported.

A held link is *reported* to the program, and the runtime acts on it only once the program answers. The report is `pending_link`(G, S), with G the link's

global name in its ground presentation (§6.1) and S the agent the name or the traffic came from; it is appended to p's network input stream through the index-0 serializer entry (Definition 6.10), within the transaction that would otherwise have acted. G names the anchor and S the agent p exchanged the end with, so the program has both and the runtime retains nothing further. The report is appended last in that transaction, after anything the transaction itself appends, so a link is reported only once the term carrying it can be read. The report records the module of the goal it reaches, and the record is released with the held link.

Four cases arise:

1. At the holder of a forwarded reader, the request of Definition 5.10 is held. A request is created exactly when the name was forwarded, so every request a holder creates is held.
2. At the holder of a forwarded writer, the value its `global_send` goal produces is held.
3. At the anchor, an arriving request is held. The agent the anchor exported the name to sends no request, so every request the anchor receives is held.
4. At the anchor, an assignment arriving from an agent it did not export the name to is held.

The program answers a report with the system predicate `authorise_link/2`, which takes the global name L of a reported link and an answer A, either `authorise` or `refuse`.

Definition 6.15 (authorise_link Predicate). *The system predicate `authorise_link/2` is defined as:*

```
authorise_link(L, A) :- known(A?) | '_authorise_link'(L?, A?).
```

On `authorise`, the builtin `'_authorise_link'` releases what was held: an outgoing message becomes undelivered, and a retained assignment is applied. On `refuse` it drops the link: the held message is removed from M_p, a retained assignment is discarded, and the link's entry, record, and `global_send` goal at p are released. A refusal is accepted in every case, the program refusing for reasons the runtime does not see.

A global name is a ground term (§6.1), so a program can write one it was never given, and a term arriving from another agent can carry one: the name is not a secret and cannot be made one. A kernel taking a global name from the program therefore acts only on a name the runtime reported to that goal's module, and does nothing otherwise. `authorise_link/2` is the only such kernel at present; any later one inherits the rule. The check matters once an agent runs a module it adopted from another (§6.12): adopted code shares the runtime, and without it could answer for links reported to its host.

A forwarded link is held at both of its ends, so two people are asked and each decides independently. The holder is asked before anything is sent on the link. The anchor is asked only when the forwarded traffic reaches it, after the holder has sent it. For example, Bob sends Alice a variable whose other end is at Carol, and Alice's agent did not exchange its name with Carol. Alice's runtime holds the variable: it sends nothing on it and applies nothing arriving on it, and

reports it to Alice's program. If Alice's program authorises it, its message goes to Carol; Carol's runtime then holds her end too and reports it to Carol's program, which also authorises or refuses it. How Alice's agent comes to know Carol is decided above madGLP.

Remark 6.16 (Authorised Runs) *A refusal drops a link, so the value it would have carried never arrives and the maGLP Communicate it implements is never taken. The correctness results below concern* ***authorised runs****—runs in which every reported link is authorised.*

6.8 madGLP Local States and Transitions

Each madGLP agent executes its local resolvent with FIFO scheduling, tracking three kinds of goals: *active* (ready to reduce), *suspended* (waiting on readers), and *failed*. A reduction succeeds, suspends on a set W of readers, or fails (Definition 3.20); when a reader in W is later instantiated, the suspended goal is reactivated (Definition 3.21).

Definition 6.17 (madGLP Local State). *The local state of agent $p \in \Pi$ is a tuple $s_p = (R_p, W_p, U_p, M_p)$ where:*

1. *$R_p = (A_p, S_p, F_p)$ is a* ***deterministic resolvent****: A_p is a queue of active goals, S_p contains suspended goals paired with blocking readers, and F_p contains failed goals*
2. *W_p is a* ***global writers table***
3. *U_p is a set of* ***imported-writer records*** *(Definition 6.5)*
4. *M_p is a set of* ***outgoing messages****; a delivered reader-name value remains in M_p, marked* ***pending****, until acknowledged, and a held message remains marked* ***held*** *until authorised (§6.7)*
5. *e_p is the epoch of p (Definition 6.1), and E_p is the* ***known epochs*** *of p: a partial function from agents to epochs*

An agent is in one epoch at a time. A global name carries the epoch its anchor was in when it allocated the name, so the name's epoch is fixed. A message carries the epoch its sender is in when it sends it; when q applies a message from p, $E_q(p)$ becomes that epoch, which is never below the epoch $E_q(p)$ held before. Each link end at q records an epoch of the agent at the other end of the link: for an end whose other end is at the anchor, the epoch in the name; for an end at the anchor, the epoch carried by the first request, acknowledgement or value from the holder, and no epoch until one arrives. So a holder that crashes before it has sent on a link is not learnt of by the anchor from its epoch, but from the `gone` its next message on the link draws (§6.5).

Definition 6.18 (Retire). *Agent q* ***retires*** *its end of a link by removing from W_q the entry of the link, or from U_q its record and from R_q the* `global_send` *goal of the link, and from M_q every message of the link.*

A retired end leaves its local variable pair in the resolvent: the writer of a removed entry is never assigned, and the reader of a removed goal is held by no link.

Definition 6.19 (madGLP Transition System). *The **madGLP transition system** over agents $P \subset \Pi$ and a GLP program is the multiagent transition system $madGLP = (C, c_0, T, \sim)$ where C is the set of madGLP configurations, c_0 is the initial configuration with empty resolvents except for the initial* `agent` *goal and index-0 serializer entry, T is the set of Reduce, Send and Input transactions defined below, and two transitions are $\sim$-equivalent when they perform the same Reduce (same agent, goal by identity, clause) or the same Send (same sender, receiver and message); an Input transition is in no class.*

Entries in the global writers table W_p route incoming assignments to local writers: (X, q) at index i binds X to assignments arriving via $_w(p, i)$ from q, and (X_q, p, i) binds X_q to assignments arriving via $_r(p, i)$ from p. Index 0 holds the serializer entry $(N_p, *)$, which is updated (not removed) on each cold-call so that multiple senders can append to the network input stream.

Definition 6.20 (madGLP Reduce Transaction). *The unary Reduce transaction for agent p where $A_p = A \cdot A_r$ transitions $s_p \to s'_p$ with exactly one of the following holding:*

1. ***Reduce:** The GLP reduction of A with the first applicable clause C succeeds with $(B, \hat{\sigma})$. Let $R = \text{reactivate}(S_p, \hat{\sigma}?)$. Then:*
$$A'_p = (A_r \cdot B \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S'_p = S_p \setminus \{(G, W) : G \in R\}, \quad F'_p = F_p$$
2. ***Suspend:** No clause succeeds, but the GLP reduction of A with at least one clause suspends. Let $W = \bigcup_C W_C$ where W_C is the suspension set for clause C. Then:*
$$A'_p = A_r, \quad S'_p = S_p \cup \{(A, W)\}, \quad F'_p = F_p$$
3. ***Fail:** No clause succeeds and no clause suspends. Then:*
$$A'_p = A_r, \quad S'_p = S_p, \quad F'_p = F_p \cup \{A\}$$

Definition 6.21 (madGLP Input Transaction). *The unary Input transaction for agent p assigns a writer X the program handed to the person—the writer of the tail of the person's input stream, or a writer in a message on the person's output stream—a term T whose variables are fresh. Let $\hat{\sigma} = \{X := T\}$ and $R = \text{reactivate}(S_p, \hat{\sigma}?)$. Then:*

$$A'_p = (A_p \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S'_p = S_p \setminus \{(G, W) : G \in R\}, \quad F'_p = F_p$$

Input is the person's act at their own agent, and belongs to no class: the person is under no obligation to act, as a volition change is in [29]. What the person writes may name any agent.

Definition 6.22 (madGLP Send Transaction). *The binary Send transaction from agent p to agent q is enabled when $(m, q) \in M_p$ is undelivered and not held (§6.7). It removes (m, q) from M_p—a reader-name value instead remains in M_p, marked **pending**, until acknowledged (§6.5)—and applies m at q, with exactly one of the following holding:*

1. ***Serializer (**$m = (_\,w(q, 0) := [T^{\uparrow} \mid _\,w(q, 0)])$ **under serial** i **of** p**'s epoch** e**):** q holds the entry (N_q, D_q) at index 0 in W_q. If $(p, e, i) \in D_q$, m is a repeat: nothing changes at q. Otherwise let N'_q be a fresh local writer, $\hat{\sigma} = \{N_q := [T_q^{\downarrow} \mid N'_q?]\}$, and $R = \text{reactivate}(S_q, \hat{\sigma}?)$. Then:*
$$A'_q = (A_q \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S'_q = S_q \setminus \{(G, W) : G \in R\}, \quad F'_q = F_q$$

The entry at index 0 in W'_q becomes $(N'_q, D_q \cup \{(p, e, i)\})$.

2. ***Writer*** *($m = (_\ w(q,i) := T^{\uparrow})$, $i > 0$):* *q holds the entry (X, o) at index i in W_q; p need not be o, the name having been forwarded (Definition 5.9), and when $p \neq o$ the assignment is held at q and applied on authorisation (§6.7). Let $\hat{\sigma} = \{X := T^{\downarrow}_q\}$ and $R = \text{reactivate}(S_q, \hat{\sigma}?)$. Then:*
$$A'_q = (A_q \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S'_q = S_q \setminus \{(G, W) : G \in R\}, \quad F'_q = F_q$$
The entry at index i is removed from W'_q.
3. ***Reader*** *($m = (_\ r(o,i) := T^{\uparrow})$):* *$q$ is the agent that localized $_\ r(o,i)$, holding the entry (X_q, o, i) in W_q. Let $\hat{\sigma} = \{X_q := T^{\downarrow}_q\}$ and $R = \text{reactivate}(S_q, \hat{\sigma}?)$. Then:*
$$A'_q = (A_q \cdot R)\hat{\sigma}\hat{\sigma}?, \quad S'_q = S_q \setminus \{(G, W) : G \in R\}, \quad F'_q = F_q$$
The entry matching (o, i) is removed from W'_q, and $(\texttt{ack}(_\ r(o,i)), p)$ is added to M'_q.
4. ***Request*** *($m = \texttt{req}(_\ r(q,i))$):* *$q$ is the anchor. A request repeating a (link, requester) pair already recorded is dropped, the link held, authorised or closed. Otherwise the request is held (§6.7); on authorisation q records p among the holders of the link $_\ r(q,i)$, and if a pending value addressed by $_\ r(q,i)$ is in M_q, a copy addressed to p is added, undelivered (§6.5). A request for a closed link is dropped.*
5. ***Acknowledgement*** *($m = \texttt{ack}(_\ r(o,i))$):* *$q$ is the sender of the acknowledged value; let V be the pending value addressed by $_\ r(o,i)$ in M_q. q records p as the holder of every link whose reader name anchored at q occurs in V, re-addressing any pending value of such a link to p; it then removes V from M_q. An acknowledgement for a closed link is dropped.*
6. ***Gone*** *($m = \texttt{gone}(G)$):* *q retires its end of the link G, where it has one (Definition 6.26): the entry, or the record and the `global_send` goal, and the messages of the link in M_q. Nothing else changes at q.*

In every case $E'_q(p)$ is the epoch m carries, and q retires (Definition 6.18) every link end whose other end is at p and whose recorded epoch of p is below it.

Remark 6.23 (Message Delivery) *Send p to q is the delivery of m: a Send class that stays enabled is taken in a correct run, as every class is, so every message is delivered. The binary Send is to madGLP what Communicate and Cold-call are to maGLP; the index-0 serializer lets several senders append to one agent's network input stream. The network below may carry a message more than once, the layer retransmitting after a lost acknowledgement. A repeat is harmless everywhere but at the serializer: a value finds its entry removed, a request or acknowledgement for a closed link is dropped (§6.5), a request repeating a recorded requester or an assignment for a link already holding one is dropped (§6.7); the serializer appends a cold call once, by its serial (Definition 6.10).*

6.9 Termination and Restart

The platform suspends an agent's application, as a phone does when the application goes to the background, and may then terminate it; a suspended application

may also resume. The networking layer's queue of undelivered messages survives a termination, and the layer delivers it when the application restarts (the networking companion paper, Networking Assumptions).

Definition 6.24 (Suspension, Termination, Crash). *An application is* ***suspended*** *from the moment the platform announces that it may terminate it until it resumes or terminates; a suspended agent takes no transition. An application* ***terminates*** *when it ends while suspended, and* ***crashes*** *when it ends otherwise.*

Definition 6.25 (Restart). *The termination of agent p's application and its restart is not a transition of madGLP: the local state s_p of p (Definition 6.17) at restart is its local state at termination.*

A madGLP run therefore continues across a restart, and every result of this section holds across restarts: a message the layer delivers after a restart finds the entry its localization created, a pending value is still pending, and a Send class that was enabled is enabled still. The runtime keeps the local state across a termination by saving it at the suspension (Persistence, §C.4), and a suspended agent takes no transition, so the state saved is the state at termination.

A crash loses the state since the last suspension. The agent restarts from the state saved at that suspension, in a new epoch, with every link end retired.

Definition 6.26 (Restart after a Crash). *The local state of agent p at a restart after a crash is its local state at its last suspension, or its initial local state where it was never suspended, with e_p incremented by one and every link end of p retired (Definition 6.18) but the serializer entry.*

So the epochs of an agent grow, and a name of p allocated in one epoch is allocated again in no later one.

After a crash every link between p and its peers ends. At p, the ends are retired at the restart, and the paired readers and writers are widowed (`widowed_reader/1` and `widowed_writer/1`, the companion GLP paper [53]), so a program waiting on one learns it. At a peer q, the end is retired when q learns the new epoch of p, from the first message it receives from p after the restart (Definition 6.22), or when q sends on the link and p answers `gone` (§6.5). What p did between the suspension and the crash is lost at p and known to the peers that received its messages: a state loss of the platform the agent runs, which the platform recovers from the peers as [13] specifies, above madGLP, making new links in place of the ended ones.

A restart may load a new version of the agent's program, as when the application is updated. The local state is then restored under the new version when every procedure called by a goal of the resolvent is defined in the new version with the same type identity [55]; otherwise the update is refused, and the agent restarts under the version it terminated under, which is the version the layer then attests. A run across an update is a run under the earlier version up to the restart and under the new version after it. A developer keeps an update compatible by keeping those procedures and their types.

The person's screen is not part of the local state. At a restart the runtime grants `refresh`, an Input addressed to no construct, in the person's place, where

it holds the person writer of a person channel of the root's Input type; on it the dispatcher has every construct draw again what it last drew [56], so the screen is drawn from the restored state.

Remark 6.27 (Globalize-Localize Correspondence) *The pairing between Globalize and Localize ensures correct dataflow:*

- ***Writer globalized at*** *p: Globalize creates entry* (Y, q) *at* p*. Localize records* (Y_q, p, i, p)*, spawns* `global_send`$(Y_q?, _w(p,i), p)$ *at* q*, and puts* Y_q *(writer) in* q*'s term. When* q *assigns* Y_q*, the spawned goal fires and sends the value to* p*, where the entry routes it to* Y*, making it available via* $Y?$*.*
- ***Reader globalized at*** *p: Globalize spawns* `global_send`$(Y?, _r(p,i), q)$ *at* p*. Localize adds entry* (Z_q, p, i) *at* q *and puts* $Z_q?$ *(reader) in* q*'s term. When* p *assigns* Y*, the spawned goal fires and sends the value to* q*, where the entry routes it to* Z_q*, making it available via* $Z_q?$*.*

6.10 madGLP Remarks

Remark 6.28 (Outgoing Messages via global_send) *The Reduce transaction does not directly generate outgoing messages. Instead, when a writer* X *is assigned,* $X?$ *becomes known, which may trigger a* `global_send` *goal watching* $X?$*. That goal's reduction (via the* `'_send'` *builtin) adds the message to* M_p*. This uniform approach handles both direct sends and forwarding.*

Remark 6.29 (Automatic Forwarding) *Send* p *to* q *assigns the local writer at* q*, removes the entry, and acknowledges. If the assigned writer's reader (*$X?$*) is being watched by a* `global_send` *goal (because it was also exported), that goal will fire on a subsequent Reduce, automatically forwarding the value. No special forwarding logic is needed in Send.*

Remark 6.30 (Transaction Degrees) *Reduce and Input are unary: Reduce operates on a single agent's resolvent, and Input writes the person's term at their agent. Send is binary: it takes a message from the sender's outgoing messages and applies it at the receiver, as maGLP's Communicate and Cold-call move an assignment or a message between their two participants. A cold call is a Send to the receiver's index-0 serializer.*

Remark 6.31 (Determinism) *madGLP is deterministic at the agent level: given a local state, exactly one Reduce transition is enabled (FIFO goal selection, first-clause matching). However, madGLP is nondeterministic at the system level due to communication asynchrony: the order in which messages arrive at an agent depends on network timing, and different arrival orders may lead to different computations.*

Remark 6.32 (Correspondence to maGLP) *The maGLP transactions are implemented in madGLP as follows:*

- ***Communicate:*** *The maGLP binary Communicate transaction, which atomically transfers an assignment from one agent's writer to another agent's reader, is implemented by the sequence: Reduce (assigns writer, triggering* `global_send`*)* $\rightarrow$ *Send p to q (applies the assignment at q).*
- ***Cold-call:*** *The maGLP binary Cold-call transaction is implemented using the index-0 serializer: the sender spawns* `global_send`*$(T, _w(q,0), q)$, which fires when T is known, followed by Send p to q (the serializer appends to q's network input stream).*
- ***Input:*** *The maGLP Input transaction is implemented by madGLP's Input, which applies the assignment at once, as Reduce applies $\hat{\sigma}$?.*

The correctness of this implementation is proved below.

Remark 6.33 (Global Writers Table Lifecycle) *An entry is added to the global writers table when a global link is established with the agent as the receiver: either when globalizing a writer (expecting an incoming assignment from the remote agent that received the writer) or when localizing a reader global name (expecting an incoming assignment from the globalizer). The entry is removed when the assignment arrives and the writer is assigned. The table tracks only those writers awaiting incoming assignments from remote agents. An imported-writer record has the mirror lifecycle: added when a writer global name is localized, removed when the value is sent, or when the writer is re-exported and its name forwarded.*

Remark 6.34 (Message Routing) *Messages use global names to enable routing:*

- *Message $_w(p,i) := T$ is sent to p (the original globalizer who exported a writer and created the entry); p has an entry at index i in its global writers table*
- *Message $_r(p,i) := T$ is sent to whoever localized $_r(p,i)$; that agent searches for an entry with remote agent p and remote index i*

6.11 Canonical Encoding

The Send transaction delivers a message from one agent to another (Definition 6.22); over a network, it travels as a byte string. Signatures and hashes of terms and artefacts are computed over these bytes independently at different agents and must agree byte for byte; the encoding is therefore part of the specification. What is signed and hashed is given in §B.4 and §B.7.

Definition 6.35 (Canonical Encoding). *A* ***canonical encoding*** *is an injective function e from globalized terms and assignment messages to byte strings.*

Injectivity makes every message decodable, as localization requires. The canonical encoding has the following properties:

1. *Hardware-independent:* integer widths and byte order are pinned; a term yields the same byte string on every machine.

2. *Address-free:* in a globalized term every variable has been replaced by a global name (Definition 5.9), so a variable appears in the encoding as its global name—agent and index—with its reader/writer polarity; no heap address appears.
3. *Globalizer's names:* the encoding is of the globalized term exactly per Definition 5.9—names are the allocating agent's: a forwarded reader keeps its original anchor's name.
4. *Canonical on ground terms:* globalization is the identity on ground terms, so a ground term T has one encoding $e(T)$, identical at every agent—the byte string over which terms are signed and hashed.

The byte-level layout of e—primitive encodings, term and message grammar, the instruction encoding, the artefact, and the loader—is given in Appendix B.

6.12 Compiled Programs as Shippable Values

madGLP ships code the way it ships terms: a compiled program is a value of the primitive type Module. A GLP program is a hierarchy of modules [55]; the sender links it into a single flat program and compiles it to an artefact, carried by a Module constant. One canonical encoding (Definition 6.35) thus serves terms and code. The compiled program ships inside an assignment message like any other value, and the receiving agent loads it at runtime; a goal is posted to a loaded module by `run/3`, whose type discipline is the module system's [55]. The correctness theorems of this paper fix the program; runtime loading sits outside them.

The engine follows the sequential abstract machine of Flat Concurrent Prolog [24]: a program is compiled to a string of instructions that the engine interprets directly. GLP departs only where the languages differ—the instruction set is GLP's (it matches rather than unifies and pairs each reader with a writer), and the byte order is fixed to one canonical order rather than the running machine's, so an artefact's bytes are read the same way on every machine and are what its compiled identity hashes. A recipient whose machine uses a different order inverts the bytes locally before executing and inverts them again before shipping the program onward, leaving the shipped, hashed bytes unchanged (Appendix B). A program is shipped on adoption (§B.7) and named by the identities of §B.4; its receiver relies on the certificate's key, one it accepts or does not [53].

6.13 Correctness Proofs

Lemma 6.36 (Globalize-Localize Correspondence) *Let T be a term with variables at agent p, let $T_p^{\uparrow}$ be its globalization by p for remote agent q, and let $T_q^{\downarrow}$ be the localization by q. Then:*

1. *For each writer Y in T: globalization creates entry (Y, q) at index i in p's global writers table; localization creates fresh pair $(Y_q, Y_q?)$ at q and spawns* `global_send`$(Y_q?, _\, w(p,i), p)$. *These form a global link implementing shared pair $(Y, Y?)$.*

2. *For each reader $Y?$ in T: globalization spawns* `global_send`$(Y?, _r(p,i), q)$ *at p; localization creates fresh pair $(Z_q, Z_q?)$ at q with entry (Z_q, p, i). These form a global link implementing shared pair $(Y, Y?)$.*
3. *The structure of T is preserved: $T_q^{\downarrow}$ has the same functor/arity structure as T, with each variable replaced by its corresponding local variable at q.*
4. *For each unbound imported reader $Y?$ in T with entry (Y, o, k) at p: globalization by p emits the original name $_r(o,k)$ and removes p's entry; localization at q creates a fresh pair with an entry recording (o,k) and sends* `req`$(_r(o,k))$ *to o. The link's anchor is unchanged and its holder is now q.*
5. *For each imported writer Y in T with record (Y, o, k, s) at p: globalization by p emits the original name $_w(o,k)$ and removes p's record and the* `global_send` *goal watching $Y?$; localization at q creates a fresh pair with the record (Y_q, o, k, p) and spawns* `global_send`$(Y_q?, _w(o,k), o)$. *The link's anchor is unchanged and its sending end is now q.*

Proof. By the definitions of Globalize (Definition 5.9) and Localize (Definition 5.10):

For (1): When Y is a writer in T, globalization replaces Y with $_w(p,i)$ and creates entry (Y, q) at index i in p's global writers table. Localization at q sees $_w(p,i)$, creates fresh pair $(Y_q, Y_q?)$, spawns `global_send`$(Y_q?, _w(p,i), p)$, and replaces $_w(p,i)$ with Y_q (writer) in $T_q^{\downarrow}$. When q assigns Y_q, the value becomes available via $Y_q?$, the `global_send` goal fires and sends message $_w(p,i) := T^{\uparrow}$ to p. Send q to p finds entry (Y, q) at index i at p and assigns $Y := T^{\downarrow}$, making the value available via $Y?$. This matches maGLP semantics where the exported writer is assigned by the remote agent.

For (2): When $Y?$ is a reader in T, globalization replaces $Y?$ with $_r(p,i)$ and spawns `global_send`$(Y?, _r(p,i), q)$ at p. Localization at q creates fresh pair $(Z_q, Z_q?)$, creates entry (Z_q, p, i), and replaces $_r(p,i)$ with $Z_q?$ (reader) in $T_q^{\downarrow}$. When p assigns Y, the value becomes available via $Y?$, the `global_send` goal fires and sends $_r(p,i) := T^{\uparrow}$ to q. Send p to q finds entry (Z_q, p, i) at q and assigns $Z_q := T^{\downarrow}$, making the value available via $Z_q?$. This implements the semantics where the exported reader receives the value from the globalizer.

For (3): Both globalization and localization traverse T structurally, replacing only variables while preserving all functors and their arities.

For (4): By the reader-forwarding case of Definition 5.9 and by Definition 5.10, the entry moves with the reader occurrence while the name and its anchor are fixed. The request reaches o, its Send being live; o redirects the pending value, or sends it once produced, to q; q's entry consumes it and q acknowledges, closing the link (§6.5). Single occurrence makes the holder unique at any time, so exactly one acknowledgement closes the link.

For (5): By the writer-forwarding case of Definition 5.9 and case 1 of Definition 5.10, the record and its goal move with the writer occurrence while the name and its anchor are fixed. Single occurrence makes the sending end unique at any time, and $_w(o,k)$ names its destination, so the value q produces reaches o's entry in one message, without request or acknowledgement.

Lemma 6.37 (Both Ends Exported) *When agent p globalizes term T containing both X and $X?$ for agent q, the resulting madGLP configuration correctly implements the maGLP semantics where both ends of the pair are available to q.*

Proof. Globalization of $T = [\ldots, X, \ldots, X?, \ldots]$ at p for q proceeds as follows:
- For writer X: entry (X, q) at index 1, no spawn, term gets $_w(p, 1)$
- For reader $X?$: spawns `global_send`$(X?, _r(p, 2), q)$, no entry, term gets $_r(p, 2)$

Localization at q creates:
- For $_w(p, 1)$: pair $(Y_q, Y_q?)$, record $(Y_q, p, 1, p)$, spawns `global_send`$(Y_q?, _w(p, 1), p)$, term gets Y_q (writer)
- For $_r(p, 2)$: pair $(Z_q, Z_q?)$, entry $(Z_q, p, 2)$, term gets $Z_q?$ (reader)

The term at q is $[\ldots, Y_q, \ldots, Z_q?, \ldots]$ with two independent pairs. Agent q does not know they are connected—that connection exists only through p's local pair $(X, X?)$.

When q assigns $Y_q := T$:
1. $Y_q?$ becomes known
2. `global_send`$(Y_q?, _w(p, 1), p)$ fires, sends $_w(p, 1) := T^{\uparrow}$ to p
3. Send q to p finds entry (X, q) at index 1 at p and assigns $X := T^{\downarrow}$
4. $X?$ becomes known at p
5. `global_send`$(X?, _r(p, 2), q)$ fires, sends $_r(p, 2) := T^{\uparrow}$ to q
6. Send p to q finds entry $(Z_q, p, 2)$ at q and assigns $Z_q := T^{\downarrow}$
7. $Z_q?$ becomes known at q

The assignment to Y_q flows through p's local pair to $Z_q?$. The same analysis applies when X and $X?$ are sent to different agents.

The Implementation Mapping.

Definition 6.38 (Reconstructed Resolvent). *Given a madGLP local state (R_p, W_p, U_p, M_p) where $R_p = (A_p, S_p, F_p)$, the **reconstructed resolvent** at agent p is:*

$$G_p = A_p \cup \{A : (A, W) \in S_p\} \cup F_p$$

Definition 6.39 (Reconstructed Readers Substitution). *Given a madGLP configuration c', the **reconstructed readers substitution** σ_p for agent p is derived from undelivered values: for each value produced at p whose link is still open—undelivered, or pending with its addressed entry not yet consumed—include the corresponding reader assignment in σ_p according to the variable correspondence.*

Definition 6.40 (Implementation Mapping π). *The implementation mapping π from madGLP configurations to maGLP configurations is defined as follows. For each agent p:*

$$\pi_p(R_p, W_p, U_p, M_p) = (G_p, \sigma_p)$$

where G_p is the reconstructed resolvent (Definition 6.38) with `global_send` goals removed and each local variable replaced by its corresponding maGLP variable according to the variable correspondence (Definition 5.11)—the two local pairs of

a global link map to one maGLP variable, a variable of the link's anchor (Definition 5.1), as its global name is keyed by that agent—and σ_p is the reconstructed readers substitution (Definition 6.39).

The global mapping $\pi : C' \to C$ applies this construction to each agent's local state.

The mapping π forgets:
- The partition of goals into active, suspended, and failed
- The suspension sets W for suspended goals
- The global writers tables W_p and the imported-writer records U_p (routing information)
- The message queues M_p (except as they contribute to σ)
- The `global_send` goals (implementation mechanism)
- The pairing of local variables—corresponding variables at different agents map to the same maGLP variable

Transaction Correspondence.

Proposition 6.41 (Reduce Implementation) *The madGLP Reduce transaction for agent p projects via π to a maGLP Reduce transaction for p followed by the Communicate p to p transactions of the values it applies to the goals holding their readers.*

Proof. madGLP Reduce selects the head of the active queue and attempts reduction with the first applicable clause. If reduction succeeds with $(B, \hat{\sigma})$, the body takes the head writers' values and the goal writers' values are applied to the local goals holding their readers, which maGLP's Reduce and its Communicate p to p do; a goal writer whose reader is at another agent has its value sent by a `global_send` goal (Proposition 6.43). π maps the result to the result. If Suspend or Fail, the goal is reclassified but remains in $G_p = \pi(R_p)$.

Proposition 6.42 (Input Implementation) *The madGLP Input transaction for agent p projects via π to a maGLP Input transaction for p followed by the Communicate p to p of its assignment.*

Proof. madGLP Input assigns the person's writer $X := T$ and applies $\hat{\sigma}?$ to p's goals, reactivating those suspended on $X?$; maGLP Input adds $\{X? := T\}$ to σ_p and Communicate p to p applies it to G_p. π maps the result to the result.

Proposition 6.43 (Communicate Implementation) *maGLP's binary Communicate transaction from p to q is correctly implemented by madGLP's mechanism: a `global_send` goal fires (via Reduce) at the writer-owner, then Send p to q applies the assignment at the reader-owner.*

Proof. maGLP Communicate(p, q) transfers assignment $\{X? := T\}$ from σ_p to G_q. In madGLP: (1) the writer-owner's Reduce assigns the local writer; (2) a subsequent Reduce fires the `global_send` goal watching the paired reader, adding message $(G := T^{\uparrow})$ to the outbox; (3) Send p to q takes the message from M_p, and at the reader's current holder finds the entry in its global writers

table, localizes, applies the assignment, and acknowledges, closing the link. If the reader was forwarded, the holder's request precedes or redirects the send (§6.5); a value reaching a past holder is dropped and the pending value is redirected to the current one. By Lemma 6.36, π maps the result to the maGLP Communicate result.

Proposition 6.44 (Cold-call Implementation) *maGLP's binary Cold-call transaction from p to q is correctly implemented by madGLP's Send transaction via the index-0 serializer.*

Proof. maGLP Cold-call(p, q) transfers $\texttt{msg}(q, X)$ from p's network output stream to q's network input stream, establishing shared variable pairs. In madGLP: p globalizes $\texttt{msg}(q, X)$ (Remark 6.12); Send p to q delivers the serializer message, which q localizes and appends to its network input stream; global links for the variables of X are established. By Lemma 6.36, π maps the result to the maGLP Cold-call result.

Lemma 6.45 (π is Correct) *The implementation mapping π is correct.*

Proof. Each agent's madGLP execution is deterministic at the local level (FIFO scheduling). For any correct madGLP run r', every madGLP Reduce that becomes enabled is eventually taken. Each such Reduce projects to a maGLP Reduce via π. A produced value stays pending at its producer until acknowledged; the current holder's entry exists from its localization, and the producer learns the holder from its own export, from the holder's request when the name was forwarded (the Send of the request being live), or from the acknowledgement of the enclosing value when that value was redirected (Definition 6.22); the Send of the value to that holder is then enabled, taken, and acknowledged—so every maGLP Communicate class that stays enabled is eventually taken. A madGLP Input projects to a maGLP Input, in no class in either. Since madGLP can perform any set of reductions that maGLP can (by Lemma 6.46), and, since r' is correct, every enabled equivalence class is eventually taken, $\pi(r')$ is a correct maGLP run.

Outcome-Completeness.

Lemma 6.46 (madGLP Performs Equivalent Reductions) *For every proper maGLP run r performing reductions $\{R_1, \ldots, R_k\}$, there exists a madGLP run r' performing equivalent reductions.*

Proof. By induction on spawn order. Goals in each agent's initial goal are in the initial queue. For any R_i reducing A_i at agent p with C_i: if A_i was spawned by an earlier reduction R_j, by induction madGLP performs R_j, so A_i enters p's queue. Since R_i succeeds in r, A_i and the head of C_i have a writer mgu. Since the writer mgu assigns only writers (Definition 3.5), it is invariant under any reader substitutions applied to A_i in the intervening run. When A_i reaches the queue head, the reduction succeeds.

Lemma 6.47 (madGLP Outcome-Completeness) *For every proper maGLP run with outcome O, there exists a madGLP run with outcome O.*

Proof. Let r be a proper maGLP run with outcome O, performing reductions $\{R_1, \ldots, R_k\}$.

By Lemma 6.46, some madGLP run r' performs equivalent reductions.

By Lemma 3.17 and Lemma 3.18, since r and r' perform equivalent reductions (with disjoint substitutions that commute), they have the same outcome O.

Correctness. Global links implement maGLP shared variable pairs via message passing; Definition 5.11 treats the case of a writer at p and reader at q (Section 5.2), with the mirror case alongside. The implementation mapping π from madGLP configurations to maGLP configurations reconstructs the resolvent and readers substitution while forgetting implementation details (goal classification, global writers tables, message queues, `global_send` goals).

Correctness rests on *disjoint substitution commutativity*, a consequence of the single-occurrence invariant: reductions at distinct goals assign disjoint variables and therefore commute, so the agent-deterministic, message-passing execution reaches the same outcomes as maGLP's nondeterministic shared-variable semantics.

Theorem 6.48 (madGLP Correctly Implements maGLP) *For authorised runs (Remark 6.16), the implementation $(madGLP, \pi)$ of maGLP is correct and outcome-complete.*

Proof. Correct: By Lemma 6.45. *Outcome-complete:* By Lemma 6.47.

Corollary 6.49 (Global Links Correctly Implement Shared Variables) *Two local pairs joined by a global link together with its `global_send` goal correctly implement a maGLP shared variable pair, including the case where both ends of a pair are exported.*

Proof. Follows from Theorem 6.48 together with Lemma 6.36 and Lemma 6.37.

7 AI Development Methodology

This paper offers a precise interface between the human designer and the AI programmer: formal operational semantics serve as a specification language, constraining the space of legal implementations. The emerging discipline [15,42,7] we advocate and employ is for the human designer and AI to jointly develop and agree upon (*i*) formal semantics (this paper), (*ii*) an informal (English+code) specification derived by AI from the math, and only then let AI attempt to write (*iii*) Dart code that complies with both.

For example, the spec defines outgoing communication between madGLP agents through a system predicate:

```
global_send(T, G, Q) :- known(T?) | '_send'(T?, G?, Q?).
```

The guard `known(T?)` succeeds once T is assigned; the builtin `'_send'` globalizes T and queues the message $(G := T^{\uparrow}, Q)$ for delivery to Q.

The AI used throughout is Claude (Anthropic). While authority flows math→spec→Dart, the three layers were harmonised via extensive back-and-forth: running the Dart implementation and deriving the specification surfaced defects, several of which proved to be at the mathematical level rather than the code level, and human review caught a conceptual error the formalism had absorbed from an earlier draft. The remarks in this paper record design rationale developed during spec derivation. The English+code specifications served the derivation; their content is now absorbed into this paper's appendices.Appendix D records the substantive revisions and the diagnostics that drove them.

Appendix D gives the full account.

8 madGLP is Grassroots

A protocol is **grassroots** [29] if any two disjoint groups of agents can each run it independently—every interleaving of their independent correct runs is a correct run of the combined group (*obliviousness*)—yet two groups that have not interacted can always do so (*interactivity*). The maGLP protocol over a program that names no agent and uses cold calls is grassroots [53]: an agent is a constant, a program *names no agent* if none occurs in it, and it *uses cold calls* if, from every reachable local state, the person's naming an agent q is followed by reductions after which the program cold-calls q. By the *madGLP protocol* over a program we mean the family with one madGLP transition system per finite set of agents P (Definition 6.19), on the local states in which no agent outside P occurs, and likewise for maGLP.

Theorem 8.1 (madGLP is Grassroots) *The madGLP protocol over a program that names no agent and uses cold calls is grassroots.*

8.1 Protocols and the Grassroots Property

We recall the definitions from [29], and the person's input from [53]. A protocol is a family of multiagent transition systems, one for each finite set of agents $P \subset \Pi$, sharing an underlying set of local states with a designated initial state.

Definition 8.2 (Local-States Function [29]). *A **local-states function** $S : 2^{\Pi} \to 2^{\mathcal{S}}$ maps every finite set of agents $P \subset \Pi$ to a set of local states $S(P) \subset \mathcal{S}$ that includes s_0 and satisfies $P \subset P' \implies S(P) \subset S(P')$.*

Given a local-states function S, we use $C(P) := S(P)^P$ for configurations over P and $c_0(P) := \{s_0\}^P$ for the initial configuration.

Definition 8.3 (Protocol [29]). *A **protocol** $\mathcal{F}$ over a local-states function S is a family of multiagent transition systems with exactly one transition system $\mathcal{F}(P) = (C(P), c_0(P), T(P), \sim(P))$ for every finite $P \subset \Pi$.*

A *correct* run of a transition system is safe and live (Definition 2.1); the runs here are safe by construction, so correctness coincides with liveness.

Lemma 8.4 (Extension [29]) *Every finite safe run of a transition system is a prefix of a correct run of it.*

To define grassroots formally, we first define the interleaving of runs of two disjoint groups.

Definition 8.5 (Interleaving [29]). *Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathcal{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathcal{F}(P')$. An **interleaving** of r and r' is a sequence $e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k\geq 0}$ and $(j_k)_{k\geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:*
1. *$(e_k)_p = (c_{i_k})_p$ for every $p \in P$,*
2. *$(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,*
3. *if e_{k+1} exists, then exactly one of: (a) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$ (a P-step), or (b) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$ (a P'-step).*

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

An interleaving is well-defined: since $P \subset P \cup P'$, the local-states function ensures $S(P) \subset S(P \cup P')$, and similarly for P', so each e_k is a valid configuration in $C(P \cup P')$. Also, $e_0 = c_0(P \cup P')$, since $(e_0)_p = (c_0)_p = s_0$ for all $p \in P$ and $(e_0)_q = (d_0)_q = s_0$ for all $q \in P'$.

Definition 8.6 (Interaction, Interactive Run [29]). *Let $Q \subset \Pi$. For a configuration c over Q and $R \subseteq Q$, write $c|_R$ for the restriction of c to R. A transition $e \to e'$ of $\mathcal{F}(Q)$ is an **interaction between** $x \neq y \in Q$ if $e|_{\{x\}} \neq e'|_{\{x\}}$ and $e|_{\{y\}} \neq e'|_{\{y\}}$, and $e|_{\{x\}} \to e'|_{\{x\}}$ is not a transition of $\mathcal{F}(\{x\})$ or $e|_{\{y\}} \to e'|_{\{y\}}$ is not a transition of $\mathcal{F}(\{y\})$. It is an **interaction between** disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$. A run $\hat{r}$ of $\mathcal{F}(Q)$ is **interactive between** P **and** P' if some transition of $\hat{r}$ is an interaction between them.*

Definition 8.7 (Oblivious, Interactive, Grassroots [29]). *A protocol $\mathcal{F}$ is:*
1. ***oblivious** if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathcal{F}(P)$ and a correct run of $\mathcal{F}(P')$ is a correct run of $\mathcal{F}(P \cup P')$.*
2. ***interactive** if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathcal{F}(Q)$ that contains one.*
3. ***grassroots** if it is oblivious and interactive.*

In an interleaving, each step changes the local states of agents in only one group; therefore, any transaction whose active participants span both groups yields a step that cannot occur in any interleaving. The interleaving and obliviousness definitions are the machine-layer specialisation, without guards, of the volitional grassroots framework of [29].

8.2 Transactions-Based Grassroots Protocols

Definition 8.8 (Transactions Over a Local-States Function [29]). *Let S be a local-states function. A set of transactions R is **over** S if every transaction $t \in R$ is a transition over participants Q and local states $S(Q')$ for some $Q \subseteq Q' \subset \Pi$. Given such R and $P \subset \Pi$:*

$R(P) := \{t \in R : t \text{ has participants } Q \subseteq P \text{ and is over } S(Q') \text{ for some } Q' \subseteq P\}$

A transaction of agents of P from local states of P may reach a local state of a larger set of agents, as when p adds to its state an agent outside P. Such a transaction is in $R(P \cup P')$ and not in $R(P)$, so an interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$ may owe it a liveness obligation that the run of $\mathcal{F}(P)$ does not owe [29]. It is excluded as follows.

Definition 8.9 (Contained [29]). *A set of transactions R over a local-states function S is **contained** if for every $P \subset \Pi$, every $t = c \to c'$ in R with participants $Q \subseteq P$ and $c \in S(P)^Q$ is in $R(P)$.*

Definition 8.10 (Transactions-Based Protocol [29]). *Let S be a local-states function and R a set of transactions over S, in which distinct transactions have disjoint P-closures for every $P \subset \Pi$, with a partial equivalence $\sim$. The **protocol $\mathcal{F}$ over R, S, and $\sim$** is defined by $\mathcal{F}(P) := (C(P), c_0(P), R(P)\uparrow P, {\sim}\uparrow P)$ for each $P \subset \Pi$, where $R(P)\uparrow P$ and ${\sim}\uparrow P$ are the P-closures (Definition 4.4) of $R(P)$ and of $\sim$ restricted to $R(P)$.*

The person's input (Definition 5.2, Definition 6.21) belongs to no class and may bring into an agent's state the name of an agent outside its group; it is added to a transactions-based protocol as in [53].

Definition 8.11 (Transactions-Based Protocol with Input [53]). *Let $\mathcal{F}$ be the protocol over R, S, and $\sim$, and I a set of unary transactions over S, each $t \in I$ over one participant p and $S(Q')$ for some $Q' \ni p$. The **protocol over R, S, $\sim$ and input** I is $\mathcal{F}^I$ with $\mathcal{F}^I(P) := (C(P), c_0(P), R(P)\uparrow P \cup I(P)\uparrow P, {\sim}\uparrow P)$ for each $P \subset \Pi$; a transition of $I(P)\uparrow P$ is in no class.*

Proposition 8.12 (Obliviousness Condition with Input [53]) *A transactions-based protocol with input over a contained set of transactions is oblivious provided that for every disjoint nonempty $P, P' \subset \Pi$, no class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathcal{F}^I(P)$ and $\mathcal{F}^I(P')$.*

Theorem 8.13 (Transactions-Based Grassroots [53]) *A transactions-based protocol with input that satisfies the condition of Proposition 8.12 and is interactive is grassroots.*

Proof (Proof of Theorem 8.1). Let M name no agent and use cold calls, and $\mathcal{F}$ be the madGLP protocol over M: for a finite $P \subset \Pi$, $S(P)$ is the set of madGLP local states in which no agent outside P occurs, an agent occurring in a local state as a constant in a goal, as the anchor of a global name, in an entry of W_p or a record of U_p, or as the addressee of a message in M_p; and

$\mathcal{F}(P)$ is the madGLP transition system over P and M (Definition 6.19) with its configurations restricted to $S(P)^P$. By Theorem 8.13, it suffices to show that $\mathcal{F}$ is a transactions-based protocol with input over a contained set of transactions, that it satisfies the condition of Proposition 8.12, and that it is interactive.

Transactions-based with input: Let R be the Reduce and Send transactions of Definition 6.19 over every finite set of agents, I its Input transactions, and $\sim$ its equivalence. Distinct transactions have disjoint closures (Section 4): two over the same participants differ on them, and a Send, which removes a message from the sender's M_p or marks it pending, agrees on the sender with no Reduce or Input, which leave M_p as it is. A transaction among agents of P between configurations of $S(P)^P$ is over $S(P)$, so $\mathcal{F}(P)$ is $(S(P)^P, \{s_0\}^P, R(P)\!\uparrow\! P \cup I(P)\!\uparrow\! P, \sim\!\uparrow\! P)$, and $\mathcal{F}$ is the protocol over R, S, $\sim$ and input I.

Contained: Let $t \in R$ have participants $Q \subseteq P$ and source in $S(P)^Q$. Every agent occurring in the target of t occurs in its source or in M. Reduce p replaces a goal by the body of a clause of M renamed apart, instantiated by a writer mgu whose terms are subterms of the goal and the clause; a `'_send'` it reduces globalizes a term of the goal, allocating names anchored at p and entries recording the agent the goal addresses, and adds to M_p a message addressed to that agent. Send p to q removes a message from M_p and applies it at q: localization creates entries and records naming the anchors the message carries and p, and adds requests addressed to those anchors and an acknowledgement addressed to p; a request records p and re-addresses a pending value to p; an acknowledgement records p. As M names no agent, the target is in $S(P)^Q$, and $t \in R(P)$.

Obliviousness condition: A class whose transactions have participants spanning both P and P' is one of Send transactions from some $p \in P$ to some $q \in P'$, or the reverse. In an interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$, the state of p is a state of the run of $\mathcal{F}(P)$, in $S(P)$, so no message in M_p is addressed to q, and Send p to q is not enabled; likewise the reverse. The condition of Proposition 8.12 is satisfied.

Interactivity: Let $Q \subset \Pi$, $P, P' \subseteq Q$ be disjoint and nonempty, $\hat{r}$ a prefix of a correct run of $\mathcal{F}(Q)$ containing no interaction between P and P', $p \in P$ and $q \in P'$. As M uses cold calls, $\hat{r}$ extends, by an Input p writing a term in which q occurs and by Reduce transactions of p, to a finite safe run of $\mathcal{F}(Q)$ at whose last configuration p's network output stream holds $msg(q, T)$; the Reduce of `send_to_net` (Remark 6.12) then adds to M_p the serializer message addressed to q, and Send p to q is enabled; extend the run by it. The Send changes the states of p and q, and p's step is not a transition of $\mathcal{F}(\{p\})$, its source naming $q \notin \{p\}$; so it is an interaction between p and q (Definition 8.6), and hence between P and P'. The result is a finite safe run of $\mathcal{F}(Q)$ having $\hat{r}$ as a prefix and containing an interaction between P and P', and by Lemma 8.4 it is a prefix of a correct run of $\mathcal{F}(Q)$. Hence $\mathcal{F}$ is interactive.

9 Related Work

Concurrent Constraint Programming. Concurrent constraint programming [46] organises computation around a shared store accessed by concurrent agents, accumulating constraints rather than assigning single-occurrence variables. GLP belongs to the family of committed-choice concurrent logic languages [47,60,10] surveyed in [48], and may be understood as Flat Concurrent Prolog [39] with SRSW added—resolving the semantic difficulties of read-only unification [27] so that term matching suffices.

Modes and Linearity. Ueda's moded Flat GHC [62,63] assigns polarity to every variable occurrence and guarantees each variable is written exactly once, establishing "programming as wiring"; his subsequent linearity analysis [61] identifies single-reader variables for compile-time garbage collection. GLP enforces both single-reader and single-writer universally as syntactic restrictions, enabling its distributed execution model. The SRSW discipline connects to linear logic [17,64,21]: each assignment is produced and consumed at most once.

Session Types and Linear Logic. GLP's writer/reader pairs connect to session types via the Curry-Howard correspondence [9,65,66]: linear logic propositions interpret as session types where channels are used exactly once. Multiparty session types [23] coordinate communication among multiple agents; a companion paper [55] develops a type system for GLP along these lines. Whereas session types address communication-protocol safety, this paper addresses correctness of a distributed implementation against an abstract specification. The forwarder role of `global_send` parallels the forwarding processes of the π-calculus encodings of synchronous into asynchronous communication [41,22,8]; a global link differs in carrying exactly one write-once assignment from its fixed anchor to the reader's current holder.

Futures and Promises. A GLP reader/writer pair is operationally a future or promise [5,16,14]: the writer creates a promise, the reader awaits fulfilment. Pruiksma and Pfenning [44] develop linear futures with write-once shared-memory semantics analogous to GLP's. Unlike mainstream futures, GLP's may carry terms with further reader/writer pairs, enabling recursive communication structures shared with channel-passing calculi [40]; none of these works address distributed implementation across independent agents—the specific contribution of madGLP's global links.

Transition Systems and Refinement. Our correctness framework uses simulation relations between transition systems, following the I/O automata tradition [34,35,36,37] and Abadi and Lamport's refinement-mapping theory [1]. We use *outcome equivalence* rather than trace refinement, and forward simulation directly (without history or prophecy variables): the SO invariant provides sufficient structure. We use simulation rather than bisimulation [45] because committed-choice clause selection means dGLP and cGLP need not be bisimilar even though they produce the same outcomes.

Atomic Transactions. Our showing that shared variables spanning agents can be implemented by local variables and message passing builds on the theory of atomic transactions [32,33,6] and on linearizability [20], which defines concurrent

correctness via linearization points. Our decomposition follows a different route: disjoint substitution commutativity (a consequence of SO) makes the order of the implementation's transactions irrelevant to outcomes.

Grassroots and Monotonic Distributed Systems. The grassroots property—that any agent subset forms a functioning subsystem—was introduced for distributed platforms [51] via multiagent transition systems with monotonicity. The definition we use follows [29]. Applications include grassroots social networks [52], cryptocurrencies [50,28], and federated governance [19,58]. CRDTs [57,26] achieve eventual consistency via commuting updates; the Blocklace [2] demonstrates the compositionality grassroots systems require.

Related Language Implementations. Several concurrent languages share design goals with GLP: Erlang [3,30], Go [43], Rust [38], session-typed languages [12,31,59], capability-based actors [11], and active-object languages with futures [25]. GLP differs in its logic programming foundation, with SRSW emerging from logical structure rather than a separate type system; none provide formal proofs that their distributed implementations correctly implement their abstract semantics.

AI-Assisted Formal Methods. AI-driven derivation of implementations from formal specifications is an emerging discipline [15,42,7]. Our approach shares the premise that formal specifications constrain AI-generated code, but employs three layers of abstraction (math→informal spec→Dart) with bidirectional feedback, and at greater scale: the specifications in this paper underwent several major revisions driven by difficulties discovered during AI implementation (Section 7).

10 Conclusion

Summary. We have presented dGLP and madGLP—deterministic operational semantics that implement cGLP and maGLP respectively—and proved them correct with respect to their abstract counterparts, madGLP for authorised runs.Disjoint substitution commutativity (from the SO invariant) enables correctness proofs that carry over directly from the single-agent to the multiagent case. The madGLP design—local variable pairs connected by global links—provides a clean separation between local computation and inter-agent communication, supporting both the formal correctness proofs and practical implementation.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Abadi, M., Lamport, L.: The existence of refinement mappings. Theoretical Computer Science **82**(2), 253–284 (1991). https://doi.org/10.1016/0304-3975(91)90224-P
2. Almeida, P.S., Shapiro, E.: The blocklace: A byzantine-repelling and universal conflict-free replicated data type. arXiv preprint arXiv:2402.08068 (2024), https://arxiv.org/abs/2402.08068
3. Armstrong, J.: Erlang. Communications of the ACM **53**(9), 68–75 (2010). https://doi.org/10.1145/1810891.1810910
4. Bachar, D., Shapiro, E.: GLP networking API specification (2026), arXiv, to appear
5. Baker, H.G., Hewitt, C.: The incremental garbage collection of processes. In: Proceedings of the 1977 Symposium on Artificial Intelligence and Programming Languages. pp. 55–59. ACM (1977). https://doi.org/10.1145/800228.806932
6. Bernstein, P.A., Hadzilacos, V., Goodman, N.: Concurrency Control and Recovery in Database Systems. Addison-Wesley (1987)
7. Blinn, A., Li, X., Kim, J.H., Omar, C.: Statically contextualizing large language models with typed holes. In: Proceedings of the ACM on Programming Languages (OOPSLA). vol. 8, pp. 1–29. ACM (2024). https://doi.org/10.1145/3689746
8. Boudol, G.: Asynchrony and the π-calculus. Tech. Rep. Research Report RR-1702, INRIA (1992)
9. Caires, L., Pfenning, F.: Session types as intuitionistic linear propositions. In: Proceedings of the 21st International Conference on Concurrency Theory (CONCUR). Lecture Notes in Computer Science, vol. 6269, pp. 222–236. Springer (2010). https://doi.org/10.1007/978-3-642-15375-4_16
10. Clark, K., Gregory, S.: Parlog: parallel programming in logic. ACM Transactions on Programming Languages and Systems (TOPLAS) **8**(1), 1–49 (1986). https://doi.org/10.1145/5001.5390
11. Clebsch, S., Drossopoulou, S., Blessing, S., McNeil, A.: Deny capabilities for safe, fast actors. In: Proceedings of the 5th International Workshop on Programming Based on Actors, Agents, and Decentralized Control (AGERE!). pp. 1–12. ACM (2015). https://doi.org/10.1145/2824815.2824816
12. Cooper, E., Lindley, S., Wadler, P., Yallop, J.: Links: Web programming without tiers. In: Proceedings of the 5th International Symposium on Formal Methods for Components and Objects (FMCO). Lecture Notes in Computer Science, vol. 4709, pp. 266–296. Springer (2006). https://doi.org/10.1007/978-3-540-74792-5_12
13. Eitan, O., Keidar, I., Shapiro, E.: Securing people and their machines against major faults. Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear (2026), https://arxiv.org/abs/2607.02304, arXiv:2607.02304
14. Flanagan, C., Felleisen, M.: The semantics of future and an application. Journal of Functional Programming **9**(1), 1–31 (1999). https://doi.org/10.1017/S0956796899003329
15. Fowler, M.: Understanding spec-driven-development: Kiro, spec-kit, and tessl. MartinFowler.com (October 2025), https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html, analyzes the shift toward using formal specifications and types as the primary interface for AI code generation
16. Friedman, D.P., Wise, D.S.: The impact of applicative programming on multiprocessing. Indiana University Computer Science Department Technical Report (TR-26) (1976)

17. Girard, J.Y.: Linear logic. Theoretical Computer Science **50**(1), 1–101 (1987). https://doi.org/10.1016/0304-3975(87)90045-4
18. Google: Dart programming language. https://dart.dev (2024), https://dart.dev/
19. Halpern, D., Procaccia, A.D., Shapiro, E., Talmon, N.: Federated assemblies. Proc AAAI 2025; arXiv preprint arXiv:2405.19129 (2024), https://arxiv.org/abs/2405.19129
20. Herlihy, M.P., Wing, J.M.: Linearizability: A correctness condition for concurrent objects. ACM Transactions on Programming Languages and Systems **12**(3), 463–492 (1990). https://doi.org/10.1145/78969.78972
21. Hodas, J.S., Miller, D.: Logic programming in a fragment of intuitionistic linear logic. Information and Computation **110**(2), 327–365 (1994). https://doi.org/10.1006/inco.1994.1036
22. Honda, K., Tokoro, M.: An object calculus for asynchronous communication. In: Proceedings of the European Conference on Object-Oriented Programming (ECOOP). Lecture Notes in Computer Science, vol. 512, pp. 133–147. Springer (1991)
23. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. Journal of the ACM **63**(1), 9:1–9:67 (2016). https://doi.org/10.1145/2827695
24. Houri, A., Shapiro, E.: A sequential abstract machine for flat concurrent prolog. The Journal of Logic Programming **7**(2), 85–123 (1989). https://doi.org/10.1016/0743-1066(89)90011-3
25. Johnsen, E.B., Hähnle, R., Schäfer, J., Schlatte, R., Steffen, M.: ABS: A core language for abstract behavioral specification. In: Proceedings of the 9th International Symposium on Formal Methods for Components and Objects (FMCO). Lecture Notes in Computer Science, vol. 6957, pp. 142–164. Springer (2011). https://doi.org/10.1007/978-3-642-25271-6_8
26. Kleppmann, M.: Making CRDTs Byzantine fault tolerant. In: Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data. pp. 8–15 (2022)
27. Levi, G., Palamidessi, C.: The semantics of the read-only variable. In: Proc. Symposium on Logic Programming. pp. 128–137. IEEE (1985)
28. Lewis-Pye, A., Naor, O., Shapiro, E.: Grassroots flash: A payment system for grassroots cryptocurrencies. arXiv preprint arXiv:2309.13191 (2023), https://arxiv.org/abs/2309.13191
29. Lewis-Pye, A., Shapiro, E.: Volition-guarded multiagent atomic transactions: Describing people and their machines. arXiv preprint arXiv:2604.25596 (2026), https://arxiv.org/abs/2604.25596
30. Lindahl, T., Sagonas, K.: Practical type inference based on success typings. In: Proceedings of the 8th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP). pp. 167–178. ACM (2006). https://doi.org/10.1145/1140335.1140356
31. Lindley, S., Morris, J.G.: Lightweight functional session types. Behavioural Types: From Theory to Tools pp. 265–286 (2017), chapter in edited volume
32. Lynch, N.A., Merritt, M., Weihl, W., Fekete, A.: Atomic transactions. Morgan Kaufmann (1988)
33. Lynch, N.A., Merritt, M., Weihl, W.E., Fekete, A.: Atomic Transactions: In Concurrent and Distributed Systems. Morgan Kaufmann Publishers, San Francisco, CA (1994)

34. Lynch, N.A., Tuttle, M.R.: Hierarchical correctness proofs for distributed algorithms. In: Proceedings of the 6th Annual ACM Symposium on Principles of Distributed Computing (PODC). pp. 137–151. ACM (1987). https://doi.org/10.1145/41840.41852
35. Lynch, N.A., Tuttle, M.R.: An introduction to input/output automata. Tech. Rep. MIT/LCS/TM-373, Laboratory for Computer Science, Massachusetts Institute of Technology (1988)
36. Lynch, N.A., Vaandrager, F.W.: Forward and backward simulations: I. Untimed systems. Information and Computation **121**(2), 214–233 (1995). https://doi.org/10.1006/inco.1995.1134
37. Lynch, N.A., Vaandrager, F.W.: Forward and backward simulations: II. Timing-based systems. Information and Computation **128**(1), 1–25 (1996). https://doi.org/10.1006/inco.1996.0060
38. Matsakis, N.D., Klock II, F.S.: The Rust language. In: Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology (HILT). pp. 103–104. ACM (2014). https://doi.org/10.1145/2663171.2663188
39. Mierowsky, C., Taylor, S., Shapiro, E., Levy, J., Safra, S.: The design and implementation of Flat Concurrent Prolog. Tech. Rep. CS85-09, Department of Computer Science, The Weizmann Institute of Science, Rehovot, Israel (1985)
40. Milner, R.: Communicating and Mobile Systems: The π-Calculus. Cambridge University Press (1999), https://www.cambridge.org/9780521658690
41. Milner, R., Parrow, J., Walker, D.: A calculus of mobile processes, I and II. Information and Computation **100**(1), 1–77 (1992)
42. Mündler, N., Guerraoui, R., Vechev, M.: Type-constrained code generation with language models. In: Proceedings of the 46th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM (2025), available at https://arxiv.org/abs/2504.09246
43. Pike, R.: Go at Google: Language design in the service of software engineering. Keynote at SPLASH 2012 (2012), available at https://go.dev/talks/2012/splash.article
44. Pruiksma, K., Pfenning, F.: Back to futures. Journal of Functional Programming **32**, e4 (2022). https://doi.org/10.1017/S0956796822000016
45. Sangiorgi, D.: On the bisimulation proof method. Mathematical Structures in Computer Science **8**(5), 447–479 (1998). https://doi.org/10.1017/S0960129598002527
46. Saraswat, V.A., Rinard, M., Panangaden, P.: Semantic foundations of concurrent constraint programming. In: Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL). pp. 333–352. ACM (1991). https://doi.org/10.1145/99583.99627
47. Shapiro, E.: A subset of concurrent prolog and its interpreter. ICOT Technical Report, TR-003 (1983)
48. Shapiro, E.: The family of concurrent logic programming languages. ACM Computing Surveys (CSUR) **21**(3), 413–510 (1989). https://doi.org/10.1145/72551.72555
49. Shapiro, E.: Multiagent transition systems: Protocol-stack mathematics for distributed computing. arXiv preprint arXiv:2112.13650 (2021), https://arxiv.org/abs/2112.13650
50. Shapiro, E.: Grassroots currencies: A foundation for a grassroots digital economy. arXiv preprint arXiv:2202.05619 (2022), https://arxiv.org/abs/2202.05619

51. Shapiro, E.: Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In: 37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391). pp. 47:1, 47:7. LIPICS, Italy (2023), https://arxiv.org/abs/2301.04391
52. Shapiro, E.: Grassroots social networking: Serverless, permissionless protocols for twitter/linkedin/whatsapp. In: OASIS '23. Association for Computing Machinery (2023). https://doi.org/10.1145/3599696.3612898
53. Shapiro, E.: Glp: A grassroots, multiagent, concurrent, logic programming language for ai (full version). arXiv preprint arXiv:2510.15747, Summary in Proc. of ICLP'26, EPTCS 450, pp. 119–133 (2025), https://arxiv.org/abs/2510.15747
54. Shapiro, E.: Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In: Proceedings of the 27th International Conference on Distributed Computing and Networking. pp. 71–81 (2026). https://doi.org/10.1145/3772290.3772309, arXiv preprint arXiv:2502.11299
55. Shapiro, E.: Types for grassroots logic programs (full version). arXiv preprint arXiv:2601.17957 (2026), https://arxiv.org/abs/2601.17957
56. Shapiro, E.: Volition elicitation: Operational semantics for people and their machines. arXiv:2607.14138 (2026), https://arxiv.org/abs/2607.14138
57. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M.: Conflict-free replicated data types. In: Stabilization, Safety, and Security of Distributed Systems: 13th International Symposium, SSS 2011, Grenoble, France, October 10-12, 2011. Proceedings 13. pp. 386–400. Springer (2011)
58. Talmon, N., Shapiro, E.: Grassroots federation: Fair democratic governance at scale. arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26 (2025), https://arxiv.org/abs/2505.02208
59. Toninho, B., Caires, L., Pfenning, F.: Higher-order processes, functions, and sessions: A monadic integration. In: Proceedings of the 22nd European Symposium on Programming (ESOP). Lecture Notes in Computer Science, vol. 7792, pp. 350–369. Springer (2013). https://doi.org/10.1007/978-3-642-37036-6_20
60. Ueda, K.: Guarded horn clauses. In: Logic Programming '85. Lecture Notes in Computer Science, vol. 221, pp. 168–179. Springer (1986). https://doi.org/10.1007/3-540-16479-0_17
61. Ueda, K.: Resource-passing concurrent programming. Proceedings of TACS 2001 pp. 95–126 (2001). https://doi.org/10.1007/3-540-45500-0_5
62. Ueda, K., Morita, M.: Moded flat ghc and its message-oriented implementation technique. New Generation Computing **12**(4), 337–368 (1994). https://doi.org/10.1007/BF03038307
63. Ueda, K., Morita, M.: I/o mode analysis in concurrent logic programming. In: Proceedings of the International Symposium on Theory and Practice of Parallel Programming. pp. 356–368. Springer (1995). https://doi.org/10.1007/BFb0026579
64. Wadler, P.: A taste of linear logic. Mathematical Structures in Computer Science **3**(4), 367–392 (1993). https://doi.org/10.1017/S0960129500000268
65. Wadler, P.: Propositions as sessions. In: Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP). pp. 273–286. ACM (2012). https://doi.org/10.1145/2364527.2364568
66. Wadler, P.: Propositions as sessions. Journal of Functional Programming **24**(2–3), 384–418 (2014). https://doi.org/10.1017/S095679681400001X

A Detailed madGLP Example Traces

This appendix provides complete formal traces for three madGLP examples: the client-monitor example (Program 5.2), the friend-mediated introduction scenario (Bob sends X? to Alice and X to Charlie), and the edge case where both ends of a variable pair are sent to the same agent.

A.1 Example 1: Client-Monitor Communication

This example corresponds to Figure 1.

Initial goal: `client1(Xs)@p, monitor(Xs?)@q`

Stage 0: After Initial Cold-call.

maGLP State	
Agent p Resolvent	**Agent q Resolvent**
Xs	$\mathsf{Xs?}$
Shared Pairs: $(\mathsf{Xs}@p, \mathsf{Xs?}@q)$	

madGLP State	
Agent p	**Agent q**
Resolvents (including global_send goals)	
$\mathsf{Xs_p}, \mathsf{Xs_p?}$, $\texttt{global_send}(\mathsf{Xs_p?}, _r(p,1), q)$	$\mathsf{Xs_q}, \mathsf{Xs_q?}$
Assignments	
(none)	(none)
Global Writers Tables	
(empty)	$\begin{array}{ll} k & \text{entry} \\ \hline 1 & (\mathsf{Xs_q}, p, 1) \end{array}$
Pending Messages: (none)	

During Globalize, the reader $\mathsf{Xs_p?}$ was globalized as $_r(p,1)$, spawning $\texttt{global_send}(\mathsf{Xs_p?}, _r(p,1), q)$ at p (Definition 5.9, case 2). No entry is created at p—the `global_send` goal handles outgoing communication. During Localize at q, the reader global name $_r(p,1)$ created pair $(\mathsf{Xs_q}, \mathsf{Xs_q?})$ with entry $(\mathsf{Xs_q}, p, 1)$ at index 1 in q's global writers table (Definition 5.10, case 2). Agent q will receive the assignment on this link.

Stage 1a: After p Reduces (before Send p to q).

Agent p reduces with a clause whose head is $[\texttt{add}|\mathsf{Xs1_p?}]$, producing the assignment $\mathsf{Xs_p} := [\texttt{add}|\mathsf{Xs1_p?}]$. This makes $\mathsf{Xs_p?}$ known, triggering the `global_send` goal.

madGLP State	
Agent p	**Agent** q
Resolvents (including global_send goals)	
$\mathsf{Xs1_p}, \mathsf{Xs1_p?}$, $\texttt{global_send}(\mathsf{Xs1_p?}, _r(p,2), q)$	$\mathsf{Xs_q}, \mathsf{Xs_q?}$
Assignments	
$\mathsf{Xs_p} := [\texttt{add}\vert\mathsf{Xs1_p?}]$ $\mathsf{Xs_p?} := [\texttt{add}\vert\mathsf{Xs1_p?}]$	(none)
Global Writers Tables	
(empty)	$\frac{k \quad \text{entry}}{1 \ (\mathsf{Xs_q}, p, 1)}$
Pending Messages	
$(_r(p,1) := [\texttt{add}\vert_r(p,2)],\ q)$	

Globalization performed:

Term before	$[\texttt{add}\vert\mathsf{Xs1_p?}]$
Term after	$[\texttt{add}\vert_r(p,2)]$
New goal at p	$\texttt{global_send}(\mathsf{Xs1_p?}, _r(p,2), q)$
New entry at p	(none—outgoing link)

The old `global_send` goal for $\mathsf{Xs_p?}$ fired (since $\mathsf{Xs_p?}$ became known) and completed after adding the message to M_p. The value $[\texttt{add}\vert\mathsf{Xs1_p?}]$ contains reader $\mathsf{Xs1_p?}$, which is globalized as $_r(p,2)$ with a new `global_send` goal spawned (Definition 5.9, case 2).

Stage 1b: After Send p to q.

Agent q receives message and localizes.

maGLP State	
Agent p **Resolvent**	**Agent** q **Resolvent**
$\mathsf{Xs1}$	$\mathsf{Xs1?}$
Assignments	
$\mathsf{Xs} := [\texttt{add}\vert\mathsf{Xs1}]$	$\mathsf{Xs?} := [\texttt{add}\vert\mathsf{Xs1?}]$
Shared Pairs: $(\mathsf{Xs1}@p, \mathsf{Xs1?}@q)$	

madGLP State	
Agent p	**Agent** q
Resolvents (including global_send goals)	
$\mathsf{Xs1_p}, \mathsf{Xs1_p?}$, $\texttt{global_send}(\mathsf{Xs1_p?}, _r(p,2), q)$	$\mathsf{Xs1_q}, \mathsf{Xs1_q?}$
Assignments	
$\mathsf{Xs_p} := [\texttt{add}\vert\mathsf{Xs1_p?}]$ $\mathsf{Xs_p?} := [\texttt{add}\vert\mathsf{Xs1_p?}]$	$\mathsf{Xs_q} := [\texttt{add}\vert\mathsf{Xs1_q?}]$ $\mathsf{Xs_q?} := [\texttt{add}\vert\mathsf{Xs1_q?}]$
Global Writers Tables	
(empty)	$\frac{k \quad \text{entry}}{2 \ (\mathsf{Xs1_q}, p, 2)}$
Pending Messages: (none)	

Localization performed:

Term before	$[\texttt{add}\vert_r(p,2)]$
Term after	$[\texttt{add}\vert\texttt{Xs1}_\texttt{q}?]$
New pair at q	$(\texttt{Xs1}_\texttt{q}, \texttt{Xs1}_\texttt{q}?)$
New entry at q	$(2, (\texttt{Xs1}_\texttt{q}, p, 2))$
New goal at q	(none—incoming link)

Reader global name $_r(p,2)$ is localized by creating pair $(\texttt{Xs1}_\texttt{q}, \texttt{Xs1}_\texttt{q}?)$, placing $\texttt{Xs1}_\texttt{q}?$ (reader) in the term, and adding entry $(\texttt{Xs1}_\texttt{q}, p, 2)$ (Definition 5.10, case 2). Entry 1 at q was removed after assigning $\texttt{Xs}_\texttt{q}$, and q acknowledged $_r(p,1)$; on its arrival p releases the pending value, closing link 1 (§6.5).

Stage 2a: After p Reduces (before Send p to q).

Agent p reduces with a clause whose head is $[\texttt{value}(\texttt{V}_\texttt{p})\vert\texttt{Xs2}_\texttt{p}?]$, producing the assignment $\texttt{Xs1}_\texttt{p} := [\texttt{value}(\texttt{V}_\texttt{p})\vert\texttt{Xs2}_\texttt{p}?]$. The clause head contains writer $\texttt{V}_\texttt{p}$ (the monitor will write the value back) and reader $\texttt{Xs2}_\texttt{p}?$ (the stream continuation that p will write to).

madGLP State	
Agent p	**Agent q**
Resolvents (including global_send goals)	
$\texttt{V}_\texttt{p}, \texttt{V}_\texttt{p}?, \texttt{Xs2}_\texttt{p}, \texttt{Xs2}_\texttt{p}?,$ $\texttt{global_send}(\texttt{Xs2}_\texttt{p}?, _r(p,4), q)$	$\texttt{Xs1}_\texttt{q}, \texttt{Xs1}_\texttt{q}?$
Assignments	
$\texttt{Xs}_\texttt{p} := [\texttt{add}\vert\texttt{Xs1}_\texttt{p}?]$ $\texttt{Xs}_\texttt{p}? := [\texttt{add}\vert\texttt{Xs1}_\texttt{p}?]$ $\texttt{Xs1}_\texttt{p} := [\texttt{value}(\texttt{V}_\texttt{p})\vert\texttt{Xs2}_\texttt{p}?]$ $\texttt{Xs1}_\texttt{p}? := [\texttt{value}(\texttt{V}_\texttt{p})\vert\texttt{Xs2}_\texttt{p}?]$	$\texttt{Xs}_\texttt{q} := [\texttt{add}\vert\texttt{Xs1}_\texttt{q}?]$ $\texttt{Xs}_\texttt{q}? := [\texttt{add}\vert\texttt{Xs1}_\texttt{q}?]$
Global Writers Tables	
$\frac{i \quad \text{entry}}{3 \ (\texttt{V}_\texttt{p}, q)}$	$\frac{k \quad \text{entry}}{2 \ (\texttt{Xs1}_\texttt{q}, p, 2)}$
Pending Messages	
$(_r(p,2) := [\texttt{value}(_w(p,3))\vert_r(p,4)],\ q)$	

Globalization performed:

Term before	$[\texttt{value}(\texttt{V}_\texttt{p})\vert\texttt{Xs2}_\texttt{p}?]$
Term after	$[\texttt{value}(_w(p,3))\vert_r(p,4)]$

$\texttt{V}_\texttt{p}$ is a writer, so it is globalized as $_w(p,3)$ and entry $(\texttt{V}_\texttt{p}, q)$ is created at index 3 in p's global writers table (Definition 5.9, case 1). Agent p will receive the assignment on this link. $\texttt{Xs2}_\texttt{p}?$ is a reader, so it is globalized as $_r(p,4)$ and $\texttt{global_send}(\texttt{Xs2}_\texttt{p}?, _r(p,4), q)$ is spawned (Definition 5.9, case 2); no entry is created for this outgoing link.

Stage 2b: After Send p to q.

Agent q receives message and localizes.

maGLP State	
Agent p Resolvent	**Agent q Resolvent**
$\mathsf{V}?,\ \mathsf{Xs2}$	$\mathsf{V},\ \mathsf{Xs2}?$
Assignments	
$\mathsf{Xs} := [\mathtt{add}\vert\mathsf{Xs1}]$	$\mathsf{Xs}? := [\mathtt{add}\vert\mathsf{Xs1}?]$
$\mathsf{Xs1} := [\mathtt{value}(\mathsf{V}?)\vert\mathsf{Xs2}]$	$\mathsf{Xs1}? := [\mathtt{value}(\mathsf{V})\vert\mathsf{Xs2}?]$
Shared Pairs: $(\mathsf{V}@q, \mathsf{V}?@p)$, $(\mathsf{Xs2}@p, \mathsf{Xs2}?@q)$	

madGLP State	
Agent p	**Agent q**
Resolvents (including global_send goals)	
$\mathsf{V_p}, \mathsf{V_p}?, \mathsf{Xs2_p}, \mathsf{Xs2_p}?,$ $\mathtt{global_send}(\mathsf{Xs2_p}?, _r(p,4), q)$	$\mathsf{V_q}, \mathsf{V_q}?, \mathsf{Xs2_q}, \mathsf{Xs2_q}?,$ $\mathtt{global_send}(\mathsf{V_q}?, _w(p,3), p)$
Assignments	
$\mathsf{Xs_p} := [\mathtt{add}\vert\mathsf{Xs1_p}?]$	$\mathsf{Xs_q} := [\mathtt{add}\vert\mathsf{Xs1_q}?]$
$\mathsf{Xs_p}? := [\mathtt{add}\vert\mathsf{Xs1_p}?]$	$\mathsf{Xs_q}? := [\mathtt{add}\vert\mathsf{Xs1_q}?]$
$\mathsf{Xs1_p} := [\mathtt{value}(\mathsf{V_p})\vert\mathsf{Xs2_p}?]$	$\mathsf{Xs1_q} := [\mathtt{value}(\mathsf{V_q})\vert\mathsf{Xs2_q}?]$
$\mathsf{Xs1_p}? := [\mathtt{value}(\mathsf{V_p})\vert\mathsf{Xs2_p}?]$	$\mathsf{Xs1_q}? := [\mathtt{value}(\mathsf{V_q})\vert\mathsf{Xs2_q}?]$
Global Writers Tables	
$\frac{i \quad \text{entry}}{3\ (\mathsf{V_p}, q)}$	$\frac{k \quad \text{entry}}{3\ (\mathsf{Xs2_q}, p, 4)}$
Pending Messages: (none)	

Localization performed:

Term before	$[\mathtt{value}(_w(p,3))\vert_r(p,4)]$
Term after	$[\mathtt{value}(\mathsf{V_q})\vert\mathsf{Xs2_q}?]$

$_w(p,3)$ is a writer global name, so localization creates pair $(\mathsf{V_q}, \mathsf{V_q}?)$, puts $\mathsf{V_q}$ (the writer) in the term, and spawns $\mathtt{global_send}(\mathsf{V_q}?, _w(p,3), p)$ with the record $(\mathsf{V_q}, p, 3, p)$ (Definition 5.10, case 1). No entry is created—this is an outgoing link from q to p. $_r(p,4)$ is a reader global name, so localization creates pair $(\mathsf{Xs2_q}, \mathsf{Xs2_q}?)$, puts $\mathsf{Xs2_q}?$ (the reader) in the term, and creates entry $(\mathsf{Xs2_q}, p, 4)$ at index 3 (Definition 5.10, case 2)—this is an incoming link. Entry 2 at q was removed after assigning $\mathsf{Xs1_q}$, and q acknowledged $_r(p,2)$, releasing p's pending value.

Global Link Summary.

After Stage 2b, the active global links are:

Link	**Entry at**	**Entry**	**Direction**	**global_send at**
V	p (index 3)	$(\mathsf{V_p}, q)$	$q \to p$	q
Xs2	q (index 3)	$(\mathsf{Xs2_q}, p, 4)$	$p \to q$	p

The "Direction" column indicates dataflow: V flows from q to p (the monitor writes the value), while Xs2 flows from p to q (the client writes events). Entries exist only at the receiving end; `global_send` goals exist only at the sending end. Each reader-name value is acknowledged by its receiver, and its sender holds it pending until then (§6.5).

A.2 Example 2: Friend-Mediated Introduction

This example demonstrates the critical case where Bob (agent b) introduces Alice (agent a) to Charlie (agent c) by sending them opposite ends of a variable pair. Bob sends reader $X?$ to Alice and writer X to Charlie. Bob's local pair $(X_b, X_b?)$ acts as a relay: when Charlie assigns his writer, the value flows through Bob to Alice.

Stage 0: Initial State at Bob.

Bob has a local variable pair $(X_b, X_b?)$ that will be shared with Alice and Charlie.

madGLP State		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Resolvents		
(empty)	$X_b, X_b?$	(empty)
Global Writers Tables		
(empty)	(empty)	(empty)

Stage 1: Bob Sends $X?$ to Alice via Cold-call.

Bob globalizes $X_b?$ (a reader) for Alice.

Globalization at Bob for Alice:

Variable	$X_b?$ (reader)
Global name	$_r(b, 1)$
Entry at Bob	(none—outgoing link)
Spawned goal	`global_send`$(X_b?, _r(b, 1), a)$

Localization at Alice:

Global name	$_r(b, 1)$
New pair	$(X_a, X_a?)$
In term	$X_a?$ (reader)
Entry at Alice	$(X_a, b, 1)$
Spawned goal	(none—incoming link)

madGLP State after Stage 1		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Resolvents (including global_send goals)		
$X_a, X_a?$	$X_b, X_b?$, `global_send`$(X_b?, _r(b, 1), a)$	(empty)
Global Writers Tables		
$\frac{k \quad \text{entry}}{1 \ (X_a, b, 1)}$	(empty)	(empty)

Bob globalized reader $X_b?$ as $_r(b, 1)$, spawning a `global_send` goal at Bob (Definition 5.9, case 2). Alice localized $_r(b, 1)$, creating entry $(X_a, b, 1)$ and

placing reader $X_a?$ in her term (Definition 5.10, case 2). Alice will receive values on this link.

Stage 2: Bob Sends X to Charlie via Cold-call.

Bob globalizes X_b (a writer) for Charlie.

Globalization at Bob for Charlie:

Variable	X_b (writer)
Global name	$_w(b, 2)$
Entry at Bob	(X_b, c) at index 2
Spawned goal	(none—incoming link)

Localization at Charlie:

Global name	$_w(b, 2)$
New pair	$(X_c, X_c?)$
In term	X_c (writer)
Entry at Charlie	(none—outgoing link)
Record at Charlie	$(X_c, b, 2, b)$
Spawned goal	`global_send`$(X_c?, _w(b, 2), b)$

madGLP State after Stage 2		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Resolvents (including global_send goals)		
$X_a, X_a?$	$X_b, X_b?$, `global_send`$(X_b?, _r(b, 1), a)$	$X_c, X_c?$, `global_send`$(X_c?, _w(b, 2), b)$
Global Writers Tables		
$\frac{k \quad \text{entry}}{1 \ (X_a, b, 1)}$	$\frac{i \quad \text{entry}}{2 \ (X_b, c)}$	(empty)

Bob globalized writer X_b as $_w(b, 2)$, creating entry (X_b, c) at index 2 (Definition 5.9, case 1). Bob will receive the assignment on this link. Charlie localized $_w(b, 2)$, placing writer X_c in his term, recording $(X_c, b, 2, b)$, and spawning `global_send`$(X_c?, _w(b, 2), b)$ (Definition 5.10, case 1). Charlie can assign X_c.

Global Link Summary after Stage 2:

Link	**Entry at**	**Entry**	**global_send at**
$_r(b, 1)$	Alice	$(X_a, b, 1)$	Bob
$_w(b, 2)$	Bob (index 2)	(X_b, c)	Charlie

Stage 3: Charlie Assigns $X_c := T$.

Charlie assigns $X_c := T$ for some term T. This makes $X_c?$ known.

madGLP State after Charlie's assignment		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Resolvents		
$X_a, X_a?$	$X_b, X_b?,$ `global_send`$(X_b?, _r(b,1), a)$	(goal completed)
Assignments		
(none)	(none)	$X_c := T,\ X_c? := T$
Pending Messages		
$(_w(b,2) := T^{\uparrow},\ b)$ from Charlie		

The `global_send`$(X_c?, _w(b,2), b)$ goal fired because $X_c?$ became known. It sent message $(_w(b,2) := T^{\uparrow}, b)$.

Stage 4: Send Charlie to Bob.

Bob receives message $(_w(b,2) := T^{\uparrow})$. He finds entry (X_b, c) at index 2 and assigns $X_b := T^{\downarrow}$.

This makes $X_b?$ known, which triggers the `global_send`$(X_b?, _r(b,1), a)$ goal.

madGLP State after Bob receives		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Resolvents		
$X_a, X_a?$	(goal completed)	(empty)
Assignments		
(none)	$X_b := T,\ X_b? := T$	$X_c := T,\ X_c? := T$
Global Writers Tables		
$\frac{k \quad \text{entry}}{1 \ (X_a, b, 1)}$	(empty)	(empty)
Pending Messages		
$(_r(b,1) := T^{\uparrow},\ a)$ from Bob		

Entry at index 2 at Bob was removed after the assignment; a writer-name value is not acknowledged—its anchor entry is fixed. The `global_send` goal fired and sent message $(_r(b,1) := T^{\uparrow}, a)$.

Stage 5: Send Bob to Alice.

Alice receives message $(_r(b,1) := T^{\uparrow})$. She searches for entry with remote agent b and remote index 1, finds $(X_a, b, 1)$, and assigns $X_a := T^{\downarrow}$.

Final madGLP State		
Agent a (Alice)	**Agent b (Bob)**	**Agent c (Charlie)**
Assignments		
$X_a := T,\ X_a? := T$	$X_b := T,\ X_b? := T$	$X_c := T,\ X_c? := T$
Global Writers Tables		
(empty)	(empty)	(empty)

All entries have been removed. The value T has flowed from Charlie through Bob to Alice; Alice acknowledges $_r(b,1)$, releasing Bob's pending value and closing the link.

Summary: Bob acted as a relay. Charlie assigned his writer X_c, the value flowed to Bob via the $_w(b,2)$ link, Bob's local pair $(X_b, X_b?)$ received it, and the `global_send` goal watching $X_b?$ forwarded it to Alice via the $_r(b,1)$ link. Alice's reader $X_a?$ now has the value T.

The stream continues directly: Suppose $T = [\texttt{hello}|\mathsf{Rest}?]$ with Rest a fresh pair at Charlie. Charlie's value to Bob carries $_r(c,1)$; Bob localizes it with entry $(\mathsf{Rest_b}, c, 1)$ and no request—the sender is the anchor. Forwarding the value to Alice, Bob globalizes the imported reader $\mathsf{Rest_b}?$: the original name $_r(c,1)$ is emitted and Bob's entry removed (Definition 5.9). Alice localizes $_r(c,1)$ from sender Bob $\neq c$, so she sends $\texttt{req}(_r(c,1))$ to Charlie, who records her as the holder. The next element flows Charlie→Alice directly: the introducer relays one value and drops out.

A.3 Example 3: Both Ends to Same Agent

This example demonstrates the edge case where Bob sends both writer X and reader $X?$ to Alice in the same term $[X, X?]$. Bob's local pair acts as a relay: when Alice assigns her writer Y_a (from the writer global name $_w(b,1)$), the value flows through Bob back to Alice's reader $Z_a?$ (from the reader global name $_r(b,2)$).

Stage 0: Initial State.

Bob has a local variable pair $(X_b, X_b?)$ and will send term $[X_b, X_b?]$ to Alice.

madGLP State	
Agent a (Alice)	**Agent b (Bob)**
Resolvents	
(empty)	$X_b, X_b?$
Global Writers Tables	
(empty)	(empty)

Stage 1: Bob Sends $[X_b, X_b?]$ to Alice via Cold-call.

Bob globalizes term $[X_b, X_b?]$ for Alice. Both the writer and reader are processed:

Globalization at Bob:

Variable	X_b **(writer)**	$X_b?$ **(reader)**
Global name	$_w(b,1)$	$_r(b,2)$
Entry at Bob	(X_b, a) at index 1	(none—outgoing)
Spawned goal	(none—incoming)	$\texttt{global_send}(X_b?, _r(b,2), a)$

Globalized term: $[_w(b,1), _r(b,2)]$

Localization at Alice:

Global name	$_w(b,1)$	$_r(b,2)$
New pair	$(Y_a, Y_a?)$	$(Z_a, Z_a?)$
In term	Y_a (writer)	$Z_a?$ (reader)
Entry at Alice	(none—outgoing)	$(Z_a, b, 2)$
Record at Alice	$(Y_a, b, 1, b)$	(none)
Spawned goal	`global_send`$(Y_a?, _w(b,1), b)$	(none—incoming)

Localized term: $[Y_a, Z_a?]$

madGLP State after Stage 1	
Agent a (Alice)	**Agent b (Bob)**
Resolvents (including global_send goals)	
$Y_a, Y_a?, Z_a, Z_a?,$ `global_send`$(Y_a?, _w(b,1), b)$	$X_b, X_b?,$ `global_send`$(X_b?, _r(b,2), a)$
Global Writers Tables	
$\begin{array}{cc} k & \text{entry} \\ \hline 1 & (Z_a, b, 2) \end{array}$	$\begin{array}{cc} i & \text{entry} \\ \hline 1 & (X_b, a) \end{array}$

Alice's term is $[Y_a, Z_a?]$: she has writer Y_a (can assign values) and reader $Z_a?$ (will receive values). These are two *independent* pairs at Alice—she doesn't know they're connected through Bob.

Writer X_b was globalized as $_w(b,1)$ with entry (X_b, a) at Bob (Definition 5.9, case 1); Alice localized it as writer Y_a with the record $(Y_a, b, 1, b)$ and a `global_send` goal (Definition 5.10, case 1). Reader $X_b?$ was globalized as $_r(b,2)$ with `global_send` at Bob (Definition 5.9, case 2); Alice localized it as reader $Z_a?$ with entry $(Z_a, b, 2)$ (Definition 5.10, case 2).

Stage 2: Alice Assigns $Y_a := T$.

Alice assigns $Y_a := T$. This makes $Y_a?$ known, triggering her `global_send` goal.

madGLP State after Alice's assignment	
Agent a (Alice)	**Agent b (Bob)**
Resolvents	
$Z_a, Z_a?$ (goal completed)	$X_b, X_b?,$ `global_send`$(X_b?, _r(b,2), a)$
Assignments	
$Y_a := T,\ Y_a? := T$	(none)
Global Writers Tables	
$\begin{array}{cc} k & \text{entry} \\ \hline 1 & (Z_a, b, 2) \end{array}$	$\begin{array}{cc} i & \text{entry} \\ \hline 1 & (X_b, a) \end{array}$
Pending Messages	
$(_w(b,1) := T^{\uparrow},\ b)$ from Alice	

The `global_send` goal sent message $(_w(b,1) := T^{\uparrow}, b)$.

Stage 3: Send Alice to Bob.

Bob receives message $(_w(b,1) := T^{\uparrow})$. He finds entry (X_b, a) at index 1 and assigns $X_b := T^{\downarrow}$.

This makes $X_b?$ known, triggering his `global_send`$(X_b?, _r(b,2), a)$ goal.

madGLP State after Bob receives	
Agent a (Alice)	**Agent b (Bob)**
Resolvents	
$Z_a, Z_a?$	(goal completed)
Assignments	
$Y_a := T$, $Y_a? := T$	$X_b := T$, $X_b? := T$
Global Writers Tables	
$\begin{array}{cc} k & \text{entry} \\ \hline 1 & (Z_a, b, 2) \end{array}$	(empty)
Pending Messages	
$(_r(b,2) := T^{\uparrow},\ a)$ from Bob	

Entry at index 1 at Bob was removed after the assignment. The `global_send` goal fired and sent message $(_r(b,2) := T^{\uparrow}, a)$.

Stage 4: Send Bob to Alice.

Alice receives message $(_r(b,2) := T^{\uparrow})$. She searches for entry with remote agent b and remote index 2, finds $(Z_a, b, 2)$, and assigns $Z_a := T^{\downarrow}$.

Final madGLP State	
Agent a (Alice)	**Agent b (Bob)**
Assignments	
$Y_a := T$, $Y_a? := T$ $Z_a := T$, $Z_a? := T$	$X_b := T$, $X_b? := T$
Global Writers Tables	
(empty)	(empty)

All entries have been removed. Bob's value to Alice, carried by the reader name $_r(b,2)$, is acknowledged by Alice; Alice's value to Bob, carried by the writer name $_w(b,1)$, needs no acknowledgement.

Summary: Alice received term $[Y_a, Z_a?]$ with two independent pairs. When she assigned $Y_a := T$, the value flowed to Bob via the $_w(b,1)$ link. Bob's local pair received it, and his `global_send` goal forwarded it back to Alice via the $_r(b,2)$ link. Now both $Y_a?$ and $Z_a?$ equal T—the two pairs at Alice are semantically linked through Bob's relay, even though Alice sees them as independent.

This correctly implements the maGLP semantics: in maGLP, Alice would have received $[X, X?]$ where both variables refer to the same shared pair. Assigning the writer makes the value available to the reader. In madGLP, this is achieved through Bob's relay.

B Code Format

This appendix is the byte-level specification of the GLP code format: the canonical encoding of globalized terms and assignment messages, the encoding of the instruction set, the compiled-program artefact, deterministic flattening, the loader, and the offer and handshake messages. The engine implements the sequential abstract machine of Flat Concurrent Prolog [24], whose reference implementation is available online.[1] The format and engine follow that design, departing only where GLP differs from FCP—the instruction set below and the fixed byte order of §B.1; departures are noted where they occur. Programs, exports, and the load-time interface check are specified in the type and module system paper [55], and the instruction semantics and the signature kernels in the companion GLP paper [53], this appendix assigning their bytes; the source and compiled identities, the module certificate, the adoption of a module, and the conversation handshake are specified here.

B.1 Primitive Encodings

Fixed-width multi-byte integers (i64, f64) are little-endian, the one canonical byte order; clen is self-delimiting and read identically on every machine. A term yields the same byte string on every machine: the bytes that are hashed and signed.

- **u8** — one byte.
- **i64** — 8 bytes, two's complement.
- **f64** — 8 bytes, IEEE 754 binary64.
- **clen** (compact length) — an unsigned integer below 2^{30} in 1, 2, or 4 bytes: values below 128 in one byte; below 16384 in two bytes, the first with high bits `10`; otherwise four bytes, the first with high bits `11`. The remaining bits, big-endian, carry the value. Encoders emit the shortest form; decoders reject longer-than-necessary forms (one value, one encoding).
- **string** — clen byte count, then UTF-8 bytes.
- **bytes** — clen byte count, then raw bytes.
- **hash** — 32 raw bytes: SHA-256 of the hashed content. The **source identity** $h(M)$ is the hash of the flattened source bytes (§B.5); the **compiled identity** is the hash of the artefact body (§B.4).
- **agent** — bytes; its content is the public key of the agent's person. The current realisation uses Ed25519 public keys (32 bytes); the simulation realisation uses UTF-8 symbolic names. The encoding does not interpret the content.

B.2 Term and Message Encoding

This section is the byte-level layout of the canonical encoding e: an injective function from globalized terms and assignment messages to byte strings.

[1] `https://github.com/EShapiro2/FCP`

Terms. A globalized term is encoded by a tagged recursion. Each node opens with a u8 tag:

- **1 constant** — followed by a u8 constant tag and its payload: **0 nil** (no payload, the empty list); **1 integer** (i64); **2 float** (f64); **3 string** (string); **4 boolean** (u8: 0 false, 1 true); **5 bytes** (bytes; raw byte strings, used only in the runtime's own vocabulary—the 32-byte hashes of §B.7—never in application terms); **6 module** (bytes; the artefact bytes of §B.4, decoding to the Module constant—the form in which compiled programs ship).
- **2 variable** — a global name per Definition 5.9: u8 polarity (0 writer $_w(p, e, i)$, 1 reader $_r(p, e, i)$), agent p, clen epoch e, clen index i. An agent is a public key, or in a simulation an atom standing for one (Remark 4.1); an index is a non-negative integer; and the reader refuses a global name whose agent or index is anything else. Names are the allocating agent's—a forwarded reader keeps its anchor's name; no other variable representation exists in the code format.
- **3 structure** — string functor, clen arity n, then the n argument encodings in order.

Lists are structures: a cell is the structure `'.'/2`; the empty list is the constant nil. The four properties of the canonical encoding hold by this layout: hardware-independent (§B.1 pins widths and byte order); address-free (a variable appears only as its global name with polarity); globalizer's names (tag 2 admits nothing else); canonical on ground terms (a ground term contains tags 1 and 3 only, so its bytes are agent-independent — these are the bytes over which terms are signed and hashed). Injectivity holds by unique decoding: every tag determines its payload form, every length is explicit, and clen is one-value-one-encoding.

Messages. A message opens with a u8 kind: **0 value**, **1 request**, **2 acknowledgement**, **3 gone**; then the clen epoch of the sender. A value $G := T^{\uparrow}$ continues: u8 polarity of G, agent of G, clen epoch of G, clen index of G, then the encoding of $T^{\uparrow}$. A request $\texttt{req}(G)$, an acknowledgement $\texttt{ack}(G)$ or a $\texttt{gone}(G)$ continues: u8 polarity of G, agent of G, clen epoch of G, clen index of G; no term follows. The kind byte is new in code-format version 2, the sender's epoch, the epoch in a name and the kind gone in version 4 (§B.8).

A serializer (cold-call) message to agent q is the assignment $_w(q, 0) := [T^{\uparrow} \mid _w(q, 0)]$: polarity 0, agent q, epoch 0, index 0, clen serial, then the encoding of the list cell whose head is $T^{\uparrow}$ and whose tail is the variable $_w(q, 0)$ (tag 2, polarity 0, agent q, epoch 0, index 0). The serial is an index the sender allocates from its global-name counter, so the sender, which the layer authenticates, its epoch in the message and the serial name the cold call for the whole run; the serial is new in code-format version 3 (§B.8). The receiver treats index 0 by serializer semantics: append unless the pair was appended before, update the entry, do not remove it.

Signed and Hashed Content. The canonical bytes of a ground term T are $e(T)$. Protocols that sign or hash compound content encode it as a ground term

and apply e; the signature kernels [53] sign e of the pair of the signing instance's source identity and the term, and the signed term carries the signer's key alongside that pair, so that a verifier recovers all three (§B.7).

B.3 Instruction Encoding

This section assigns bytes to the instruction set; the instruction semantics are those of the companion GLP paper [53]. The scheduling of `spawn` (0x50) and `requeue` (0x51) is dGLP's, not the companion paper's: `spawn` enqueues a body goal, `requeue` continues the clause's tail goal under the b-bounded tail discipline of Definition 3.33. An encoded instruction is a u8 opcode followed by its operands, in the order listed.

Operand kinds.
- **polarity** — u8: 0 writer, 1 reader (the `isReader` flag; same convention as the variable tag).
- **varIndex, argSlot, arity, count, slots, regIndex** — clen.
- **constant** — the constant payload of §B.2 (u8 constant tag, then its payload); one constant representation serves terms and code.
- **functor** — string.
- **proc** — clen index into the artefact's symbol table (§B.4). The entry's kind determines the target: a compiled procedure, or a codeless body kernel or builtin guard bound by name at load. Procedure references are symbol-table indices, never names; clause targets are byte offsets (**ctarget**), not instruction indices.
- **ctarget** — a clen byte offset to a clause target within the current procedure's code (clause targets are procedure-relative); the engine branches by program-counter offset, as in FCP [24]. Assembly labels do not exist in the code format.

Opcode table.

Opcode	Mnemonic	Operands
0x01	`clause_try`	—
0x02	`clause_next`	ctarget
0x03	`no_more_clauses`	—
0x04	`commit`	—
0x05	`proceed`	—
0x06	`halt`	—
0x07	`nop`	—
0x10	`head_constant`	constant, argSlot
0x11	`head_nil`	argSlot
0x12	`head_structure`	functor, arity, argSlot
0x13	`head_list`	argSlot
0x14	`head_variable`	polarity, varIndex

Opcode	Mnemonic	Operands
0x15	get_variable	polarity, varIndex, argSlot
0x16	get_value	polarity, varIndex, argSlot
0x20	unify_variable	polarity, varIndex
0x21	unify_constant	constant
0x22	unify_void	count
0x23	unify_structure	functor, arity
0x24	push	regIndex
0x25	pop	regIndex
0x30	put_variable	polarity, varIndex, argSlot
0x31	put_constant	constant, argSlot
0x32	put_nil	argSlot
0x33	put_list	argSlot
0x34	put_structure	functor, arity, argSlot
0x35	set_variable	polarity, varIndex
0x36	set_constant	constant
0x37	allocate	slots
0x38	deallocate	—
0x39	put_bound_const	constant, argSlot
0x3A	put_bound_nil	argSlot
0x40	guard	proc, arity
0x41	ground	varIndex
0x42	known	varIndex
0x43	unknown	varIndex
0x44	no_readers	varIndex
0x45	ground_equal	varIndex, varIndex
0x46	otherwise	—
0x50	spawn	proc, arity
0x51	requeue	proc, arity

Reserved and excluded. Opcodes 0x47–0x4F are reserved for the comparison guards (guard_less and its five siblings), assigned when they are implemented; their arrival is an instruction-set version change carried in the artefact header (§B.4). The following implementation classes are not part of the code format: Label (assembly-time, erased by the operand kinds); TryNextClause (unused); GuardFail, SuspendEnd, TailStep, BodySetConst, BodySetStructConstArgs, HeadBindWriter, GuardNeedReader, and the *Arg test variants (legacy).

B.4 Program Artefact

The artefact is the byte string a compiler produces from a flat program; it travels as a module constant (§B.2, constant tag 6). It is a **body** followed by a **certificate**. Its **compiled identity** is the SHA-256 of the body; its **source identity** $h(M)$ is the hash of the flattened source (§B.5). A receiver concludes from the certificate that the person of its key vouches that the body was compiled from

the source of identity $h(M)$, and whether to accept that key is the receiver's [53]; a compiled program carries what the load-time interface check needs [55]. The body's sections, in order:

1. **Header** — magic `GLPW` (4 bytes); u8 code-format version (this document: 4); string instruction-set version; string program name. Neither identity is in the header: both are in the certificate, where they are signed.
2. **Interface table** — the program interface, carried as declaration source text, from which the loader derives the type automata: a string giving the reachable type definitions, in canonical print (§B.5); then a clen export count, and per export a string name, clen arity, and string declaration text. Carrying text rather than compiled automata keeps one source of truth: type identity is source-based throughout, and equal sources define equal automata. An export that is the instance of a parameterised declaration D of an ancestor scope—an entry template, in the sense of the Typed GLP paper [55]—carries D's declaration in its declaration text, after its own and unexported, and the type definitions include the types D reaches; the loader takes D out before it builds the tables, and records beside the export table the type identity of D's opaque instance at the export's type and the type identity of that type, which `opaque_type/3` reads.
3. **Symbol table** — clen count; per entry a string name, clen arity, and a u8 kind. Kind **0 compiled** — a procedure defined in this program — is followed by clen byte offset into the code section and clen byte length; kind **1 codeless** — a body kernel or builtin guard with no code in this artefact, bound by name to the local runtime at load (§B.6) — carries nothing further. The **proc** operands of §B.3 index this table. Exported procedures are the compiled entries named in the interface table; the loader aliases exactly them.
4. **Code section** — clen byte count, then the concatenated procedure bodies in the instruction encoding of §B.3. There is no constant pool: constants are inline in instructions.

The **certificate** follows the body: **agent**, the public key of the agent for which the program was compiled; then **hash** *HSrc* and **hash** *HBin*, the two identities; then **bytes**, that person's signature over $e(\texttt{ids}(\mathit{HSrc}, \mathit{HBin}))$, the canonical encoding of the 2-ary structure `ids` whose arguments are the source identity and the compiled identity, both hash constants. The functor `ids` is fixed by this specification. The key is the agent's, which the compiling runtime makes and supplies to the networking layer (§C.2); the simulation realisation signs with the key its router holds for the compiling agent. The runtime makes the key where it has no networking layer too (`glpc`, the test suite). It is written at compilation, whether or not the program is ever sent, so a compiled program states which source it was compiled from and who is answerable for it. A certificate is refused to a compiled program that calls an OS-privileged predicate, so a certified program reaches the social graph and the person only through variables its host gives it. The compiled identity is the hash of the body alone, so the certificate sits inside the artefact without hashing itself, and `decompose_module/4` reads the key and both identities from it.

The body is code only. Source distribution is an ordinary value exchange: the flattened source travels as a blob in a term, and the receiver verifies it by hashing against $h(M)$.

B.5 Deterministic Flattening

The **flattened source** of a program — the preimage of the source identity $h(M)$ — is the canonical print of the linked, pruned program: the program is discovered, type-checked, renamed, resolved, and pruned to the procedures reachable from the root's exported entry points and the type definitions their declarations reference, from `self.glp` files outside the program's tree among them [55]; the result is printed canonically and hashed.

Canonical print: UTF-8, LF line ends, no comments, tokens separated by single spaces, one declaration or clause per line. Order: (*i*) the reachable type definitions, lexicographically by type name; (*ii*) the exported procedure declarations, lexicographically by name then arity; (*iii*) the procedures, lexicographically by renamed name then arity — within a procedure, clauses keep their source order, which is semantic (first-applicable-clause selection). Procedure order is not semantic and is fixed by sorting.

The same byte string is what a grassroots contract's participants read and agree to, what the handshake compares by hash, and what versioning names.

B.6 Loader

Given a received artefact and the offered $h(M)$ (§B.7), the loader:

1. Computes SHA-256 of the body and verifies it equals the compiled identity in the certificate; verifies the certificate's signature under the key the certificate carries; verifies the certificate's source identity equals the offered $h(M)$; refuses unsupported code-format or instruction-set versions (§B.8).
2. Derives the type automata from the interface table's declaration text and runs the load-time interface checks when linking the program against local callers: the callee's exported type must accept the caller's imported type [55].
3. Resolves **proc** indices through the symbol table per §B.3: a compiled entry resolves to the procedure at its offset; a codeless entry binds by name to the local runtime's kernel or builtin guard of that name, the instruction (`spawn`/`requeue` versus `guard`) selecting which, and an unknown name is a failsafe refusal (§B.8). Generates unqualified aliases for the exported procedures only.
4. Registers the program under its two identities; artefacts are cached and deduplicated by compiled identity.

Admission of conversation traffic is not the loader's: the runtime admits application traffic on a conversation only after its handshake succeeds, and aborts failsafe otherwise (§B.7).

After verification, if the running machine's byte order differs from the canonical order the loader inverts the artefact's fixed-width multi-byte fields to local

order for execution; before the program is shipped onward it is inverted back, so the hashed artefact is always the canonical bytes (§B.1).

B.7 Offer and Handshake Messages

These are ground terms between two runtimes over a channel between friends, encoded per §B.2; hash values inside terms are bytes constants of 32 bytes. Adoption of a compiled program M by q from p proceeds in this order: p offers; q's person consents or declines; where q lacks it, p ships it and q verifies it through the loader (§B.6); the two runtimes create the conversation's root channel, p's runtime creating the pair of variable ends and shipping one to q; and the handshake follows.

- `offer(HSrc, tau(HSrcDef, TypeName))` — the adoption offer: the program's source identity, and the root channel's type identity — the pair of the source identity of the type's defining source and the type's name (a string).
- `accept` / `decline` — the consent outcome of the offeree's person.
- `ship(Artefact)` — the compiled program as a module constant. The two identities and the key answerable for them are in the artefact's own certificate (§B.4), so the message no longer carries them.
- `handshake(HSrc, tau(HSrcDef, TypeName))` — exchanged by the two runtimes on a conversation before any application traffic; both ends must present equal values, or the runtime aborts the conversation failsafe, and an aborted conversation is silent to the program: nothing further is delivered on its variables. Derived variables undergo no handshake.
- **Signed content** — `sign(T, K, S)` assigns S the signed term of the ground term T under the key K [53]. S is **agent**, the signer's public key; then **bytes**, the signature; then $e(\texttt{sig}(\mathit{HSrc}, T))$, the canonical encoding of the 2-ary structure `sig` whose arguments are the source identity of the signing program and the ground term. The signature is over those encoded bytes. The functor `sig` is fixed by this specification. `signature(S, R)` assigns R the term `signed(K, H, T)` — the signer's key, the source identity and the signed term — where S is a signed term whose signature verifies under K, and the constant `unsigned` otherwise [53]. The runtime supplies *HSrc* and the caller cannot choose it, so the verifying predicate does not run the signing program.

B.8 Format Versioning

The header carries two versions: the code-format version (u8) — this document, version 4 — and the instruction-set version (string). A loader refuses an artefact whose code-format version or instruction-set version it does not support; refusal at adoption time means the offer fails before any conversation exists. New instructions enter by instruction-set version (reserved ranges, §B.3); changes to the encodings of §B.1–§B.2 or the artefact layout of §B.4 are code-format version changes: version 2 added the message kind byte, version 3 the serializer message's serial, version 4 the epoch in a global name, the sender's epoch in a message and the kind gone. Opcode and name assignments are append-only: an

assigned opcode never changes meaning, and a kernel or guard name, once in use, is never reassigned to a different operation; additions consume only reserved or unused opcodes and new names, so a newer runtime runs older artefacts unchanged. Removing an operand from an assigned opcode, or an opcode, is an instruction-set version change after which the runtime refuses the versions before it; the current version is `glp-isa-3`. Forward incompatibility is always a clean refusal, never silent misexecution: an older runtime rejects a newer instruction-set version at adoption, and a codeless name the runtime does not provide fails to bind at load (§B.6). A version of a compiled program is its source identity $h(M)$ and nothing else: two agents interoperate on a program exactly when their $h(M)$ agree, so a new version is a new program, adopted afresh, and an upgrade is per conversation.

C Implementation Notes

This appendix records implementation decisions of the Dart runtime derived from dGLP and madGLP: the heap representation of variable pairs and suspensions, agent execution and boot, the networking interface of the multiagent runtime, and the type checker's error text. The code lives in the public repository; this appendix states the invariants it maintains.

C.1 Heap: Variables, Dereferencing, Binding, Suspension

Variable pairs. The engine follows the FCP sequential abstract machine [24] in its heap design. A variable pair is two heap cells, writer and reader, each a tagged reference; while the pair is unbound the two cells point to each other, so either end reaches its counterpart by following a pointer — no address arithmetic relates them. A variable occurrence in a goal or term is the cell itself, a reference and not an address — the heap is no array, a cell lives while a goal, a suspension, a wait or a table reaches it and is reclaimed by the host's collector when none does; the cell's tag gives its polarity.

Dereferencing. Dereferencing follows references until reaching a cell that is not a reference: a value, or an unbound writer (one whose pointer leads to its paired reader). A chain never runs writer to writer: a writer binds to a value or to a reader, never to a writer, and dereferencing checks this SRSW invariant. Path compression then rewrites the pointer of the starting cell when it is a writer, and never of a reader, whose pointer names its paired writer: a writer whose chain ends at a value is pointed at the value; a writer whose chain ends at an unbound writer is pointed at that writer's reader, the last reader of the chain, so the invariant holds after compression as before. Path compression is integral to the design, not optional.

Binding. A writer is bound at most once, to a value or to a reader. Binding to a value converts the writer cell to a value cell; binding to a reader stores a pointer to it, extending the dereference chain.

Suspension. A goal that suspends on a reader is registered on the paired writer's cell; an unbound writer carries its suspension list alongside its reader pointer.

Binding the writer to a value activates the registered goals. Binding the writer to a reader routes its suspensions to wherever the reader's dereference chain leads: to the unbound writer at the end of the chain; and, when the chain already ends in a value, the binding determines the writer's value, so the suspensions are activated at once. The invariant: a suspension is activated exactly when its variable becomes determined, wherever along the chain that happens.

Ground mark. A term that is ground stays ground, every variable being assigned once. So when a ground test finds a list cell or a compound term ground, the engine marks the term, and a later ground test of it succeeds at once. The mark changes the result of no construct; it departs from the FCP machine [24], which has none, so that a guard testing a long list ground costs one step after the first test.

The `global_send` goal. The specification puts `global_send` in the resolvent (Definition 6.7), where the suspension machinery above wakes it. The runtime registers it on the writer's cell as a callback instead. A callback is not woken by that machinery: a commit may bind a writer to a reader, and the chain is resolved to a value by rewriting cells rather than by binding the cell the callback sits on. The runtime therefore fires the registered callbacks at the end of a commit, where suspended goals are activated; it holds one registration per open global link, and each fires at most once. Without that pass a value written into the tail of a stream whose reader has already crossed a link is produced and never sent.

Imported readers. An imported reader — one whose writer is at another agent — is the reader of an ordinary local pair created by Localize (Definition 5.10), and its paired local writer is the one named by a global writers table entry. That writer carries the suspension queue and receives the incoming assignment (Definition 6.22), so dereferencing an imported reader reaches an unbound local writer like any other. The entry is found from the arriving name, by its remote agent and index.

Imported writers. An imported writer — one localized from a writer global name — is an ordinary local writer carrying its imported-writer record (Definition 6.5), keyed by the writer's cell and so reached from its cell in constant time: the anchor and index of the link its value must reach, the agent the name came from, and the `global_send` goal watching its reader. The record is what makes re-export a forwarding: Globalize reads it from the writer's cell and removes both it and the goal, so the exporting agent leaves the link instead of relaying its values (Definition 5.9).

Global names in the heap. A variable that a message names by its global name is held in the heap as a term of its own kind, holding the polarity, the agent, the epoch and the index, and never as a structure; only such a term is encoded as a variable (§B.2), and every structure, $_w$ and $_r$ included, as a structure. A program is given a global name as the ground structure $_w(P, E, I)$ or $_r(P, E, I)$ (§6.1). Ground content, such as a signed term, carries no variable, so a variable in it is refused, and a signed term carrying one is reported unsigned.

Programs of vGLP. The linker also links programs of vGLP [56]. A `self.vglp` that carries procedures, in a directory with no `self.glp`, is linked as that di-

rectory's `self.glp`; where the layers are not known, at a boot's hand-over or a single-module load, its layer joins it as an empty module, as it joins a `self.glp`. The root `self.vglp` carries widget declarations, and the linker refuses anything else in it, text it cannot read included. A program emitted from a directory carries a `self.glp` only at or below that directory.

C.2 Agent Execution and Boot

Clause try. Head matching against a clause traverses the arguments, accumulating a tentative writers substitution and the set of blocking readers, position by position as Definition (Term Matching) of the companion paper has it; a mismatch at any position fails the clause, whatever it suspended on before, and a reader is never resolved by an assignment of the same match. An empty set selects the clause; otherwise the set is added to the goal's suspension set, the tentative substitution is discarded, and the next clause is tried — when no clause selects, the goal suspends on the accumulated set if it is nonempty and fails otherwise (Definition 3.24). A guard is tried member by member: a member that suspends adds its readers to the clause's suspension set and the next member is tried, a member that fails fails the clause whatever stood before it, and the clause is selected when every member holds. The tentative substitution reaches the heap only at commit, after the guards hold; a failed or suspended clause leaves no trace.

Event-driven execution. Each agent runs in its own isolate and is event-driven: an event is the initial start, an incoming assignment message (Definition 6.22), the person's input on the UI channel, or a place event from the networking layer (§6.4). On each event the agent binds what the event delivers, reduces until quiescent — FIFO with the b-bounded tail schedule of Definition 3.33 — and then performs its Sends (Definition 6.22); there is no clock and no polling. An exhausted tail budget (Definition 3.33) yields to the host event loop, so timers, I/O, and the UI stay live. Every event that may unblock a goal is followed by such a cycle, and a suspended goal is re-enqueued solely through the activations its binding returns, so no goal is enqueued twice. Agents do not detect or report termination; shutdown is the embedding application's.

Boot. A program's `boot` procedure declares its agents: each body goal `agent(ch(_?,_),ch(_?,_))@`p directs the runtime to spawn an isolate for agent p, create its index-0 serializer entry (Definition 6.10), and post the goal there, every agent's initial goal being the same, as the companion paper's Definition Multiagent GLP has it. The runtime holds the writer of the first channel's reader for the person's input, renders the first channel's output to the person, holds the writer of the second channel's reader as the index-0 serializer entry, and takes the second channel's output as the agent's network output. When the runtime spawns an agent, before anything is compiled or sent, it makes the agent's key pair from a seed drawn from outside the program — the SHA-256 of the clock, the device's hardware identifiers and the persona's name, which is the agent's name p in the boot clause, combined by exclusive or — and keeps it across restarts (Persistence, §C.4), so the key pair is made once, at the

agent's first start. The runtime supplies the key pair to the networking layer whenever it instantiates the layer, which authenticates the agent's messages under it, and certifies under it the modules compiled for the agent (§B.4). A boot clause names the person scripted in GLP at p by a second goal P`@`p, whose one channel argument, written `ch(_?,_)`, is the person's end of p's person channel, its other arguments ground: the runtime runs P there in the person's place, its writes being the person's input. An agent learns the keys of its peers from its person: the ground arguments of P carry the keys the person knows, and the person gives them to the agent as a person makes an acquaintance; a readable name is a variable standing for a key (Remark 4.1). A boot defines no `agent/2`: the program it loads answers the initial goal, in a simulation as on a phone, and the person goal's first act names the entry the program starts. A boot is for a simulation; a deployed agent is spawned by its application with the same initial goal and its key made the same way, the person being the person. At the agent's first start the application sends on the person channel the act that starts its program, with the agent's own key and the keys of the peers the person knows, none at a first start; later peers come by acquaintance. An address the smartphone hands the application, an invitation link the person tapped [4], is the person's act: the application sends on the person channel the act that opens the link, as if the person had entered its text. Inter-agent message routing is the runtime's, by the destination in the message's global name. The `boot` clause is stripped on the boot path, so there the `@` notation never reaches the type checker and what remains of the file is an ordinary module. A directory loaded as a program is not on that path and its boot clause is not stripped, so a program directory carries none. A boot program is placed under the program root, which is therefore its ancestor; placed outside it, the program inherits neither the root's declarations nor the modules the root exposes.

The scope a boot source is checked in. A boot source is loaded on top of a program already in the engine, so it is checked in the scope the engine holds when it is handed over: the linked program's entry points and the boot file's ancestor chain of `self.glp` declarations, the root among them. A boot source is not itself a program: it declares no imports and qualifies nothing, naming the linked program's exports and the root's procedures, since a source carrying a cross-module call is neither a self-contained module nor a directory with a `self.glp` [55]. A call to a procedure the program does not export is refused by the check, which names the call, and does not fail at run time. A posted goal is checked likewise, in the root and the loaded units' entry points, and a callee's clauses in the scope of their layer.

C.3 The `when_idle` Guard

The guard succeeds when the agent's run queue is empty and, in madGLP, its outbox holds no message that can be sent: a sendable message is a Communicate still to make, a held one is not. A goal whose guard finds the machine busy suspends on a reader the scheduler holds, as a goal on `wait/1` does, and whenever the queue empties the scheduler wakes the waiting goals one at a time, in order

of arrival: the guard is decided when the goal is taken, so waking two at once would let the second pass with the first's work queued.

C.4 Networking Seam

One interface. The multiagent runtime reaches the network through one interface, the one the GLP Networking API specifies [4]; what the runtime relies on of it is stated below. A simulation realisation of the same interface backs the tests and plays, so transport integration is a backend swap. Where no layer is bound at all — the REPL, a test run without the simulation — the runtime stands in for a layer that observes nothing and does nothing: `peer_address` and `peer_app` assign `none`, a place declaration records the place and assigns its stream `unobservable`, the stream closed by its removal or a further declaration as ever, the closing being the runtime's and not the layer's, a nearby or reachable declaration stands and its stream carries nothing, and the other seam predicates return having done nothing.

The contract. The runtime relies on this contract: an agent's identity is its public key, and the runtime supplies the key pair to the layer when it instantiates it, the layer keeping it unchanged for its lifetime; communication is point-to-point delivery of opaque payload bytes, the sender authenticated by the layer; delivery is fair — every message sent is eventually delivered (Remark 6.23) — and unordered, and a message is delivered until the runtime acknowledges it (Persistence, below); the layer refuses to send a message it cannot save, and the runtime waits for its answer and, on a refusal, keeps the message and sends it again with the agent's next Sends, or when the layer reports a peer connected; the layer reports peer discovery, connection, disconnection, and reachability; and each proximity underlay has a cold-call trust level, open or closed, set by a trust declaration (Definition 6.9), closed admitting no first contact from an unknown agent over that underlay. What the layer attests of a peer and its session is the API's to specify, the version it attests being the one the runtime gives it, that of the program the agent runs; the runtime records the app the layer reports attested in its session with each peer, for `peer_app`. The layer also observes a peer's address, opens a UDP path to an address, declares a place of a given radius, removes a declared place, sets a proximity underlay's cold-call trust level, takes a peer the program knows, a peer's address and a rendezvous server and removes them, and reports entry to and exit from a declared place together with when it stops and resumes reporting them; these back the six seam predicates (§6.4). The runtime gives the layer a place's name, a constant, as its printed form.

Payloads. A payload is one assignment message in the canonical encoding (§B.2); a cold-call is an assignment to the index-0 serializer, distinguished by its global name, not by a message kind. The runtime signs with the agent's key pair, which it holds, for the signature kernels; the signed content is §B.7.

Global-name indices. Each agent allocates global-name indices from one counter shared by Globalize and Localize, starting at 1 — index 0 is the serializer (Definition 6.10). A removed entry's index is never reused, and a name

carries the epoch it was allocated in (Definition 6.1), so a global name denotes one link for the whole run, across every crash.

Early messages. Delivery is unordered, so an assignment to $_r(p, i)$ may arrive before the message whose localization creates its global writers table entry. The Send transaction applies an assignment only where the entry exists (Definition 6.22); the runtime holds such an early assignment, keyed by its global name, and applies it when localization creates the entry. A value for a reader name whose value the agent has applied already is a repeat, not an early assignment, and is dropped; to tell the two apart the agent keeps the set of reader names it has applied. The record lives for the agent's life: the sender's one counter numbers every name it allocates, and the receiver applies only some of them, so no prefix of the indices need fill and no representation of the set is bounded; bounding the record needs the sender's or the layer's help, and is left open.

Link release. A reader-name value is retained by its sender, pending, until its acknowledgement arrives (§6.5); a request re-addresses the pending value to the reader's current holder. Traffic for a closed link — a late value, request, or acknowledgement — is dropped; indices are never reused, so the drop is failsafe. Traffic for a lost end is answered `gone` (§6.5), and a link end whose other end is known to be in a retired epoch is retired (Definition 6.26): its entry or its record and goal are released as a refused link's are (§6.7), and its messages are dropped.

Held links. A link end whose other end is at an agent the runtime did not exchange the name with is held and reported (§6.7). The test is structural — the anchor named in the global name, or the agent an entry was exported to, against the agent that sent the traffic — and the runtime does not consult the layer's reachability or trust state for it. A held outgoing message stays in M_p marked held; a held incoming assignment is retained keyed by its global name, as an early assignment is. `authorise_link/2` releases it, or drops the link.

Persistence. The agent's local state survives the termination of its application (Definition 6.25). The runtime keeps it in a store the host gives it, beside the layer's queue.

1. *What is saved.* The agent's key pair; the compiled program the agent runs, with the files of its directory, which `load_file/2` reads; the local state s_p (Definition 6.17) — the resolvent with its goal identities and suspension sets, the global writers table with the serializer's set of appended cold calls, the imported-writer records, the outgoing messages with their pending and held marks, the epoch and the known epochs; the global-name counter; the set of applied reader names, and the early and held assignments retained; and every standing declaration — the peers, the addresses, the rendezvous servers, the trust levels, the places, each with the centre it was declared about, a latitude and a longitude in degrees, and the nearby, reachable, public-address and network declarations.
2. *When.* The host saves the state at the suspension of the application (Definition 6.24): on a phone, when the application goes to the background. After the save the agent takes no event until the application resumes. At every start and at every resumption the runtime sets a mark, *running*, in the store, and

the save clears it. The runtime saves at no other time but at a restart after a crash, below. The runtime acknowledges a delivered message to the layer in the cycle that applies it.

3. *Restart.* A restart resumes from the state the store holds. If the running mark is set at a start, the application crashed: the runtime increments the epoch and retires the link ends (Definition 6.26), and saves the state at once, so that a second crash before the next suspension opens a further epoch. The networking layer keeps no declaration across a launch: at a restart, before the first goal is reduced, the runtime declares to the layer again every declaration the saved state holds, a place about the centre saved for it.
4. *Failure.* If a save fails, the store keeps the state saved before it, and the runtime reports the failure. If the runtime cannot read or cannot restore the saved state, as one written under an earlier format of the store, or under a program the store no longer keeps, the state is lost: the runtime reports it and starts the agent afresh, under the key pair the store keeps.

An update. At a restart under a new version of the program the runtime compares, before restoring anything, each goal of the resolvent, queued or suspended, with the new version: the goal's procedure, named by its path-qualified name and arity in the linked program, must be defined there with the same type identity, a parameterised procedure's identity being compared at every opaque instance. A procedure moved to another file is therefore not defined in the new version. Goals of a guest activated by `run/3` are not compared. Where every goal passes, each is restored to the entry of its procedure in the new code, and every reference in the state to the agent's module stands for the new version; otherwise the agent runs the compiled program the store holds, and `load_file/2` reads the files the store holds with it. What the layer keeps of its own across a restart, the queue, is the networking companion paper's.

Simulation realisation. The simulation realisation runs agents as isolates behind the same interface, with a router owning the identifier–key directory, the adjacency relation, and per-pair queues; a readable name stands for a key (Remark 4.1), so a destination, in a global name or in a message, is a key or a readable name standing for one, and the seam resolves both to the one agent; a program runs unchanged in a simulation and over the network. The adjacency relation is total by default, every agent a PAN peer of every other, and the harness cuts and restores it, so disconnection, queuing and redelivery are tested; harness controls partition pairs and defer delivery, exercising fair delivery and order independence. The PAN's cold-call trust level is enforced as the trust declaration sets it: under `closed`, first contact from an agent not declared a peer is not answered, and waits until the receiver declares the sender a peer, opens its PAN trust level, or contacts the sender. The harness counts the work it hands each agent, its start and each message delivered to it, against the agent's reports that it has finished each, sent after the Sends that work made. It starts the run when every agent reports ready, and fails the boot when an agent reports that it cannot start, faults, or its isolate ends before it is ready. It takes the agents as quiescent when every piece of work it handed out is reported finished, and it

waits on these reports and on no clock. Not provided: the WAN underlay and the rendezvous server, so `peer_address/2`, `punch_udp/1` the effect of an address or a rendezvous declaration, and a reachable declaration's stream, which carries nothing, and a public-address declaration's, which carries `none`; and the local network, so a network declaration's stream, which carries `none`; and the platform's geofencing, so a place declaration and its removal; the reported underlay is the PAN throughout. A nearby declaration on the PAN is given `seen` for an adjacent agent the declaring agent has declared a peer, and for any other adjacent agent once the PAN trust levels of both are open, at the moment the second opens; it is given `gone` when the harness cuts the adjacency. The harness sets no trust level at spawn. The router reports each connection and disconnection to the agents, and reports every peer attested with the app the harness names, its developer, name and version, and every peer unattested where it names none. The harness is told the app by its configuration, and the REPL by `:app` followed by the developer, the name and the version, which holds for every boot after it; `:app none` clears it.

C.5 Self-Module

The kernel predicate `self_module`, whose body kernel is `'_self_module'`, returns, as a `Module` constant, the compiled artefact (§B.4) of the app that calls it. An agent ships a running app to a friend by sending this value; the friend adopts it (§B.7) and runs it. On the wire the constant travels as its artefact bytes under the module tag (§B.2), in an assignment message like any value (§6.12).

Every goal points to a module. An initial goal is started on a module by `run(Goal, Type, Module)`, and every goal spawned from it carries a program counter into that module. Whether an agent runs one module or a million, it is the same code. `self_module` is the inverse of `run`: it takes no argument and returns the module its goal points to.

The `Module` constant carries the artefact — the code, and the certificate over its source and compiled identities — so the adopter verifies the certificate, checks its source identity against the offered $h(M)$, and runs the code (§B.6). The constant kind `Module` is the type system's [55]; the kernel predicate and the module-value representation are the runtime's.

C.6 Type Checker Error Text

A term may be of any size [55], so the checker walks a list along its spine, and its error text is bounded by the first fault and not by the term. An ill-typed term's error gives its first failing path and the number of the others. An input-coverage error ([55], Definition "Well-Typed GLP Program", condition 2) gives, for each argument, its first uncovered alternative and the number of the others. A path is written out only when its error is read.

D Development History

This appendix records how the madGLP specification and its Dart implementation co-evolved. The discipline of Section 7 is bidirectional: authority flows from the mathematics to the specification to the code, but implementation runs and specification derivation surface defects, and several such defects proved to be at the mathematical level rather than the code level. Human review, in turn, caught a conceptual error the formalism had absorbed from an earlier draft of the paper. The episodes below are the substantive ones; each names what changed, how it was found, and which layer was at fault.

The implementation arc. The implementation passed through five stages: cGLP, the concurrent source language; dGLP, the single-agent deterministic implementation; maGLP, the abstract multiagent specification; irmaGLP, a first multiagent implementation organised around a binary network transaction; and madGLP, the current implementation, in which a shared variable pair is realised by two local pairs joined by a global link, with forwarding carried by spawned `global_send` goals and a reserved index-0 serializer for cold-calls. The migration from irmaGLP to madGLP, in early 2026, is where most of the corrections below occurred.

Localize stored the wrong half of the variable pair. The Localize operation stored, in the global writers table entry, the wrong half of the local variable pair: for an incoming globalized writer $_w(p, i)$, the entry must store the local writer X_q (so that an arriving assignment can flow to its reader X_q? in q's resolvent), but it stored the reader instead. The error was found by tracing a complete communication scenario during specification derivation: p assigns $X := T$, the message reaches q, and q cannot find the local variable to assign. The correction touched the Localize and Receive definitions, the madGLP Reduce transition, and the Globalize–Localize correspondence lemma. The fault was at the specification level and was fixed there.

Globalizing at production time — a layering decision later reversed. At one point the abstract maGLP semantics was revised to carry an outbox: each agent's local state became a pair (G_p, O_p), Reduce queued each reader assignment into the outbox at production time, and Communicate was reduced to a passive sender (which merely removes the delivered message from its outbox) and an active receiver. The apparent purpose was to align the abstract semantics with the implementation, where outgoing communication is already produced at reduction time by `global_send` goals; the change carried no recorded rationale beyond this reading. The change was subsequently reversed at the abstract level: the outbox was judged too concrete for the abstract semantics, which was restored to the simpler form in which a local state is an asynchronous resolvent (G_p, σ_p). The production-time, passive-sender treatment was retained one layer down, in madGLP, where it belongs — and it survives in the `global_send` mechanism of Section 7, whose body predicate queues the outgoing message for delivery. The episode is an instance of the layering discipline deciding what belongs at which level, rather than a defect.

Eliminating variable migration. The earlier irmaGLP design tracked, per agent, which variables had their paired counterpart on another agent, and relayed readers leaving an agent's scope through fresh relay pairs with forwarding goals. An audit of the implementation against the specification during the irmaGLP-to-madGLP migration exposed edge cases in which the routing was missed or misordered. The design was replaced wholesale by the push-based model now in the paper: a shared pair is realised by two local pairs joined by a global link, and every variable remains in its originating agent's resolvent, with no migration. The defect surfaced in the implementation; the resolution was a redesign reflected in both specification and code.

Unifying cold-calls with regular communication. A cold-call — a message between agents that share no variable yet — was first handled by a dedicated binary network transaction. That mechanism was removed once as out of scope, then reintroduced, the separate path recurring because it duplicated machinery that regular communication already had. It was finally unified: cold-calls now use the same `global_send` mechanism as established links, addressed to a reserved index-0 serializer, making a cold call a Send like any other and aligning the implementation with the source-language paper. The redundancy surfaced in the code; the unification was carried through specification and paper together.

Removing the false "callback" asymmetry. An earlier draft of the paper described the case of exporting a reader as a "callback," a term that then propagated faithfully into the specification and the code comments. Human review caught that the term encoded a conceptual error, not merely an infelicity: replacing the word alone would have left the misunderstanding intact. Exporting a writer and exporting a reader are the same symmetric operation — an agent holds a local pair and spawns a `global_send` goal to send its reader's value outward once known, with a table entry at the receiving end. "Callback" wrongly implied that one direction was primary and the other a response. The term was removed throughout the paper, the appendix traces, and the implementation, and the symmetric account of global links put in its place. The fault originated in the paper and was corrected there first.

Simplifying the global writers table. The variable table initially stored four kinds of entry, distinguished by origin (Globalize versus Localize) and by variable kind (writer versus reader). Specification derivation made clear that entries are needed only for writers awaiting an incoming assignment, since all outgoing communication is handled uniformly by `global_send` goals. The table was renamed the global writers table and its entry kinds reduced from four to two.

A cold-call polarity error, found by running the code. Running the multiagent implementation exposed a polarity error that was present in the abstract specifications, not in the code: the cold-call transaction transferred the reader $X?$ to the receiving agent q, whereas q must receive the writer X. Because the single-occurrence restriction makes the globalize-reader dataflow always run from the exporting agent to the receiver, the reverse setup was simply wrong. The fix was applied across the madGLP paper, the source-language paper, and

the specification together, with the affected single-occurrence-preservation proof reworked. The diagnostic that exposed it was a runtime trace in which a goal failed where it should have suspended:

```
[MAD agent2] send: globalized term =
  msg(agent3, data(.(got(1), .(got(2), _r(agent2, 1)))))
[MAD agent3] registered global_send goal: _r(agent2, 1) -> agent2
consumer_init(agent3, [msg(agent3, data([got(1), got(2) | X2])) | X3?]) -> failed
```

Agent 3 received a writer where it expected a reader, so its `ground` guard failed definitively instead of suspending, and the goal terminated. This single line — a goal reaching `failed` where the semantics required suspension — drove the polarity correction at the paper level. It is the clearest case of the loop the methodology depends on: the implementation, by being run, falsified the specification.

Diagnosing the fault at the right layer. The same investigation first produced a fix at the code level — a write-back mechanism at the receiver and additional table entries — which was then itself judged wrong. Tracing the dataflow showed that the single-occurrence restriction forces the globalize-reader flow to run from the exporting agent to the receiver, whereas the specification had set up the reverse; the two failing tests were caused by this specification-level defect, not by the implementation. The provisional code-level fix was removed and the correction made in the specification and paper instead. This is the discipline's central caution in practice: a defect that first presents as an implementation bug is not patched around in the code once it is traced to the specification.

A missing reactivation hook. A related implementation bug blocked all inter-agent communication using a response variable: the spawned `global_send` goal was never registered to fire when its watched reader became known, so a cold-call expecting a reply waited forever. This was a genuine code-level defect — the specification was correct — found by running a concrete request-and-response scenario. It is recorded here as the converse case: not every defect surfaced by running the code is a paper-level fault, and distinguishing the two is the point of keeping all three layers aligned.

A suspension bug that clarified the specification. In the core single-agent engine, the soft-fail step that advances a goal to its next candidate clause discarded the clause's suspension set instead of merging it into the goal's, so goals failed where they should have suspended. Fixing the code exposed that the specification had not stated the merge invariant explicitly; the invariant, together with the three-valued reading of guards, was then added to the runtime specification. Here a runtime bug drove a clarification of the specification rather than a change to the mathematics.

From relayed links to anchored request/acknowledge transfer. The relay design kept every past holder of a reader in the loop: a holder that passed a reader on chained a fresh pair to it, so a stream reaching a re-exported reader crossed every past holder forever. The 2026-07 redesign fixes a link's global name — its *anchor* — for life: an unbound imported reader is forwarded under its original name, the new holder requests the value from the anchor, a sent value stays

pending at the anchor until the current holder acknowledges, and stale traffic is dropped on never-reused indices. A value now crosses once, to the current holder; past holders drop out, and a redirected stream continues to the new holder directly. This partly reverses the "eliminating variable migration" episode above: the reader occurrence does move between agents, but against a fixed anchor and under the request/acknowledge discipline, so the routing races that sank irmaGLP's migration do not arise. Epochs on readers were considered and dropped: the acknowledgement alone closes a link, and a request from a holder the anchor did not itself export to is what reports the move.

An internal trace inconsistency. Reserving index 0 for the cold-call serializer left the worked traces and the figure still using index 0 for ordinary global links — an inconsistency internal to the mathematical layer, introduced by the unification above. The traces, figure, and appendix proof were shifted to start at index 1.